

ULTM8 Platform

Technical Specification & System Architecture

Derived from the connected Figma design files and the confirmed technical stack

Prepared for: Code Studio

Date: 21 August 2026 (grading/ranks section updated 24 August 2026)

Source files: "ULTM8 - New Versions (1)" & "Application (1)" (Figma)

1. Executive Summary

This document is a technical specification produced by auditing two connected Figma files end-to-end and mapping every screen, flow, and field found in them onto the confirmed engineering stack. No feature, role, or business rule has been assumed beyond what the designs literally show or what was explicitly confirmed by Code Studio during this session (see the locked decisions below). Anything the designs leave ambiguous, inconsistent, or undesigned is called out explicitly in Section 12 (Gaps & Open Questions) rather than filled in with a guess.

1.1 What the Figma project actually contains

- File 1 — "ULTM8 - New Versions (1)": a single Figma page named "School Portal" containing the entire web back-office experience: authentication, school/franchise profile management, franchise & branch administration, payment-gateway configuration, instructors, weekly timetables, classes, membership plans, transaction/ledger history, a screen-level translation (i18n) CMS, account settings, and a KPI dashboard. It also contains an extensive belt/stripe icon library (448 unique frames total, of which roughly 360 are belt-rank icon assets rather than screens).
- File 2 — "Application (1)": a single Figma page named "Working" containing the Student mobile app: onboarding, authentication, academy/school discovery, class booking, membership purchase, card payments, transaction history, liability waivers with e-signature, QR check-in, belt/rank progression, coach profiles, notifications, search/filters, and account settings (94 unique frames).
- Two of these three planned applications (School Portal web, Student mobile) are directly represented in the Figma files. "Platform Admin" is not a separate file or section in the designs at all — see decision 1.2 and Section 4 for how it is scoped as its own application going forward.
- The connected component libraries on both files are generic community kits (Material 3, Simple Design System, and several Apple OS kits) — there is no bespoke design-token or component library specific to this product. Screens are built from ad-hoc components, not a shared, named design system.

1.2 Decisions locked in for this specification

The Figma designs conflict with the confirmed stack and contain structural ambiguities in several places that materially change the architecture. These were resolved directly with Code Studio before writing the rest of this document; the App Boundary decision below was subsequently revised after a follow-up architecture discussion (21 Aug 2026) — the version below is current, and Section 4 carries the full reasoning and the isolation architecture it commits us to.

- **Payment scope** — Stripe (cards) is the only live payment gateway to be built. Cash and Bank Transfer are supported as manually-recorded, non-gateway payment methods (staff mark them Pending/Successful by hand — corrected 26 Aug 2026, Pass 5; the real Transaction status enum has no separate "Paid" value, Section 6.1). GoCardless and PayPal, both present in the Figma payment-method screens, are out of scope for this build.
- **App boundary (revised)** — School Portal and Platform Admin are TWO separately deployed React applications from day one — not one RBAC-gated app, and not a later split either. They share a common component library, typed API client, and auth utilities via a monorepo. Platform Admin gets its own subdomain, its own identity realm, and its own release pipeline. Full rationale and architecture in Section 4.
- **Canonical design version** — Where a feature exists twice in File 1 with different fidelity, the newer, dashboard/KPI-driven, card-based screen set is treated as the source of truth (e.g. the "Dashboard" frame with Attendance/Membership/Training-Time/Booking KPI cards), not the older plain list/table CRUD screens.
- **Tenancy hierarchy** — Franchise → Schools → Branches. A Franchise account can own multiple Schools, and each School can have multiple physical Branches. Per Code Studio's follow-up note, independent Schools (not affiliated with any Franchise) are also a first-class case — a School's franchise relationship is optional, not mandatory.

- **Payment-credential custody (new)** — Rather than encrypting each tenant's raw Stripe secret key at rest, ULTM8 moves toward Stripe Connect so tenant secret keys are never held by the platform at all, with a dedicated secrets manager as the fallback where Connect isn't viable for a given flow. Full rationale in Section 10.4.
- **White-label mobile app distribution (new)** — Every School is included in a single shared "ULTM8" Student app by default; a paid subscription upgrade adds a School's own branded app on the Apple App Store and Google Play, built and published on the School's behalf by ULTM8. The publishing pipeline is built alongside the initial platform, not deferred, so it is one of the features Schools are told about from launch. Full rationale and architecture in Section 5.

1.3 Domain summary

This is a multi-tenant SaaS platform for martial-arts school management (branding: "ULTM8", built by "IMAS – Innovative Martial Arts Systems" per sample data in the designs). Core domain concepts confirmed by the designs: belt/rank progression per discipline (Jiu Jitsu, Karate, Judo, Krav Maga, Taekwondo, MMA, Wrestling, etc.), class scheduling against a weekly recurring timetable, membership sales (subscription / single-pass / trial), liability waivers with e-signature, QR-code class check-in, and multi-language (4 languages) / multi-currency (6 currencies) support administered through a CMS-style translation tool.

The commercial loop end-to-end (confirmed, 25 Aug 2026): a Student's Membership must be Active for them to book a Class; Classes are linked to the curriculum/skills the School teaches (see Skill/Lesson, Section 6.1 and Section 12.2); a Booking's QR-attendance completion is what feeds grading progress (StudentRank.classesAttended, Section 2.3/Section 6.1); and the loop repeats monthly — MembershipPlan's recurring Stripe subscription (Section 10.2) charges the Student each billing cycle, and continued access to book Classes depends on that renewal succeeding.

1.4 How to read this document

This document mixes three different kinds of content, and a returning reader or a developer picking it up for the first time needs to know which is which before treating anything in it as an instruction to build against.

- **Sections 1–11 — current-state canonical reference:** build against these. Every fact here already reflects every decision made through 27 Aug 2026 — Section 12.1's entries are folded directly into the data model (Section 6), the API (Section 7), the job list (Section 9), payments (Section 10), and the NFRs (Section 11) as they're resolved, not left to live only in decision-log prose. If Sections 1–11 and Section 12.1 ever appear to disagree on a resolved point, that's a drift bug worth flagging — not a case where the decision log overrides the canonical section.
- **Section 12.1 (Decision log) — the audit trail, not a second source of truth:** read this to understand why a design decision was made and what alternatives were rejected, or to trace a specific decision number cited elsewhere in the document. Don't derive new behavior from it that isn't already reflected in Sections 1–11 — if it's not in the canonical section, treat it as unresolved and check Section 12.2 first.
- **Section 12.2 (Open items) — explicitly not resolved:** everything here is a known gap, an unconfirmed assumption, or a question still pending product or legal input. Nothing in Section 12.2 should be built against as if it were decided; several items name a specific person or team whose confirmation is still needed before implementation starts. Treat a Section 12.2 item the same way you'd treat a JIRA ticket still in “needs decision” status.

The subsystem index below maps each functional area to the sections and decision numbers that govern it, so a developer starting work on one part of the platform can jump straight to what's relevant instead of reading the document end to end. A Glossary of entities, status enums, and platform terms follows as Section 14, at the end of the document.

Subsystem index

- **Identity, auth & roles** — Section 4.4 (Platform Admin identity realm), Section 6.1 (User, RoleGrant), Section 8 (screen-to-role mapping). Key decisions: 12, 24, 28, 43.
- **Multi-tenancy & RLS** — Section 4.3 (tenant isolation), Section 6.2 (tenancy hierarchy), Section 11.6 (composite-index convention), Section 4.8 (CI gate). Key decisions: 16, 30.
- **Payments, billing & subscriptions** — Section 10 (Stripe integration), Section 6.1 (PaymentAccount, Transaction, FranchiseFeeCharge, PlatformCharge). Key decisions: 35, 38, 42, 54, 55, 68.
- **Classes, booking & waitlist** — Section 7 (ClassesModule), Section 6.1 (Class, Booking). Key decisions: 59, 66.
- **Grading & ranks** — Section 6.1 (StudentRank, PromotionEvent), Section 7 (RanksModule), Section 2.3 (working prototype). Key decision: 58.
- **Attendance & check-in** — Section 7 (AttendanceModule, InstructorsModule), Section 6.1 (ConsentRecord). Key decisions: 57, 66, 69.
- **Compliance, privacy & audit** — Section 4.7 (AuditLogEntry), Section 6.1 (ConsentRecord), Section 12.2 (open legal-review items). Key decisions: 43, 44, 57, 69.
- **Franchise & white-label** — Section 5 (white-label mobile app distribution), Section 6.1 (TenantAppConfig, FranchiseFeeCharge). Key decision: 42.
- **Background jobs & observability** — Section 9 (job table), Section 11.6 (environments & delivery). Key decisions: 23, 31.
- **Platform Admin & internal tooling** — Section 4.4 (identity realm), Section 4.7 (audit logging). Key decision: 43.

2. Figma Audit — What Was Found

This section is a direct inventory of the two connected files. Every screen name below is taken verbatim from the Figma layer names; nothing is inferred.

2.1 File 1 — School Portal (Web)

URL: figma.com/design/txowXWQFNw5LdeCN2umyDb — Page: "School Portal" — 541 top-level frames, 448 with unique names. ~90 are real application screens; the remainder are belt/stripe rank icon assets (see 2.3).

Feature area	Key screens found	Notes
Authentication	loginScreen, registerScreen, forgotPassword, verificationPin, restPassword	Email + 6-digit passcode, social login (Microsoft ×2, Apple, Facebook, Google), phone-based OTP
School / franchise profile	createSchoolProfile, updateSchoolProfile, SchoolInformation, franchiseCreate, franchiseUpdate, franchiseList, franchiseInformation	Business name, mobile, address (autocomplete), business type, activities, facilities, ranks toggle, default language/currency, description
Franchise commerce	franchiseSubscriptionPlans (Pay Now / Subscribe)	Franchise/school pays into a platform plan; resolved as UTM8's own platform SubscriptionPlan billing — see Section 10.2
Payment gateway config	stripePaymentInfo, gocardlessPaymentInfo, bankAccountPaymentInfo, addPaymentMethod, franchiseInformation (Payment Information table)	Each school/franchise can store its OWN live credentials for 5 methods: Stripe, GoCardless, Bank Account, PayPal, Cash
Instructors	instructorList, instructorCreate, instructorUpdate, instructorInformation	Name, photo, belt/ranking, specializations, phone, years of experience, bio
Timetable	timeTableList, createTimeTable, updateTimeTable, timeTableInformation	Weekly recurring grid (Mon–Sun), multiple slots/day, start/end time, breaks, on/off status
Classes	classesList, createClasses, updateClasses	Title, activities, banner image, description, start/end date, capacity, booking window, QR-attendance window, refund-fee date, cancellation charge, terms/waiver toggle
Membership plans	membershipList, createMembership, MembershipInvoice	Title, type (Subscription / Class Pack — renamed from "Single Pass", Section 6.1 / Weekly Pass / Friend Pass / Trial Membership), price, expiry, visibility, refund-fee date, cancellation charge, terms/waiver toggle
Transactions / ledger	transactionsHistory	Per-gateway account balances (GoCardless / Cash / Stripe: Pending & Available), invoice list with status + downloadable invoice
Translations (i18n CMS)	translationsList, translationsCreate, translationsUpdate, translationsDetail	Label-key → content-string management per screen and locale; hints at Student/School/Admin/User

Feature area	Key screens found	Notes
		content grouping — see Section 12
Account settings	Account settings, updateName/Email/Phone/DateOfBirth/Gender/Nationality/Language/Currency/Address/Accessible, Security, Preferences, Email notifications, deleteAccount	Granular single-field update screens
Dashboard (canonical)	Dashboard (id 40000023:3474)	KPI cards: Attendance (Present/Absent), Membership (Active/Expired), Training Time (Weekly/Monthly hrs), Booking (Total/Active); latest-transactions table showing Stripe/GoCardless/Cash; "Explore Classes" carousel; embedded member profile view (belt/kimono size, weight category, ranks)
Static / legal	Help & Support, Terms and Conditions, Privacy Statement	Content screens, no CMS fields beyond static text observed

Sidebar navigation (present on nearly every School Portal screen): Dashboard, Logout, Setting, QR Code, Booking (Time Table), Classes, School, Branches, Franchises, Instructor, Membership, Rules, Notification, Transactions History, Subscription History, Reports, Help & Support, Customer Services, Currency, Language. Of these, QR Code, Notification, Rules, Subscription History, Reports, and Customer Services have a nav entry but no corresponding designed screen — see Section 12.

2.2 File 2 — Student Mobile App

URL: figma.com/design/6ewBjbt2FJoMmFbloXXUzE — Page: "Working" — 105 top-level frames, 94 with unique names.

Feature area	Key screens found	Notes
Onboarding	splash, gender, birthday, bodyType, activitiesGoals	Post-registration "personalize your plan" wizard: gender, DOB, height/weight, preferred disciplines
Authentication	loginScreen, registerScreen, forgotPassword, resetPassword, verificationPin	Same email + 6-digit passcode + social login + SMS OTP pattern as web
Discovery	academies, academiesDetails, Academy Details, academyDetails, mapScreen, listScreen, Search + filters (activityFillter, distnccFillter, timeFilter)	Browse Schools / Franchise / Branches tabs; per-academy contact info, activities, weekly timetable, membership plans
Booking	Booking, classBooking, classBookingConfrim_Message, classesDetails	Upcoming / Completed / Cancelled tabs; confirm booking shows instructor, date, time, class name
Membership	membership, Membership, membershipDetails, Membership Detial	Active / Pending / Expire states; purchase flow (Subscription / One-Time / Trial)
Payments	paymentMethod, paymentInformation, paymentSuccessful	Card list with set-default; add-card form (holder, number, MM/YY, CVV, country, zip) — Stripe-Elements-style
Transactions	transactions	Invoice list labelled by gateway, types Subscription / One-Time / Trial Period

Feature area	Key screens found	Notes
Waivers	waiver, waiverDetail, waiverSchool	E-signature (full name + signature), per-school status: Signed / Unsigned / Expired / Pending
Check-in	qrCode	Camera-based QR scan for class attendance
Ranking	rankingScreen, Profile_Ranks	Per-discipline belt rank, last/next grading date, class-count readiness (Ready / Almost Ready / Not Ready), progress %
Coach / instructor view	Coach Profile	Bio, stats, upcoming classes, Book button (no separate instructor login found in the Figma designs; resolved — Instructor now has its own login, Section 8.2)
Notifications	Notification, All Notification	List of alerts, mostly waiver-signature requests in the sample data
Profile hub	Profile_Booking, Profile_Class, Profile_Gallery, Profile_Membership, Profile_Ranks, Profile_School	Tabbed hub; unlabeled stat numbers (870 / 120k / 354k / 3.8Gb) — meaning unclear, see Section 12
Settings	setting, currency, language, helpAndSupport, termsAndConditions, reportBug, faqs	Profile / Regional (language+currency) / Help & Support / Laws & Regulations (9 sub-items) / Permissions (7 types) / Logout
Content / misc	offerScreen, videoScreen, dealsDeals, summaryAndCart, sideMenu	Minimal field detail in the designs — see Section 12

Bottom navigation is inconsistent across the file: most screens show HOME / RANKING / SCHEDULE / SEARCH / NOTIFICATION / PROFILE, but Coach Profile and video-adjacent screens show HOME / VIDEO / SEARCH / OFFERS / PROFILE. See Section 12.

2.3 Belt / rank icon library

File 1 contains roughly 360 individual icon frames enumerating every belt colour and stripe-count combination used by the grading system (e.g. "Blue Belt 2 Stripes", "Grey & Black Belt 3 Yellow Stripes", "Green & White Belt 1 Yellow Stripes"). These are visual assets, not screens, but they confirm that belt grading is a first-class, highly-detailed domain concept: rank = belt colour + colour combination (for some disciplines, two-colour belts) + stripe count + stripe colour, per discipline, per student. This directly informs the Ranks/Grading data model — see the working reference implementation below, which expands Section 6.1 and Section 7.

Working reference implementation (new)

This subsection is new, added after the original audit — in the same spirit as Section 4 and Section 5, it is engineering validation Code Studio commissioned on UTM8's advice, not something drawn in Figma. Rather than inferring the grading data model from static belt/stripe icon names alone, a working, interactive prototype of the grading system was built and iteratively tested (automated regression pass, zero console errors, coach-facing screens reviewed against the exact rank ladder, stripe tiers and skill gates below). It validates real coach workflows the icon library by itself cannot show, and materially expands the Belt/Rank and StudentRank entities in Section 6.1 and the RanksModule endpoint list in Section 7.

- **Rank ladder & grading requirements** — an ordered ladder of ranks per style/discipline; each rank has a primary belt colour and an optional secondary colour (two-colour belts); stripes are modelled as an ordered

list of tiers, each with its own count and colour — not the single flat “stripe count” field originally inferred from the icon library, which cannot represent a belt that changes stripe colour partway through. Each rank also carries its own classes-required, minimum-days-in-rank, weekly-class cap, and which class types count toward it.

- **Skill-gating** — a rank can additionally require specific named skills to be signed off (Not Started / Learning / Signed Off), as a gate that sits alongside — not instead of — the class-count and time-in-rank requirements.
- **Years-in-rank gating for Black Belt and above** — once a student reaches Black Belt, class-count and day-count requirements stop applying entirely; a minimum tenure in years is the sole gate instead. This is a structural difference in how a rank is evaluated, not just different numbers, and needed its own explicit flag on the rank entity.
- **Promotion / downgrade / bulk grading** — a single student can be promoted or downgraded, with every change written to an auditable rank history (from-rank, to-rank, date, graded by). Multiple students can also be graded in one pass, each judged individually rather than as a single batch action. Grading is permitted even when a required skill isn't yet signed off, but only behind an explicit written acknowledgement — the system warns, it doesn't block.
- **Grading Board** — a coach-facing view buckets every student, per style, into Just Starting / Getting There / Ready to Grade from their live progress toward the next rank, with a manual drag-and-drop override for the edge cases an automatic bucket gets wrong.
- **Curriculum linkage (candidate, see Section 12.2)** — short instructional lessons (video + text) attach to the specific skill they teach, so relevant instruction surfaces automatically at the point a coach or student needs it — the skill library, a student's own profile, the grading flow's own skill-gate warning — instead of living in a separate, self-serve video library. This is offered as a plausible answer to the undefined mobile videoScreen (Section 12.2), not a confirmed decision.
- **What this prototype is for (confirmed, 25 Aug 2026):** it validates the grading/ranks data model and interaction flow — stripe-as-its-own-rung, per-rung class scope, manual-only promotion, bulk grading, skill sign-off gating — which the production RanksModule and Grading Board should implement as-is rather than redesign from the Figma audit alone. It is NOT source code to port: the prototype is a single hand-rolled HTML/JS file, while the production stack is React (web) and React Native (mobile), Section 4.2 — the real frontend is a fresh build against these same validated flows, not a port of this file.

3. System Architecture

Architecture mapped directly onto the confirmed stack: Node.js/NestJS (TypeScript) backend, PostgreSQL, Redis + BullMQ for background work, React for the web apps, React Native for the Student app, Stripe for card payments. Per the locked decisions in Section 1.2 (App Boundary, as revised — full detail in Section 4), this is a single NestJS backend serving TWO separately deployed React web apps — School Portal and Platform Admin — plus one React Native mobile app (Student), with Stripe (moving toward Connect, Section 10.4) plus manually-recorded Cash/Bank-Transfer payments.

3.1 High-level component map

Layer	Component	Responsibility
Clients	React — School Portal	Tenant-facing web app: franchise/school/branch management, classes, memberships, instructors, timetable, transactions. Own deployment (Section 4.2).
Clients	React — Platform Admin	Internal-only web app: translations CMS, platform subscription plans, cross-tenant management. Own subdomain, own identity realm, own deployment (Section 4).
Clients	React Native app	Student mobile app: discovery, booking, payments, waivers, QR check-in, ranking. Distributed as a shared UTM8-branded app AND, per School, as an optional white-label rebrand on both app stores (Section 5) — one React Native codebase, one App/Play Store listing per active brand, not one app per School by default.
API	NestJS HTTP API	Modular REST API (module breakdown in Section 7); DTO validation via class-validator; Swagger/OpenAPI for the contract; admin-scoped endpoints namespaced and guarded separately from tenant endpoints (Section 4)
API	Auth layer	JWT access + refresh tokens for customers; a separate identity realm for Platform Admin staff (SSO + mandatory 2FA, Section 4.4); Passport strategies for email/passcode and social OAuth (Microsoft, Apple, Facebook, Google); role guards
Data	PostgreSQL	System of record with Row-Level Security enforcing tenant isolation at the database (Section 4.3) — see data model, Section 6
Data	Redis	BullMQ queue backing store, response/translation caching, rate-limiting store for OTP endpoints, revocable refresh-token session store (Section 8.3)
Async	BullMQ workers	Background jobs — OTP/SMS delivery, Stripe webhook processing, invoice generation, notification fan-out, membership-expiry reminders, QR-attendance processing, translation cache invalidation, waiver-signature requests, class-occurrence generation, class-cancellation processing, booking-no-show expiry, waitlist-promotion processing, dashboard-KPI aggregation, Apple developer-membership-renewal reminders, tenant-app-credential grace-period purge, franchise-fee usage reporting, account deletion processing (18 jobs total — see Section 9; updated Pass 8 Round 3 to add the new chargeback-pattern-restriction job, Decision 68; previously updated Pass 8 to add the account-deletion-processing job, Decision 44; and 25 Aug 2026, Pass 4, to add the franchise-fee usage-reporting job from Section 10.2)

Layer	Component	Responsibility
Payments	Stripe (+ Connect)	Card payment methods, one-time charges, subscriptions, webhooks (per locked decision, Section 1.2 and Section 10); moving to Stripe Connect so tenant secret keys are never custodied by the platform (Section 10.4)
Storage	Object storage / CDN — Cloudflare R2 + Cloudflare Images	Banner images, gallery photos, profile photos, signed-waiver PDFs, invoice PDFs — resolved 26 Aug 2026, Pass 5: Cloudflare R2 (storage) + Cloudflare Images (transforms), Section 11.3, Section 12.1
Notifications	Twilio Verify (SMS/OTP) + FCM/APNs (push) + Postmark (email)	Resolved 26 Aug 2026, Pass 5 — Section 11.4, Section 12.1
Audit	Audit log (proposed)	Immutable record of every Platform Admin action against tenant data — see Section 4.7
Build & Release	White-label build/publish pipeline (new, 25 Aug 2026)	Per-tenant native builds, code-signing, and app-store submission automation for the branded-app model (Section 5) — doesn't fit inside the NestJS apps/api module boundary (Section 4.2)

3.2 Request flow examples (derived from the designs, not assumed)

Student books and pays for a class

- React Native app → GET class list & availability (from Timetable/Classes screens) → student selects a class → classBooking confirmation screen (Instructor, Class Timings, Class Date, Class Name) → POST booking.
- If the class/membership requires payment: paymentMethod screen either reuses a saved card (Stripe PaymentMethod) or collects a new one (paymentInformation form) → NestJS creates a Stripe PaymentIntent/subscription → BullMQ worker processes the resulting webhook asynchronously → booking + transaction records are finalized, and the Membership itself is created Active at this same step (Section 6.1's Membership-creation-timing model, Pass 5) → paymentSuccessful screen.

School staff configure a payment method

- School Portal → addPaymentMethod / stripePaymentInfo screens start a Stripe Connect onboarding flow rather than collecting a raw secret key directly → the resulting Connected Account id is stored against the School/Franchise's PaymentAccount record (secrets-manager fallback for any credential that still requires one — see Section 10.4) → reflected immediately in franchiseInformation's Payment Information table and in transactionsHistory's per-gateway balances.

OTP verification

- registerScreen / forgotPassword submit a phone number → NestJS enqueues an SMS-OTP BullMQ job → Twilio Verify (resolved 26 Aug 2026, Pass 5, Section 11.4) delivers the code → verificationPin screen submits the 4-digit code for verification within its expiry window.

Platform Admin investigates and rotates a tenant's payment credential

- Platform Admin authenticates via its own identity realm (SSO + mandatory 2FA, Section 4.4) into the apps/platform-admin app (Section 4.2) → a Billing/Payments Ops-tier engineer views PaymentAccount configuration status for the affected School through NestJS's PaymentsModule, scoped by the same franchise_id/school_id RLS policy as any tenant query (Section 4.3) → the engineer never receives a decrypted secret (Section 10.4).
- To rotate the credential, the engineer initiates a new Stripe Connect onboarding/rotation flow (Section 10.4) rather than handling a raw key directly → the action is written to the immutable Platform Admin audit log —

who, when, what, on which tenant (Section 4.7) → if the situation instead requires genuine ad-hoc database access, the break-glass path applies: a short-lived credential from the secrets manager, a second engineer's approval, an automatic post to a monitored channel, and independent pgAudit logging at the database level (Section 4.3).

4. Platform Admin Isolation Architecture

This section supersedes the original "App boundary" call in Section 1.2. The Figma designs show no separation at all between day-to-day school management and franchise/platform-level administration — one flat sidebar covers both. The initial recommendation was to keep that as a single, RBAC-gated React application, since it matched the designs and was the fastest path to shipping. On review, Code Studio asked for the best long-term approach rather than the quickest one, particularly given that Platform Admin will have visibility into every tenant's data, including live payment credentials (Section 10). This section replaces that recommendation with a design that removes the risk by construction rather than by discipline.

4.1 Why a shared, role-gated app isn't the best answer here

A single application with client-side role checks has three compounding weaknesses once real tenant payment data is involved: the only thing separating a school user from platform-wide data is a permission check that lives in the same bundle they can inspect; a change to admin-only tooling (reporting, translations CMS, tenant management) forces a redeploy and re-test of the exact app school staff depend on daily; and admin-only code ships to every school user's browser unless code-splitting is followed with discipline on every PR. None of these are hypothetical for ULTM8 specifically — the designs already show each school able to enter live Stripe secret keys, so a Platform Admin surface here is a surface with direct access to payment credentials across every tenant on the platform.

4.2 Frontend: two applications, one monorepo

School Portal and Platform Admin are built and deployed as two independent React applications, inside a single monorepo (Turborepo or Nx) so they can still share code without sharing a runtime, a bundle, or a deployment.

Package	Contents	Consumed by
apps/school-portal	Franchise, School & Branch management, classes, memberships, instructors, timetable, transactions — the tenant-facing surface	School owners, franchise owners, instructors (Instructor role confirmed, Section 8.2)
apps/platform-admin	Cross-tenant oversight: translations CMS, platform subscription plans, tenant management, Reports/Customer Services once designed (Section 12)	ULTM8 platform/ops staff only
apps/student	React Native Student app	Students
apps/api	NestJS backend — all modules from Section 7, shared by every client above	All three apps
packages/ui	Shared design-system components (buttons, tables, forms, tokens)	school-portal, platform-admin
packages/api-client	Typed SDK generated from the NestJS OpenAPI spec — one source of truth for request/response shapes	All three frontend apps
packages/auth	Shared token-handling utilities (not shared identity — see Section 4.4)	All three frontend apps
packages/build-pipeline	Native build tooling, long-running async jobs, and per-tenant signing-secret management for the white-label pipeline (Section 5) — doesn't fit as a NestJS module inside apps/api (new, 25 Aug 2026)	Internal build/release tooling only

This costs a second CI/CD pipeline and some monorepo tooling overhead up front. In exchange, Platform Admin code never reaches a school user's browser, admin releases never force a school-portal re-test, and there is no future "lift

the admin views out into their own app" migration to get wrong — the seam is the deployment boundary from day one, not a promise to build one later.

4.3 Tenant isolation enforced at the database, not just in application code

Rather than trusting every NestJS service to remember a `WHERE franchise_id = ...` clause, PostgreSQL Row-Level Security (RLS) policies key every tenant-scoped table on `franchise_id` / `school_id`, and additionally on `branch_id` for `TimetableSlot`, `Class`, `Instructor`, and `Booking` — the entities genuinely tied to one physical location, with `Booking` gaining a denormalized `branch_id` specifically to support Branch Staff RLS (confirmed 25 Aug 2026, extended Pass 2; `Student` and `Membership` stay School-scoped, since a `Student` may train across multiple Branches under one `Membership`). A query missing a tenant filter returns nothing, rather than silently returning another tenant's rows — the database enforces the boundary, not just the application layer. School owner/manager roles get an explicit "sees all Branches under this School" grant, the same pattern Platform Admin's own bypass uses one level up. Platform Admin's cross-tenant access is never a raw database credential handed to a human (confirmed 25 Aug 2026): routine cross-tenant work goes entirely through the API, where it lands in Section 4.7's audit log like any other action. A rare break-glass path covers genuine ad-hoc SQL needs — a short-lived, auto-expiring credential issued through the secrets manager (Section 4.5), requiring a second engineer's approval and posting automatically to a monitored channel the moment it's checked out, with that specific role's activity captured independently at the database level via `pgAudit` (or equivalent) — closing the gap where direct SQL access would otherwise be invisible to both the audit log and Section 4.8's CI gate, which only ever sees traffic through the application. That credential is deliberately provisioned as a non-superuser, non-table-owner Postgres role, so the DB-level restrictions protecting the audit table itself (Section 11.5) can't be bypassed even during emergency access.

4.4 Identity: Platform Admin is a separate realm, not a separate role

Internal platform staff authenticate against a distinct identity realm or audience (a separate Auth0/Keycloak/Cognito pool, or at minimum a separate JWT issuer and audience) rather than a ``role: admin`` column on the same user table used by customers. This realm carries policies a customer login is never subject to: SSO against Code Studio's own identity provider, and mandatory 2FA. Within that realm, Platform Admin is split into three least-privilege sub-roles (confirmed 25 Aug 2026), not one all-powerful Admin bucket: `Support` (read-only) — account-level status metadata across tenants only (name, subscription plan, payment status) for triage, plus a time-boxed, audited, tenant-scoped impersonation session (Section 4.7) to actually help with a specific issue; never a Stripe Connected Account id, a secrets-manager reference, or any tenant's data outside an active impersonation session. `Billing/Payments Ops` — a separate, narrower specialist tier that can view `PaymentAccount` configuration status and initiate a Stripe Connect credential rotation, but never sees a decrypted secret (Section 10.4). `Full Platform Admin` — the only tier that can assign sub-roles to other staff and invoke the break-glass bypass procedure (Section 4.3). Access for any tier is revoked immediately on offboarding, not just blocked on future logins.

4.5 Payment-credential custody

Directly related to isolating Platform Admin: rather than Platform Admin (or anyone) having access to decrypted per-tenant Stripe secret keys, ULTM8 moves toward Stripe Connect so the platform never custodies a tenant's raw secret key at all — only a Connected Account id. Where Connect isn't viable for a specific flow, credentials fall back to a dedicated secrets manager — AWS Secrets Manager specifically, resolved 26 Aug 2026, Pass 5, Section 11.6 — rather than an application-level encrypted column. Current AWS pricing is \$0.40 per secret/month plus \$0.05 per 10,000 API calls — cheap per secret, but a real cost risk if application code fetches a secret on every request rather than caching it in memory/process, since that pattern scales with request volume, not tenant count. Caching with a bounded TTL and rotation-aware invalidation is the standard mitigation, not new engineering, but worth stating explicitly rather than assuming it's obvious. Full detail and rationale is in Section 10.4; it's listed here because it is a direct input into what Platform Admin can see.

4.6 Network & deployment isolation

- Platform Admin is served from its own subdomain (e.g. admin.ulttm8.app), never the same origin as School Portal or the marketing/customer-facing surface.
- Its own WAF rules, and the option of an IP allowlist or VPN requirement — controls that cannot be cleanly applied to a domain paying customers also log into.
- Separate environment promotion (dev/staging/prod) tracked independently of the customer-facing apps' release cadence.

4.7 Audit trail

Every Platform Admin action that touches tenant data — viewing or editing another tenant's records, viewing or rotating a payment credential, impersonating a tenant user for support — is written to an immutable audit log: who, when, what, on which tenant. This is what lets Code Studio demonstrate, rather than merely assert, that internal staff access is bounded — table stakes if ULTM8 pursues SOC 2 later, which is likely given it holds payment credentials.

4.8 CI gate: cross-tenant isolation tests

An automated test suite — run on every pull request, required to merge — asserts that a token scoped to Tenant A can never read or write Tenant B's resources, across every endpoint. This is what actually prevents a regression six months from now, when someone adds a new endpoint under time pressure and forgets the tenant filter; it turns a code-review habit into a build failure.

4.9 Rollout sequencing

Not everything above needs to exist before the first commit. Sequenced by when each piece stops being optional:

When	What has to be in place
Before writing much CRUD code	Monorepo split (apps/school-portal, apps/platform-admin, apps/student, apps/api); Postgres RLS policies; separate Platform Admin identity realm. Retrofitting tenant isolation into an app that already assumes a single query pattern is far more expensive than starting with it.
Before onboarding any real school with a live payment credential	Stripe Connect (or secrets-manager fallback) in place — this is a hard line, not a nice-to-have, given Section 10's finding that the designs currently capture raw secret keys. Section 4.8's CI cross-tenant isolation gate is required to merge from this point too (moved up from General Availability, confirmed 25 Aug 2026) — real payment data must never exist with zero automated tenant-isolation testing.
Before general availability	Audit trail on all Platform Admin actions; least-privilege sub-roles within Platform Admin defined.

5. White-Label Mobile App Distribution

Code Studio's direction: once the platform is live, each School should be able to have its own branded app — its own name and icon — listed separately on the Apple App Store and Google Play, powered by ULTM8 underneath. This section is engineering architecture Code Studio has adopted on ULTM8's advice, decided in a follow-up discussion on 21 Aug 2026; it is not something drawn in Figma.

5.1 Why this needs its own architecture

This is a common pattern among gym/studio SaaS platforms, but it is regulated by both stores, not a simple rebrand toggle:

- **Apple, Guideline 4.2.6:** an app generated from a commercialized template is rejected unless it is submitted directly by the business it belongs to. In practice, each School must be the App Store Connect account holder for its own branded app — ULTM8 cannot publish it under ULTM8's own developer account.
- **Google Play — white-label guidance:** Google Play's Developer Program Policies do not mandate one distribution pattern the way Apple's Guideline 4.2.6 does; the safer reading favors decentralized account management (each client on its own developer account, or a managed model where ULTM8 keeps admin access but the School still holds a distinct account), since a policy strike against a shared account risks taking every tenant's app down with it.
- **Engineering:** one React Native codebase has to produce a distinct signed binary per School — its own bundle identifier/package name, app icon, splash screen, display name, theme, push-notification credentials, and deep-link domain — baked in at build time, then submitted through that specific School's own store account.

5.2 The model: one shared app included, a branded app as a paid upgrade

Apple's guidelines explicitly permit an alternative to per-client apps: a single "aggregator" app that lists many businesses' content (a directory/picker model), as long as it isn't a pile of near-identical rejected template apps. This gives a clean two-tier structure:

Tier	What the School gets	Distribution model
Included (all Schools)	Their School is discoverable/joinable inside the shared "ULTM8" consumer app — same Student experience as designed, no extra cost or setup.	One ULTM8-branded app, one ULTM8 developer account, submitted once. No per-School store presence.
Upgrade — Branded App	Their own named, iconed app with its own listing on both stores, built from the same codebase and pointed at their tenant.	Per-School binary, built and published under that School's own Apple Developer + Google Play accounts (Section 5.4).

Fallback if the compliance risk (Section 12.2) can't be resolved: the two-tier model above is contingent on clearing the Apple 4.2.6/4.3 template-farm risk flagged in Section 12.2. If that risk can't be resolved before launch, the branded-app tier is dropped from v1 entirely — every School ships on the shared ULTM8 app only — rather than shipping a feature that risks the whole app family being rejected or pulled. The branded-app upgrade would be revisited as a later add-on only once (if) a compliant path is confirmed.

5.3 Entitlement — gated through the existing SubscriptionPlan model

The upgrade is modeled as an entitlement on the platform-level SubscriptionPlan entity already in Section 6.1 (Data Model), not a separate product: a whiteLabelApp flag/tier on SubscriptionPlan, managed from the Platform Admin app (Section 4) alongside every other cross-tenant billing control. This keeps the billing surface in one place and reuses the Platform Admin identity/audit model already decided for anything that touches tenant-level entitlements. Toggling whiteLabelApp itself is a Full Platform Admin action — the same rare, structural class of change as creating a

SubscriptionPlan — not something the narrower Publishing Ops sub-role introduced in Section 5.4 can do; Publishing Ops operates the pipeline once the entitlement already exists, it does not grant it (Section 4.4, Section 8.2).

- **Why gate it at all:** the ongoing cost isn't the first build — it's that every future ULTM8 core release means rebuilding and resubmitting a binary per branded School, plus each School's own \$99/year Apple Developer fee and \$25 one-time Google Play fee. Pricing the upgrade as a recurring add-on (not one-time) is what makes that sustainable.

5.4 Publishing pipeline — ULTM8-managed, built alongside the initial platform

Two decisions were made directly with Code Studio on this: ULTM8 runs the publishing pipeline on each School's behalf (rather than the School self-serving updates), and the pipeline is built from day one alongside the rest of the platform — not deferred until the first paying upgrade — specifically so the branded-app option is one of the features Schools are told ULTM8 offers from launch, not a later add-on nobody knows exists.

- A new TenantAppConfig record (Section 6.1) per School holds: display name, bundle ID / package name, icon and splash assets, theme tokens, APNs key reference and FCM project reference, Play App Signing enrollment status, and the School's own App Store Connect / Google Play Developer API credentials — stored in the same secrets manager infrastructure used for Stripe custody (Section 4.5), never in the application database. Unlike Stripe's move toward Connect, there is no Connect-equivalent for Apple or Google: these credentials are always held by ULTM8, not a fallback path, so they get the same monitored-rotation treatment already planned for APNs/FCM credentials — a scheduled check flags an expiring or revoked key before it silently breaks a School's build.
- A white-label build pipeline (Fastlane, or EAS Build if the Student app moves to Expo) is parameterized per tenant and triggered from a "Publish my app" flow in the School Portal once a School's branding is complete.
- ULTM8 does not request interactive Admin access to a School's Apple Developer or Google Play account — a shared login has no meaningful blast-radius limit on either platform, and doesn't fit a continuous, automated pipeline anyway. Instead, each School's Account Holder provisions a scoped, revocable App Store Connect API key and a Google Play Developer API service account — scoped for build/submit, credential status-read, and dual-approval-gated unpublish (Section 5.5) from the outset, not upgraded later — through a guided setup flow in the School Portal; ULTM8 stores only that credential in the secrets manager and fetches it per build job rather than holding a standing session. Apple's key file is downloadable exactly once at creation, so the guided flow must capture it at that moment or the School generates a fresh one. One step has no automatable workaround: only the Account Holder can personally accept Apple's and Google's developer agreements, which ULTM8 cannot do on a School's behalf — a documented support-escalation limitation, not a security gap. The School remains the legal publisher of record (satisfying Guideline 4.2.6); ULTM8 operates the pipeline against the scoped credential, never a shared account.
- Every core-app release therefore fans out into N branded rebuilds. This is real, ongoing release-engineering work and should be sized into the team's release cadence from the first sprint it's built, not bolted on afterward — it changes what "ship a release" means once any School has upgraded.
- When a School upgrades, its existing Students are already on the shared ULTM8 app (Section 5.2), not the new branded one — the upgrade doesn't move them automatically. Affected Students get a persistent, non-naggy in-app prompt to install the branded app, visible until they do or dismiss it, rather than a single push notification that's easy to miss and can't be meaningfully reissued; the School can optionally turn on a firmer, less dismissible nudge after a set delay if adoption stays low. Some tail of Students never migrating is an accepted, not eliminated, outcome — unlike the downgrade path (Section 5.5), which needs zero Student action since there's no separate app to move to.

Set expectations with Schools up front

- Apple App Review typically takes 1–2 days per submission; Google Play review is usually faster but not instant. A branded app is not available the moment a School upgrades.
- Each School needs its own Apple Developer Program membership (\$99/yr) and Google Play Console account

(\$25 one-time) before ULTM8 can publish for them — this becomes a step in the upgrade flow, not an ULTM8-side cost.

- Apple's Organization-type Developer Program enrollment (required for any School operating as a business) needs a D-U-N-S number, which can itself take one to several weeks to obtain if the School doesn't already have one — start this well before a School plans to launch its branded app, not the day they upgrade.
- Google Play requires a new app to run a closed testing track with a minimum tester count for a set period before it's eligible for production release — factor this into the same expectation-setting conversation as App Review timing.

5.5 Downgrade / cancellation — decided

If a School downgrades the white-label add-on (its SubscriptionPlan no longer includes whiteLabelApp, but the School itself stays active), its branded app is frozen at its last build rather than pulled: the app stays live on both stores and continues to work against the shared API, but stops receiving core-feature rebuilds until the School re-upgrades. “Stays live” describes ULTM8's own behavior — it will not proactively unpublish the app — not immunity from Apple's or Google's own review and takedown policies, which continue to apply to a frozen build exactly as they would to an active one.

ULTM8 retains the School's Apple/Google credential in the secrets manager (Section 4.5) for a 90-day grace period after downgrade, in case the School re-upgrades — long enough to cover a billing hiccup or a considered decision, short enough that ULTM8 isn't indefinitely holding access it has no active use for. After 90 days without re-upgrade, a scheduled job (Section 9) purges ULTM8's own stored copy of the credential; the School's underlying Apple/Google account and its own copy of the credential are untouched — this only removes ULTM8's ability to push further builds, which it already couldn't do post-downgrade anyway. Re-upgrading after the purge means the School re-provisions the credential through the same guided flow as initial setup (Section 5.4).

Each School's own store fees are independent of the ULTM8 subscription and continue regardless of whiteLabelApp status — disclosed at upgrade time (Section 5.4) and repeated as a reminder at downgrade, since a School that stops paying ULTM8 but forgets to also cancel its own Apple Developer Program membership will keep being charged with nothing to show for it. Because Google's \$25 Play Console fee is one-time, not recurring, this reminder and the renewal-monitoring job below apply to Apple only. Since a lapsed Apple Developer membership pulls every app under it — not just the branded one — from sale, an Apple-specific background job (Section 9) escalates a reminder to the School at 30 days, 7 days, and the day of the membership's renewal date, sourced from the same credential status-read Publishing Ops already has (Section 5.4).

If a School cancels its subscription entirely — not just the white-label add-on, but the whole account — the branded app moves to a graceful degraded state in-app (a clear “no longer active” message in place of live data) rather than either silently breaking against a since-suspended backend or being auto-unpublished from the stores. Store-side removal, if the School wants it, is a distinct, slower, dual-approval-gated Publishing Ops action (Section 8.2) — not the automatic, in-app-only step above, and not the break-glass procedure (Section 4.3), which covers only ULTM8's own emergency credential access, not customer-initiated removal. General tenant/School cancellation and offboarding is not otherwise specified anywhere in this document; this paragraph scopes the branded-app-specific handling only and flags the broader gap as a new open item (Section 12.2).

Where a build or submission fails or is delayed for a reason on ULTM8's side (a pipeline bug, a missed rebuild after a core release), the affected billing period is creditable at support's discretion; a rejection caused by the School's own content (App Review flags copyright, inappropriate content, or a broken feature in what the School submitted) is not — support judges this case by case rather than the platform trying to auto-classify fault.

5.6 Data model & module impact (see Section 6 and Section 7)

- New entity: TenantAppConfig — branding, store identifiers, and credential references per School (Section 6.1).

- SubscriptionPlan (Section 6.1) gains a whiteLabelApp entitlement tier.
- Two new scheduled background jobs (Section 9): an Apple developer-membership-renewal reminder (escalating cadence) and a credential grace-period purge job (Section 5.5).
- New module: MobileAppPublishingModule under the Platform Admin surface (Section 7) — wraps the build/submit pipeline, TenantAppConfig CRUD, and credential status-read, guarded by the same admin identity realm and audit trail as the rest of Platform Admin (Section 4.4, Section 4.7), scoped to the new Publishing Ops sub-role (Section 8.2) rather than requiring Full Platform Admin for routine operation; store-side unpublish requires dual approval (Section 5.5).

6. Data Model

Entities below are derived directly from the fields and lists shown on screen. Where a relationship is inferred rather than explicitly drawn, it is marked "(inferred)" and reflects only the tenancy decision locked in Section 1.2 — no other relationships have been invented.

6.1 Core entities

Entity	Key fields (from Figma)	Relationships
Franchise	Name, mobile number, address, type, activities, facilities, default language/currency, description, logo/banner, feeModel (Flat or Per-Headcount — franchise-fee billing basis, School-under-Franchise, configurable per Franchise; confirmed 25 Aug 2026, charging mechanics deferred to Pass 4)	Has many Schools (optional — see Section 6.2); has one PaymentAccount set — this is where the Franchise's own franchise-fee income actually lands, charged from each School via the new FranchiseFeeCharge entity (Section 10.2); subscribes to a platform SubscriptionPlan
School	Business name, mobile, address, business type, activities, facilities, ranks toggle, default language/currency, description, logo/banner, class-cancellation-policy setting (Manual / Auto-Refund / Auto-Credit — School-configurable, default Manual; confirmed 25 Aug 2026), waitlistClaimWindow setting (School-configurable, default 2 hours, stored as an integer number of minutes — default 120; confirmed 25 Aug 2026, unit clarified 26 Aug 2026, Pass 8, Decision 24), franchiseFeeSubscriptionStatus (Active / Past Due / Canceled — synced from the underlying Franchise-fee Stripe Subscription via webhook, visible to the School itself as well as the Franchise Owner's existing roster read, Section 8.2; distinct from FranchiseFeeCharge's own per-charge Successful/Pending/Failed/Refunded/Disputed history, which tracks each individual charge's outcome, not the ongoing subscription's health; added 25 Aug 2026, Pass 4)	Belongs to Franchise (nullable, per Code Studio's independent-school note); has many Branches, Instructors, Classes, MembershipPlans, TimetableSlots; has one PaymentAccount set; subscribes to a platform SubscriptionPlan (straight fix, 25 Aug 2026 — Pass 4: this relationship already existed on SubscriptionPlan's own entity row and on Franchise's; School's was the one missing it)
Branch	(inferred container under School — no dedicated "Branch" screen was found; only the sidebar item and "branchInformation"/"updateBranch" frame ids exist without extracted field detail)	Belongs to School
PaymentAccount	Provider (Stripe/Cash/Bank Transfer per Section 1.2 decision), account title, country, status (Active/De-active), mode (Live/Test), Stripe Connected Account id (no raw secret keys held — Section 4.5, Section 10.4)	Belongs to School or Franchise — a Franchise's own PaymentAccount is where its FranchiseFeeCharge income lands (Section 6.1, Section 10.2), never a School's PaymentAccount directly (confirmed, Pass 2, Section 8.2)
SubscriptionPlan (platform)	Plan name, price (e.g. "£30.00 Per Month"), description, feature list, payment type (Auto Renew), payment schedule, whiteLabelApp entitlement flag (new — Section 5.3; priced 25 Aug 2026 — corrected same day, no flat fee: a metered \$0.99-\$1.99/active-student/month component only, exact rate still to be set, Section 12.2 — reported via	Subscribed to by Franchise/School ("Franchise Subscription Plans" — Pay Now / Subscribe) — resolved, 25 Aug 2026 (Pass 4): this is ULTM8's own platform billing, not a plan a Franchise resells onward — ULTM8 charges the Franchise/School directly for platform

Entity	Key fields (from Figma)	Relationships
	Stripe usage-based billing, billed by UTM8 to the School, or the Franchise where it manages rollout centrally; the payer decides privately whether/how to recoup this from Students — pass-through, embed in membership pricing, or absorb — no Student-facing charge is built), status (Active / Past Due / Canceled — synced via the customer.subscription.updated/deleted webhook, drives the read-only degraded-state consequence, Section 10.2; straight fix, 25 Aug 2026, Pass 4: billed as its own Stripe Subscription separate from the whiteLabelApp metered component, so a failure in one can't take down the other)	access and, where entitled, the white-label add-on (Section 5.3); a Franchise's own fee charged to its member Schools (feeModel, Section 6.1 Franchise entity) is a wholly separate relationship, collected via the new FranchiseFeeCharge entity below, not SubscriptionPlan. Payment-failure consequence resolved 25 Aug 2026, Pass 4: on cancellation, the subscribing tenant's portal access degrades to read-only — full view of existing data (rosters, schedules, waivers, financial history), no new Classes/Bookings/payments — rather than deletion or forced logout; a School's own failure degrades only that School, and a Franchise's own failure degrades only the Franchise Owner's own actions (profile/branding/feeModel edits), never cascading to member Schools' own, separately-paid SubscriptionPlan access. Full detail in Section 9, Section 10.2, Section 12.1.
Instructor	Name, photo, belt/ranking, specializations (activities), phone, years of experience, bio; linked User account for its own login (Instructor customer-facing role, Section 8.1, added 25 Aug 2026)	Holds one or more RoleGrants, each School- and Branch-scoped (Section 6.1, Section 8.2) — an Instructor may hold grants at more than one School (confirmed 25 Aug 2026); teaches many Classes; has many TimetableSlots
TimetableSlot	Week day, start time, end time, start break, end break, status (On/Off)	Belongs to School (and Instructor, inferred); recurring weekly pattern, not date-specific; has many generated Class occurrences (confirmed, 24 Aug 2026)
Class	Title, activities, banner image, description, start/end date, capacity (a Class is Full once total confirmed attendees — the Student plus every guest in each Booking's attendee list, not Booking count — reaches this number; confirmed 25 Aug 2026; stored as a nullable positive integer, null = unlimited capacity — resolved 26 Aug 2026, Pass 8, Decision 24), booking-end date/time, QR-attendance-end date/time, refund-fee date, cancellation charge, terms/waiver-required flag, membership-inclusion flag (School-configurable — whether this Class is bookable using a general Membership such as Subscription/Weekly Pass, or always requires its own scoped Class Pack ticket; defaults to ticket-required for one-off Classes, confirmed 25 Aug 2026)	Belongs to School; taught by Instructor; has many Bookings; optionally generated from a recurring TimetableSlot (nullable timetableSlotId) — an independent one-off Class (e.g. a workshop) has none (confirmed, 24 Aug 2026); by default requires its own scoped Class Pack ticket even if the Student holds an active Subscription/Weekly Pass — the School can opt a specific Class into general-membership access instead via its membership-inclusion flag; the Booking-requires-Active-Membership rule applies either way (confirmed, 25 Aug 2026); booking also requires the Student's StudentRank.currentStripe

Entity	Key fields (from Figma)	Relationships
		in this Class's discipline to have this Class's type in its eligibleClassTypes, cumulative across the flattened belt+stripe sequence (corrected 25 Aug 2026, Section 6.1) — a hard block by default, unless an Instructor/Staff member sets Booking's overriddenBy field (confirmed, 25 Aug 2026; the override bypasses the rank gate ONLY — it does not bypass capacity, so an override on a Full Class still routes through WaitlistEntry rather than overbooking directly, logic review 25 Aug 2026); when Full (Section 6.1 capacity definition), booking instead offers a WaitlistEntry
MembershipPlan	Title, type (Subscription / Class Pack / Weekly Pass / Friend Pass / Trial Membership — Friend Pass added 25 Aug 2026: School-gifted, not purchased, always £0; Class Pack replaces the earlier “Single Pass” concept, revised 25 Aug 2026), price (fixed at 0 for Friend Pass), expiry (the plan's default duration/credit rule — used to compute each Membership's actual expiry date and/or starting credit count at the moment the Membership is created (generalized 26 Aug 2026, Pass 5, to mean Membership-creation time uniformly — Stripe charge success, staff Cash/Bank confirmation, or immediate creation for a £0 plan — not the original checkout-click timestamp, so a delayed cash confirmation doesn't silently eat into the plan's validity window); not used by Weekly Pass, which is unlimited within its own date window rather than credit-based — corrected 25 Aug 2026), classes included (credit quantity for Class Pack / Friend Pass plans — 1 for a single-class ticket or a single-guest gift; School-set higher tiers e.g. 5 or 10 are a CLASS PACK-ONLY upsell — a Friend Pass is always capped at classesIncluded=1, one gift, one guest, one visit, since nothing verifies which guest actually shows up (logic review, 25 Aug 2026); separately, ANY plan with a scoped Class set — Class Pack or Friend Pass — is capped at classesIncluded=1 regardless of type, since a one-off Class has only one occurrence and additional credits against it could never be spent, stranding prepaid credit (validation rule, logic review 25 Aug 2026)), scoped Class (optional for Class Pack and Friend Pass alike — when set, restricts the pack to that one specific Class, e.g. a single ticket to one workshop or a guest invited to one specific session; when null, general access to any of the School's regular classes — School chooses per gift for Friend Pass, confirmed 25 Aug 2026), visibility (Public), refund-fee date, cancellation charge, terms/waiver-required flag; requires Stripe as the payment method when type=Subscription (straight fix, 25 Aug 2026, Pass 4: auto-recurring billing has no Cash/Bank Transfer equivalent — only Class Pack,	Belongs to School; purchased by Students as Membership

Entity	Key fields (from Figma)	Relationships
	Weekly Pass, Friend Pass, and Trial Membership, all one-time purchases, remain Cash/Bank Transfer-eligible)	
Membership	Status (Active/Expired — Pending removed 26 Aug 2026, Pass 5: the Membership row itself is now created only once payment settles, rather than created upfront and status-flipped later — see Membership creation timing below), start date, frequency (One Time / recurring), classes remaining (credit count — set for Class Pack / Friend Pass types ONLY, decremented at Booking creation, not on completion — credited back (not a money refund — see Refund vs Credit, Section 10.2; reworded 25 Aug 2026, Pass 4, for consistency now that the two terms are formally distinct) if the Student cancels before the Class's cancellation-charge cutoff, not credited back on a No-Show; logic-review fix, 25 Aug 2026, see Section 12.1), expiry date (time-boxed access window, computed from the MembershipPlan's expiry rule), scoped Class (inherited from the MembershipPlan when it names one — restricts which Class this Membership can book), gifted-by (Instructor/Staff reference, Friend Pass only — distinguishes a School comp from a paid purchase) — corrected 25 Aug 2026: Weekly Pass is non-consuming, not a credit-bucket type — it never sets classesRemaining and goes Expired only when its expiry date passes; Class Pack / Friend Pass Memberships go Expired when credits reach 0 OR the expiry date passes, whichever comes first (confirmed, 25 Aug 2026); date-based expiry is a live-computed predicate evaluated at every access/booking check against the stored expiry date, not a scheduled batch flip, so there's no lag window where an expired Membership still reads Active (resolved 26 Aug 2026, Pass 8, Decision 26); straight fix, 25 Aug 2026, Pass 4: a credit restored onto an already-Expired Membership (Auto-Credit, Section 10.2, or the pre-cutoff case above) reactivates it to Active only if it expired from hitting 0 credits — if it expired because its own expiry date had already passed, the restored credit isn't usable and the Membership stays Expired. Membership creation timing (new, 26 Aug 2026, Pass 5): a Membership row is created, not status-flipped, at the moment payment settles. Any Stripe charge succeeding — a one-time PaymentIntent for Class Pack/Weekly Pass/Trial Membership, or a Subscription — creates it Active immediately via the stripe-webhook-processing job (Section 9); a Cash/Bank Transfer purchase stays uncreated until School Owner/Manager confirms the underlying Transaction via PATCH /transactions/{id}/confirm (Section 7), which atomically flips the Transaction to Successful and creates the Membership Active; any Cash/Bank-eligible plan priced at £0 is created Active immediately, with no Transaction or wait at all. Section 2's screen inventory and Section 9's membership-expiry-reminders evidence both cite "Membership Active/Pending/Expire states" as literal	Belongs to Student; references a MembershipPlan; while Active, grants the Student booking access to that School's Classes — for Subscription AND Weekly Pass plans (corrected 25 Aug 2026: Weekly Pass is non-consuming, matching Subscription's access model rather than Class Pack's), access is governed by Active status and, for Weekly Pass, its expiry-date window; for Class Pack / Friend Pass plans, access is governed by the classes-remaining/expiry-date fields instead; a scoped Class Pack or Friend Pass further restricts Bookings to that one named Class only; a Subscription cancellation sets Stripe's cancel_at_period_end (no proration) — the Membership stays Active through the paid period, then Expires when the period ends, at which point a same-day sweep cancels the Student's own future Bookings at this School dated after the Expire date, freeing those seats back to the Class/waitlist rather than leaving them dangling against a lapsed Membership (logic review, 25 Aug 2026); a failed renewal charge (Section 10.2) reuses this exact same mechanism — Stripe's own retry schedule keeps the Membership Active while the underlying subscription is past_due, Expiring it (and triggering the identical sweep) only once Stripe exhausts retries and cancels the subscription, so a payment failure and a voluntary cancellation land on the Student identically (resolved 25 Aug 2026, Pass 4); a downgrade takes effect at the next renewal, not immediately — Subscription plans only, since Class Pack/Weekly Pass/Friend Pass are not recurring and have no renewal event to downgrade at (clarified 25 Aug 2026); a Student may not hold two simultaneous Active general-access (Subscription/Weekly Pass) Memberships at the same School — this is an atomic, transactionally-locked check, matching the same atomicity requirement already established for the credit-decrement

Entity	Key fields (from Figma)	Relationships
	Figma evidence — that on-screen Pending badge reflects the underlying Transaction's own Pending status, surfaced against a not-yet-created Membership, not a raw Membership.status value, which is why Pending no longer appears here.	and waitlist-claim races (Section 11.5), re-validated at Membership-creation time rather than at Transaction/checkout time (generalized 26 Aug 2026, Pass 5, so there is never an ambiguous choice of which one a Booking draws against). If a Stripe charge has already succeeded when the check fails, the charge is refunded immediately via the Stripe Refund API — and, for a Subscription, the newly-created Stripe Subscription is also cancelled immediately (not cancel_at_period_end), so the Student isn't billed again for a Membership that was never created; if that refund call itself fails, it falls back to Decision 4's existing Manual-bucket-plus-staff-notification pattern, not a new mechanism. For a Cash/Bank confirm-path collision, the second PATCH /transactions/{id}/confirm call is simply rejected, its Transaction stays Pending, and staff sees an error — no reversal needed since nothing was charged (logic review, 25 Aug 2026) (confirmed, 25 Aug 2026)
Booking	Class date/time, status (Upcoming/Completed/Cancelled/No-Show — No-Show added by logic review, 25 Aug 2026: set by the new booking-no-show-expiry job, Section 9, when a Class's end time passes with no QR scan), sourceMembershipId (which Membership's credit this Booking drew against for the Student's own attendance — added by logic review, 25 Aug 2026, needed to decrement/refund credits correctly), refundResolution (nullable: null / Refunded / Credited — set only when this Booking's Class is cancelled and the Student's own credit needs resolving, Section 10.2) and resolvedBy (Instructor/Staff reference, nullable — null when resolved automatically by the School's class-cancellation-policy, populated when a staff member resolved it manually; both added 25 Aug 2026, Pass 4), overriddenBy (Instructor/Staff reference, nullable) and overrideReason (required together — records who bypassed the rank-gating hard block and why, confirmed 25 Aug 2026), branch_id (denormalized from Class, added 25 Aug 2026 — enables Branch Staff row-level scoping on Booking without a join through Class, Section 8.2), attendee list ("whoJoinYou") — each additional attendee beyond the Student requires either their own valid Membership, or a Friend Pass the School has gifted for that guest (confirmed, 25 Aug 2026); a Student's own Class Pack/Subscription does not cover guests automatically;	Belongs to Student and Class; status is set to Completed by the qr-attendance-processing job when the student's QR scan is matched to this Booking — this is also what increments StudentRank.classesAttended (confirmed, 24 Aug 2026; no separate Attendance entity); creating a Booking requires the Student to hold an Active Membership for that School (confirmed, 25 Aug 2026); if the Student holds more than one Active Membership, a non-consuming general-access one (Subscription/Weekly Pass) is checked first so a paid Class Pack/Friend Pass credit is never spent unnecessarily, falling back to a consuming Membership only if no free-access one covers this Class (logic review, 25 Aug 2026); when the Class's terms/waiver-required flag is set, Booking creation also requires a Signed WaiverSignature for that Student (logic review, 25 Aug 2026 — closes a gap where Waiver/WaiverSignature

Entity	Key fields (from Figma)	Relationships
	corrected 25 Aug 2026, Pass 4: each attendee-list entry now carries its own membershipId (the guest's covering Membership/Friend Pass), refundResolution, and resolvedBy, since a single Booking can draw against several different Memberships at once (the Student's own plus one Friend Pass per guest) and a single Booking-level field couldn't represent independent outcomes for each	existed but was never wired into the booking rule)
WaitlistEntry (new, confirmed 25 Aug 2026)	Position (FCFS queue order), status (Waiting / Notified / Claimed / Expired / Cancelled), joinedAt, notifiedAt, claimByDeadline (notifiedAt + the School's waitlistClaimWindow setting, Section 6.1, capped by the Class's own start time)	Belongs to Student and Class; created instead of a Booking when the Class is Full (Section 6.1); costs no credit while Waiting — only becomes a real Booking (and only then draws a credit) once Claimed
Transaction / Invoice	Invoice id, member, billing date, amount, currency (added 26 Aug 2026, Pass 5, mirroring FranchiseFeeCharge's own field — records which currency was actually charged, now that MembershipPlan pricing is confirmed per-currency, Section 11.2), status (Successful/Pending/Failed/Refunded/Disputed — Failed and Refunded added 25 Aug 2026: a declined payment_intent.failed webhook and an Auto-Refund class cancellation each need a terminal status the original two-value enum had no room for; manual Cash/Bank Transfer transitions land on Successful, not a separate value, to keep one enum — logic review; Disputed added Pass 8, Decision 55, only reachable for Stripe-paid Transactions since Cash/Bank Transfer rows have no Stripe charge id to match), disputedAmount (nullable, taken straight from the Stripe dispute event payload — added Pass 8, Decision 55), payment method (Stripe/Cash/Bank Transfer — GoCardless appeared in the designs but is excluded per Section 1.2; straight fix, 25 Aug 2026 — Pass 4: corrected from the stale Stripe/Cash/GoCardless list to the three methods actually built), refundedAmount (nullable, cumulative across possibly-multiple partial refunds against the same Transaction — e.g. several classes cancelled separately off one multi-class pack — validated so it can never exceed the original amount; added 25 Aug 2026, Pass 4), downloadable invoice	Belongs to School (ledger) and Student (payer); references PaymentAccount, and the MembershipPlan being purchased — once payment settles and the Membership is created (Section 6.1's Membership-creation-timing note), the Transaction also references the resulting Membership (added 26 Aug 2026, Pass 5)
Waiver	Title, body text, per-school assignment	Belongs to School; has many WaiverSignatures
WaiverSignature	Signer full name, signature, signed date, status (Signed/Unsigned/Expired/Pending)	Belongs to Waiver and Student
Rank (per Style/Discipline)	Name, order (position in the ladder), primary colour, optional secondary colour (two-colour belts), stripe tiers (ordered list — corrected 25 Aug 2026: each stripe tier is now its own gating/progress checkpoint, carrying its own STABLE ID (not a positional index, which would break if tiers are reordered — logic review 25 Aug 2026), count, colour, classes-required, minimum days, AND	Belongs to a Style/Discipline (ordered ladder); may require many Skills; referenced by StudentRank

Entity	Key fields (from Figma)	Relationships
	eligibleClassTypes, rather than one flat set for the whole belt — a White 3-stripe Student commonly has different class access than a White 0-stripe Student, so belt-level-only gating was too coarse; the number of stripe tiers per belt is set independently by the School for each belt — no fixed count across the ladder, e.g. White might have 4 tiers and Blue only 2 (clarified 25 Aug 2026); eligibleClassTypes remains dual-purpose — grading credit AND booking eligibility — cumulative across the FLATTENED belt+stripe sequence for the discipline, e.g. White-0, White-1, White-2, White-3, Blue-0, Blue-1..., so a School can let a White 3-stripe tier's list reach into Blue's class types directly; the School Portal validates that Rank.order and each stripe tier's position are unique and contiguous per discipline, since the cumulative eligibleClassTypes computation depends on a well-defined sequence — validation rule, logic review 25 Aug 2026), weekly class-count cap, required skills, years-in-rank flag (Black Belt and above)	
StudentRank	Current rank (references Rank), currentStripe (references a stripe tier within that Rank — added 25 Aug 2026; required now that gating/grading eligibility vary per stripe, not just per belt, Section 6.1; reset to the new Rank's first stripe tier on every Promotion or Downgrade — confirmed 25 Aug 2026, since a belt change has no other well-defined starting stripe), date of current rank, classes attended toward next stripe/rank (resets to zero at every checkpoint in the flattened belt+stripe sequence — each stripe and each belt is its own complete grading stage, not a fraction of one running belt-level count; confirmed 25 Aug 2026), per-skill sign-off status (Not Started/Learning/Signed Off) for any skills required at the CURRENT checkpoint — stripe or rank, not the whole belt (clarified 25 Aug 2026), readiness bucket and progress % (both computed, not stored, scoped to the current checkpoint), per discipline — kept distinct from the student's own join/member date; the working prototype (Section 2.3) surfaced a defect from conflating the two	Belongs to Student; one row per discipline the student trains in; has many PromotionEvent (rank history)
Style / Discipline (new, validated)	Name (e.g. Jiu Jitsu, Karate, Judo, Krav Maga), class types offered (e.g. Kids Fundamentals, Adult Sparring, Competition Team)	Belongs to School; has many ordered Ranks (the ladder)
Skill (new, validated)	Name, description, discipline	Belongs to a Style/Discipline; required by many Ranks; referenced by many Lessons (confirmed, Decision 58)
PromotionEvent (new, validated — rank history)	From-rank, to-rank, from-stripe, to-stripe (from-stripe/to-stripe added 25 Aug 2026 — a Stripe Award leaves from-rank/to-rank unchanged, so without these two fields the event carried no record of which stripe actually changed; a belt Promotion/Downgrade sets from-stripe/to-stripe to the outgoing and incoming belts' respective first stripe tier, per the reset rule on StudentRank, Section 6.1), date, graded by (Instructor),	Belongs to Student; references graded-by Instructor and the from-rank/to-rank Ranks

Entity	Key fields (from Figma)	Relationships
	type (Promotion / Downgrade / Bulk Promotion / Bulk Stripe Award / Stripe Award — Bulk Stripe Award added 25 Aug 2026: stripe ceremonies are graded far more often, and in larger batches, than individual belt promotions, so bulk grading needed a stripe-level equivalent to Bulk Promotion; Stripe Award added 25 Aug 2026: a stripe increment within a belt is not a belt promotion, so it gets its own type for an accurate rank history now that stripes are independent progress checkpoints, Section 6.1 — no PromotionEvent of any type is ever created automatically; every belt Promotion and every Stripe Award is a coach-initiated action, and a Stripe Award is not offered once a Student is already at a belt's highest configured stripe tier — the coach's only next action from there is a manual Promotion (confirmed, 25 Aug 2026)), acknowledgement flag (set when graded despite a required skill not yet signed off)	
Lesson (confirmed, Decision 58)	Title, category, duration, instructor, description, linked skill(s), format (Prerecorded/Live), videoRef, captionStatus (Pending/Processing/Ready/Failed), captionTrackRef	Belongs to a Category; linked to one or more Skills — resolves the mobile videoScreen candidate (Section 12.2), confirmed scope per Decision 58
User (base)	Email, phone (E.164 format, validated at registration — resolved Pass 8, Decision 24), first name, surname, username (mobile only), 6-digit passcode, DOB (ISO 8601 date, no time component — resolved Pass 8, Decision 24), gender, nationality, language, currency, address, profile photo	Customer-facing only: Student, SchoolStaff, Instructor, Guardian (see Section 8) — each holds one or more RoleGrant records (below) rather than a single fixed role column. Platform Admin staff are a separate identity, not a row here — Section 4.4.
RoleGrant	user_id, role (Student / School Owner-Manager / Branch Staff / Instructor / Franchise Owner / Guardian — Platform Admin excluded, separate realm, Section 4.4; corrected 27 Aug 2026 to match Section 8.2's actual 6-role model — the enum previously read "SchoolStaff", a name matching neither School Owner/Manager nor Branch Staff, and omitted Branch Staff entirely), scope (school_id and/or branch_id and/or franchise_id, mapped per role by a database CHECK constraint: Student, School Owner-Manager, and Instructor carry school_id — Branch Staff and Instructor additionally carry branch_id, School Owner-Manager does not since that role spans every Branch under its School — Franchise Owner carries franchise_id only, and Guardian carries no tenant scope at all, since guardianship is a relationship to a specific linked minor Student profile, not a grant against a School, Branch, or Franchise), granted_by, granted_at, revoked_at (nullable)	Belongs to User; the sole source of truth for the roles/scopes a User currently holds — sessions and JWTs are built from the full set of currently-active (revoked_at IS NULL) grants at login/refresh time; a User may hold more than one grant, including the same role at more than one School (confirmed 25 Aug 2026 — e.g. an Instructor teaching at two Schools)
Guardian (linked minor profile)	Guardian: the User(base) row above, holding payment method and waiver-signing authority. Linked Student profile (minor): name, DOB, rank — no independent login by default	Guardian has many linked (minor) Student profiles; an optional limited login for a linked Student (provisional age 13, pending legal review) grants read-only own-schedule/rank access only — never payment or waiver capability; at the age of majority

Entity	Key fields (from Figma)	Relationships
		(provisional 18, pending legal review) the Student self-registers a fresh, independent account rather than migrating the linked profile (confirmed 25 Aug 2026); revoking or deleting a Guardian account cascades revoked_at onto every linked minor's own RoleGrant, so a minor's limited login doesn't outlive the Guardian relationship that authorized it (resolved 26 Aug 2026, Pass 8, Decision 28)
AdminUser (new, proposed)	Email, name, sub-role (Support / Billing-Payments Ops / Full Platform Admin — Section 4.4), SSO identity reference	Lives in the separate Platform Admin identity realm (Section 4.4); never joined against tenant User records
AuditLogEntry (new, proposed)	Actor (AdminUser), action, target tenant/entity, before/after, timestamp, subjectUserId (nullable — populated whenever the target entity is itself owned by or references one Student/Guardian/Instructor record, left null for tenant-wide actions with no single subject; added Pass 8, Decision 43)	Immutable; written on every Platform Admin action against tenant data (Section 4.7)
TenantAppConfig (new, proposed)	Display name, bundle id/package name, icon/splash assets, theme tokens, APNs key ref, FCM project ref, Play App Signing enrollment status, App Store Connect/Google Play credential refs — scoped for build/submit, status-read, and unpublish (secrets manager only — Section 4.5)	Belongs to School only, not Franchise (a Franchisor's narrow read-only Schools roster, Section 8.2, does not extend to School-level publishing config); created only when whiteLabelApp is entitled (Section 5.3, Section 5.4); credential retained for a 90-day grace period after downgrade before ULTM8 purges its own copy (Section 5.5)
Translation	Screen name, label key, content string, locale	Standalone CMS table; cached, not joined to other entities
Notification	Title/body, read state, target user	Belongs to User
FranchiseFeeCharge (new — Section 10.2)	Franchise, School, billing period, fee basis snapshot (Flat or Per-Headcount, copied from Franchise.feeModel at charge time so a later feeModel change never rewrites history), computed amount, currency, active-student-count snapshot (Per-Headcount only, captured at charge time — active means holding at least one Membership there with status=Active and type in Subscription/Class Pack/Weekly Pass; Friend Pass and Trial Membership excluded, Decision 54; Trial Membership itself capped at a platform-enforced 30-day expiry maximum to close the resulting headcount-dodge gap, same decision), Stripe PaymentIntent/Invoice id, status (Successful/Pending/Failed/Refunded/Disputed — same enum as Transaction, Disputed added Pass 8, Decision 55), refundedAmount (nullable, cumulative — added 25 Aug 2026, Pass 4, same as Transaction, Section 6.1), disputedAmount (nullable, taken straight from the Stripe dispute event payload — added Pass 8, Decision 55,	Belongs to Franchise (payee) and School (payer); references both parties' PaymentAccount; Flat fee generated by a standing Stripe Subscription, Per-Headcount fee generated monthly by the new franchise-fee-usage-reporting job (Section 9) reporting each School's active-student-count as Stripe metered usage; charged via a Stripe Direct/Destination charge against the School's saved payment method, settling into the Franchise's own connected PaymentAccount — the same mechanism Membership billing already uses for Student→School, aimed instead at School→Franchise (confirmed 25 Aug 2026, Pass 4);

Entity	Key fields (from Figma)	Relationships
	same as Transaction)	ULTM8 takes no cut (confirmed 25 Aug 2026). A disputed or erroneous charge is refunded (full or partial, via Stripe Refund API) only by the Franchise Owner — the Franchise's own PaymentAccount is the one holding the money, so the Franchise, not the paying School, is the party who'd issue it back (confirmed 25 Aug 2026, Pass 4). See Section 10.2, Section 12.1.
PlatformCharge (new — Section 10.2, Decision 42)	School/Franchise reference, charge type (SubscriptionPlan tier fee or whiteLabelApp entitlement fee), amount, currency, Stripe charge/invoice reference, status (Successful/Pending/Failed/Refunded/Disputed — same enum as Transaction and FranchiseFeeCharge, Decision 55), disputedAmount (nullable, taken straight from the Stripe dispute event payload — Decision 55, same as Transaction and FranchiseFeeCharge)	Belongs to whichever School or Franchise the platform-level charge is against; mirrors FranchiseFeeCharge's shape but bills ULTM8's own platform-level fees (SubscriptionPlan tier, white-label app entitlement) rather than School ↔ Franchise fees (Decision 42). See Section 10.2, Section 12.1 Decision 42.
DeviceToken (new — Section 12.1 Decision 46)	userId, platform, token, app variant reference (shared Student app or a specific School's TenantAppConfig), last-seen timestamp	Belongs to User; referenced by push-notification delivery to route a push to the correct credential set — the shared Student app's or a specific School's white-label app's — since nothing else in this model previously distinguished them (Decision 46). See Section 12.1 Decision 46.
ConsentRecord (new — Section 12.1 Decision 57)	Guardian reference, linked-minor (Student) reference, policyVersion, timestamp, status (Active/Withdrawn, with its own withdrawal timestamp), tier (baseline or camera-specific)	Belongs to Guardian and the linked Student; the operative authorization for a Guardian's consent on a minor's behalf, tier-scoped so camera-specific consent can be withdrawn independently of baseline consent (Decision 57). See Section 12.1 Decision 57.

6.2 Tenancy hierarchy (as locked in Section 1.2, refined)

- A Franchise MAY own many Schools ("Affiliated School Information" was seen on the franchiseInformation screen).
- A School's franchise_id is nullable — independent, non-franchised Schools are a first-class case per Code Studio's clarification.
- A School MAY have many Branches (sidebar exposes "Branches" as distinct from "School" and "Franchises"; no Branch-specific field screen was extracted, so Branch fields are assumed to mirror School's address/contact fields pending confirmation — see Section 12). Branch is a real, RLS-enforced access boundary (confirmed 25 Aug 2026) — the same mechanism as tenant isolation (Section 4.3), keyed on branch_id for TimetableSlot, Class, Instructor, and Booking specifically, since those are the entities genuinely tied to one physical location (Booking gained a denormalized branch_id in Pass 2 to support Branch Staff RLS); Student and Membership stay School-scoped, since a Student may train across multiple Branches under one Membership. This settles, at minimum, that Branch needs its own id and a school_id foreign key — narrowing, not fully resolving, the open field-schema question above.

- Both Franchise-level and School-level PaymentAccount configuration were observed. Resolved, Pass 2 (25 Aug 2026, Section 8.2): a School's PaymentAccount is always its own — a Franchisor has no access to it, consistent with the Franchise Owner redefinition's privacy model. Franchise-fee billing (the Franchise entity's feeModel setting, Section 6.1) is a wholly separate payment relationship, not a School-level PaymentAccount at all, so no RLS discriminator between Franchise-level and School-level PaymentAccount rows is needed — there is only ever the School's.

7. API Surface / NestJS Module Breakdown

Proposed module structure for the NestJS backend, one module per bounded feature area identified in the Figma audit. Endpoints listed are the operations the screens visibly require (list/detail/create/update/delete/submit actions seen on each screen) — this is a starting contract, not a final OpenAPI spec. Platform-wide conventions (resolved 26 Aug 2026, Pass 8, Decision 22): a standard `{error: {code, message}}` envelope on every non-2xx response; cursor-based pagination (`?cursor=&limit=`) on every list endpoint; and a `/v1` URI prefix from the first deploy, so a future breaking change has somewhere to go without an undocumented flag day.

Module	Purpose	Representative endpoints (from the screens)
AuthModule	Login, registration, OTP, password/passcode reset, social login	POST <code>/auth/login</code> , <code>/auth/register</code> , <code>/auth/otp/send</code> , <code>/auth/otp/verify</code> , <code>/auth/forgot-password</code> , <code>/auth/reset-password</code> , <code>/auth/social/{provider}</code>
UsersModule	Base user profile shared by all roles	GET/PATCH <code>/users/me</code> , PATCH <code>/users/me/{field}</code> (mirrors the granular <code>updateName/Email/Phone/...</code> screens), DELETE <code>/users/me</code> (self-service account deletion, requires re-authentication — added Pass 8, Decision 44)
TenantsModule	Franchise, School, Branch CRUD; role-grant assignment/revocation (RoleGrant, Section 6.1); Franchise Owner School-roster read	CRUD under <code>/franchises</code> , <code>/schools</code> , <code>/branches</code> ; POST/DELETE <code>/users/{id}/role-grants</code> ; GET <code>/franchises/{id}/schools</code> (Franchise Owner roster read, Section 8.2)
PaymentsModule	PaymentAccount config (Stripe Connect onboarding, Cash/Bank-Transfer records) + Stripe checkout/subscriptions/webhooks — no raw tenant secret keys handled, Section 4.5/Section 10.4	CRUD <code>/payment-accounts</code> , POST <code>/payments/connect/onboard</code> , POST <code>/payments/charge</code> , POST <code>/payments/subscribe</code> , POST <code>/payments/webhooks/stripe</code> , GET <code>/payments/methods</code> (student card list), POST <code>/payments/franchise-fees/charge</code> (Flat), POST <code>/payments/franchise-fees/report-usage</code> (Per-Headcount, feeds Stripe metered billing) — added 25 Aug 2026, Pass 4 (Section 10.2), POST <code>/payments/refund</code> (money-back path — Stripe Refund API or a manual Cash/Bank ledger reversal; School Owner/Manager only for a Membership purchase Transaction, Franchise Owner only for a FranchiseFeeCharge, since authorizing money leaving the business is kept separate from who can process the underlying sale), POST <code>/bookings/{id}/credit-restore</code> (credit-back path — restores a spent classesRemaining credit for one named credit-record on the Booking, targeted by membershipId since a Booking can hold several, Section 6.1; no money movement; Branch Staff-accessible and branch_id-RLS-scoped the same as their other Booking access, Section 8.2 — same narrow-action shape as the existing membership-status-check endpoint, no raw Transaction/Membership exposure) — added 25 Aug 2026, Pass 4 (Section 10.2, Section 8.2), PATCH <code>/transactions/{id}/confirm</code> (Cash/Bank Transfer settlement — School Owner/Manager-accessible only, not Branch Staff, since Section 8.2 gives Branch Staff no raw Transaction/Membership access and Transaction carries no branch_id to scope by; atomically flips the Transaction Pending→Successful and creates the resulting Membership Active, Section 6.1; idempotent — a repeat call on an already-Successful Transaction no-ops, the same pattern already established for credit-restore) — added 26 Aug 2026, Pass 5

Module	Purpose	Representative endpoints (from the screens)
		(Section 6.1, Section 10.2)
SubscriptionPlansModule	Platform-level plans a Franchise/School subscribes to (Platform Admin app only for plan authoring — Section 4)	GET /plans, POST /plans/{id}/subscribe ("Pay Now"/"Subscribe")
InstructorsModule	Instructor CRUD; grading and booking-override actions now attributed to an authenticated Instructor identity (Section 8.1)	CRUD /schools/{id}/instructors; POST /classes/{id}/attendance-scan (see Section 12.2 — undocumented second scan endpoint), PATCH /bookings/{id}/override (Instructor- or Staff-scoped per Section 8.2)
TimetableModule	Weekly recurring schedule	CRUD /schools/{id}/timetable (per weekday/slot)
ClassesModule	Class CRUD + booking + waitlist	CRUD /schools/{id}/classes, POST /classes/{id}/book (rank-gated, Section 6.1; returns 409 + offers a waitlist join when Full), PATCH /bookings/{id}/cancel, PATCH /bookings/{id}/override (Instructor/Staff only — sets overriddenBy/overrideReason, bypasses the rank gate, Section 6.1), GET /bookings/me, POST /classes/{id}/waitlist (join), DELETE /waitlist/{id} (leave), POST /waitlist/{id}/claim — expanded 25 Aug 2026 for rank-gating and the waitlist (Section 6.1, Section 9)
MembershipsModule	Membership plan CRUD + student purchase; Branch Staff membership-status check (Section 8.2)	CRUD /schools/{id}/membership-plans, POST /membership-plans/{id}/purchase (narrowed 26 Aug 2026, Pass 5: creates a Membership immediately only for a successful Stripe charge or a £0 plan; for Cash/Bank Transfer it creates the Transaction, Pending, only — the Membership is created later via PaymentsModule's confirm endpoint once staff confirms, Section 6.1, Section 10.2), GET /memberships/me, GET /students/{id}/membership-status (Branch Staff-accessible computed active/expired signal only, no raw Membership/Transaction rows — mirrors RanksModule's GET /students/{id}/eligibility pattern, confirmed 25 Aug 2026)
TransactionsModule	Ledger / invoice history	GET /schools/{id}/transactions, GET /transactions/{id}/invoice (download)
WaiversModule	Waiver assignment + e-signature	CRUD /schools/{id}/waivers, POST /waivers/{id}/sign, GET /waivers/me
RanksModule	Belt/grading reference data + student progression, including promotion/downgrade, bulk grading, and skill-gating (validated by a working prototype, Section 2.3)	GET /styles/{id}/ranks (full ordered ladder), POST/PATCH/DELETE /styles/{id}/ranks/{rankId} (manage the ladder), GET /students/{id}/ranks, GET /students/{id}/eligibility (computed readiness bucket + progress, powers the Grading Board), POST /students/{id}/ranks/{disciplineId}/promote, POST /students/{id}/ranks/{disciplineId}/downgrade, POST /students/{id}/ranks/{disciplineId}/stripe-award (added 25 Aug 2026 — a distinct, coach-initiated action from promote; never system-triggered, and unavailable once the Student is at a belt's highest configured stripe tier, Section 6.1), POST /schools/{id}/ranks/bulk-promote (grade multiple students in one pass, each judged individually), POST /schools/{id}/ranks/bulk-stripe-award (added 25 Aug 2026, mirrors bulk-promote for the far more common stripe-ceremony case), PATCH /students/{id}/skills/{skillId} (cycle

Module	Purpose	Representative endpoints (from the screens)
		sign-off status), GET /students/{id}/rank-history — expanded from the original 3-endpoint placeholder to match the flows the working prototype validated (Section 2.3)
CurriculumModule (confirmed, Decision 58)	Lesson content linked to specific skills; resolves the mobile videoScreen/VIDEO nav tab (Section 12.2), confirmed scope per Decision 58	GET /skills/{id}/lessons, GET /lessons/{id}, CRUD /admin/curriculum/lessons (authoring)
AttendanceModule	QR check-in — marks the matching Booking Completed and drives StudentRank.classesAttended for grading (Section 2.3/Section 6.1); no separate Attendance entity (confirmed, 24 Aug 2026); checks that Student's camera-tier ConsentRecord status server-side before accepting a scan, rejecting it if withdrawn (Decision 57, Decision 69)	POST /attendance/scan
TranslationsModule	i18n CMS (authored in Platform Admin app only — Section 4)	CRUD /translations, GET /translations?locale=&screen=
AcademiesModule	Mobile-facing discovery (read-optimized view over Tenants/Classes/Memberships)	GET /academies, GET /academies/{id}, GET /academies/{id}/timetable
NotificationsModule	In-app/push/email notifications	GET /notifications/me, PATCH /notifications/{id}/read
SettingsModule	Language, currency, legal content, permissions metadata	GET /settings/languages, GET /settings/currencies, GET /legal/{doc}
PlatformAdminModule (new)	Cross-tenant management, admin sub-role assignment; every endpoint requires the separate Platform Admin identity realm (Section 4.4) and writes to AuditLogEntry (Section 4.7)	GET /admin/tenants, GET /admin/tenants/{id}, POST /admin/audit-log (internal), GET /admin/audit-log
MobileAppPublishingModule (new)	TenantAppConfig CRUD, white-label build/submit pipeline, and credential status-read for the branded-app upgrade; ULTM8-managed on the School's behalf via scoped API keys (Section 5.4), scoped to the new Publishing Ops sub-role (Section 8.2) with dual approval required for store-	GET/PATCH /admin/tenants/{id}/app-config, POST /admin/tenants/{id}/app-config/publish, GET /admin/tenants/{id}/app-config/status, POST /admin/tenants/{id}/app-config/unpublish (dual-approval-gated)

Module	Purpose	Representative endpoints (from the screens)
	side unpublish (Section 5.5)	

Endpoints under PlatformAdminModule and MobileAppPublishingModule, and the admin-facing surfaces of SubscriptionPlansModule and TranslationsModule, are namespaced separately (e.g. under /admin/*) and sit behind their own guard chain tied to the Platform Admin identity realm — never the tenant JWT guard used by every other module above (Section 4.4).

8. Authentication & Role / Permission Model

8.1 Authentication, as designed

- Both apps use the same pattern: Email + a 6-digit "Passcode" (rather than an arbitrary-length password), with "Remember Me", plus social login buttons for Microsoft, Apple, Facebook and Google (Microsoft appears twice on the same screen — likely a design artifact, see Section 12).
- Registration collects first name, surname, (mobile app also collects a username), email, phone number with country code, and the passcode + confirmation.
- Forgot/reset password is phone-based: an SMS OTP is sent, entered on a 4-digit verificationPin screen with a visible countdown, followed by a "Create New Password" (still passcode-style) screen.

Open question — see Section 12 for full detail

- Whether the 6-digit "Passcode" IS the account credential (unusual — needs strict brute-force/lockout protection if so) or is a PIN layered on top of a longer password not shown in the designs.

8.2 Roles observed

The translations CMS and the canonical Dashboard's side menu group content under four labels: Student, School, Admin, and User. Combined with the revised app-boundary decision (Section 1.2, Section 4), the proposed role model is: A person can hold more than one role — including the same role at more than one School — via the RoleGrant model (Section 6.1); the table below describes each role's scope, not a fixed one-role-per-person assumption.

Role	App & identity	Scope
Student	React Native app · customer identity	Own profile, bookings, memberships, payments, waivers, ranks (see Guardian, below, for guardian-managed minor accounts)
School Owner / Manager ("School")	React — School Portal · customer identity	Manage their own School/Branches: Instructor and Branch Staff invitations, timetable, classes, memberships, transactions, waivers, payment accounts. See Branch Staff and Instructor, below, for the narrower Branch-scoped and coaching-scoped roles now split out of this row. Degrades to read-only if the School's own platform SubscriptionPlan payment fails and Stripe exhausts retries — full view, no new Classes/Bookings/payments — until billing is resolved (Section 6.1, Section 10.2; added 25 Aug 2026, Pass 4).
Franchise Owner	React — School Portal · customer identity	Franchise-level only, by design — a Franchisor never gets a School Owner's access to that School's own data or accounts (confirmed 25 Aug 2026; distinct from the Guardian model's full-access relationship, Section 6.1): manage the Franchise profile/branding and the feeModel (Flat or Per-Headcount) setting; a read-only roster of Schools under the Franchise — address, owner/operator name and contact, brand-compliance/good-standing status, franchise-fee payment status, and active student count (for headcount billing). Not an RLS escalation into School-scoped data — a purpose-built narrow read, mirroring Branch Staff's status-check pattern below. Fee-calculation/charging mechanics resolved, Pass 4

Role	App & identity	Scope
		(Payments), 25 Aug 2026: Stripe-automated via the new FranchiseFeeCharge entity (Section 6.1, Section 10.2), mirroring Membership billing's Student→School pattern; ULTM8 takes no cut. A disputed FranchiseFeeCharge is refunded by the Franchise Owner only (Section 6.1, Section 10.2), since the Franchise's own PaymentAccount holds the money. Legal review still recommended on what a Franchisor may collect under franchise-disclosure law (Section 12.2).
Platform Admin — Full	React — Platform Admin (separate app) · separate identity realm, Section 4.4	Cross-tenant oversight: translations CMS, platform subscription plans, tenant management, (Reports/Customer Services once designed — see Section 12)
Platform Admin — Support (new, least-privilege)	React — Platform Admin (separate app) · separate identity realm, Section 4.4	Account-level status metadata across tenants only (name, subscription plan, payment status) for triage, plus a time-boxed, audited, tenant-scoped impersonation session to actually help with a specific issue; never a Stripe Connected Account id, a secrets-manager reference, or tenant data outside an active impersonation session — Section 4.4
"User" (generic)	Unclear	Appears as a distinct label in the translations CMS grouping alongside Student/School/Admin; could not be mapped to a concrete role from the designs. Caution: do not implement as a literal additional role — every concrete permission need identified this pass is covered by Student, School Owner/Manager, Branch Staff, Instructor, Franchise Owner, Guardian, or a Platform Admin sub-role; treat this label as a stale CMS grouping artifact pending final confirmation (Section 12.2).
Branch Staff	React — School Portal · customer identity, Branch-scoped view	Real Branch-scoped access via branch_id Row-Level Security on TimetableSlot, Class, Instructor, and now Booking (Section 4.3); no raw row access to Membership or Transaction (genuinely School-wide, not Branch-scopable) — instead, a narrow purpose-built membership-status-check endpoint returning only an active/expired signal, never underlying records (confirmed 25 Aug 2026). Extended 25 Aug 2026, Pass 4, on the same narrow-action principle: Branch Staff can resolve a cancelled Class's affected Bookings via Auto-Credit (POST /bookings/{id}/credit-restore — no money involved), but not via Auto-Refund — authorizing money back out is kept to School Owner/Manager, the same separation-of-duties boundary most POS systems draw between taking a sale and authorizing a refund. Two guards keep this narrow despite writing to the otherwise off-limits Membership entity (straight fix, 25 Aug 2026, Pass 4): credit-restore enforces the same Booking-level branch_id RLS as Branch Staff's other Booking access — a Booking outside their Branch 403/404s — and it only ever acts on a credit-record the class-

Role	App & identity	Scope
		cancellation-processing job has already flagged pending, so a call can't manufacture a credit outside a genuine cancelled-Class resolution.
Instructor	React Native and/or React Portal (role-gated coach view) · customer identity, own login (new, 25 Aug 2026)	School- and Branch-scoped: attendance scan, grading (promote/downgrade/Stripe Award, bulk versions), booking override, view own schedule. Not payments, School settings, or instructor management. Provisioned by School invite, not self-registration; no new app required (confirmed 25 Aug 2026).
Platform Admin — Billing/Payments Ops (new, least-privilege)	React — Platform Admin (separate app) · separate identity realm, Section 4.4	View PaymentAccount configuration status and initiate a Stripe Connect credential rotation; never sees a decrypted secret (Section 10.4, Section 4.4).
Guardian	React Native app (Student app) · customer identity, full adult login	Full access to own profile plus every linked minor Student profile: payment method, waiver-signing authority, bookings, memberships. A linked minor may optionally get a limited login (provisional age 13, pending legal review) for read-only own-schedule/rank access only — never payment or waiver capability. At the age of majority (provisional 18, pending legal review) the person self-registers a fresh, independent Student account (confirmed 25 Aug 2026, Section 6.1).
Platform Admin — Publishing Ops (new, least-privilege)	React — Platform Admin (separate app) · separate identity realm, Section 4.4	TenantAppConfig CRUD, publishing-pipeline retry/support operations, and credential status-read for renewal monitoring across tenants; store-side unpublish requires dual approval (Section 5.5). Does not grant or revoke a School's whiteLabelApp entitlement itself — that stays a Full Platform Admin action (Section 5.3, Section 4.4).

Instructors now have their own authenticated login (Instructor role, above; confirmed 25 Aug 2026) — this closes the open question previously raised here.

8.3 Recommended mechanics (proposed, not shown in Figma)

- JWT access + refresh tokens issued by AuthModule for all customer-facing identities (Student, School, Franchise, Instructor, Guardian): 15-minute access tokens plus a Redis-backed, per-session revocable refresh-token store (Section 3.1) with rotation and reuse detection — a reused/already-consumed refresh token revokes the entire token family, forcing re-login. This bounds even a live permission downgrade (not just full offboarding) to the access-token TTL window — an accepted, not eliminated, limitation (confirmed 25 Aug 2026).
- Platform Admin staff authenticate against a separate identity realm entirely — a distinct issuer/audience with SSO to Code Studio's IdP and mandatory 2FA, never the customer AuthModule (Section 4.4). Same Redis-backed revocable-session mechanism as customer identities, but with a stricter 5-minute access-token TTL and no long-lived “Remember Me” refresh equivalent (confirmed 25 Aug 2026) — access for any tier is revoked immediately on offboarding, not just blocked on future logins.
- Passport strategies: local (email+passcode) and OAuth2 per social provider actually confirmed for customer identities (see Section 12 on the duplicate Microsoft button); a separate SSO strategy (e.g. SAML/OIDC against the company IdP) for Platform Admin.
- NestJS Guards + a `@Roles()` decorator checking for the presence of any required role+scope in a session's currently-active RoleGrant set (Section 6.1) at the controller level; row-level tenant scoping (Franchise/School/Branch id) enforced via PostgreSQL Row-Level Security, not just in service code (Section

4.3) — extended this round to include branch_id on Booking. Franchise Owner's School-roster read (Section 8.2) is a purpose-built endpoint, not an RLS escalation into School-scoped tables.

- Rate limiting on /auth/otp/* and /auth/login to mitigate passcode brute-forcing given the short 6-digit credential space.
- **RoleGrant is the sole source of truth for who holds what role where (confirmed 25 Aug 2026):** user_id, role, scope, granted_by, granted_at, revoked_at (Section 6.1) — sessions/JWTs are built from the full set of currently-active grants at login/refresh time, not an inferred or implicit role column. This is what lets one identity hold, say, an Instructor grant at one School and a Staff grant at another, without a data-model change.

9. Background Jobs / Queues (Redis + BullMQ)

Figma cannot show backend jobs directly — the candidates below are proposed because the screens explicitly require the underlying async behaviour (an OTP screen implies OTP delivery; a Download action implies file generation; a Pending payment status implies webhook-driven state changes, etc.). Each is derived from a specific screen, not invented independently.

Queue / job	Triggered by	Evidence in the designs
otp-delivery	Registration, forgot password	verificationPin screens (web + mobile) with visible SMS countdown; delivered via Twilio Verify since 26 Aug 2026, Pass 5 (Section 11.4)
stripe-webhook-processing	Stripe events	paymentSuccessful / PaymentMethodUpdateMessage screens; Pending→Successful status transitions in transactionsHistory; extended 25 Aug 2026, Pass 4, to also process invoice.payment_failed — fired on each Stripe Smart Retry attempt — feeding notification-fanout so a Student/School sees a payment-failure warning during the retry window rather than a silent lockout once retries exhaust (Section 10.2); and, since 26 Aug 2026 (Pass 5), to create the resulting Membership Active on payment_intent.succeeded, not just flip the Transaction's status (Section 6.1)
invoice-generation	Completed transaction	"Download" action per row in transactionsHistory
membership-expiry-reminders	Membership nearing/at expiry	Membership Active/Pending/Expire states; franchise-plan "Auto Renew"
waiver-signature-requests	Waiver assigned to a student	waiverSchool status list (Pending/Signed/Unsigned/Expired) and Notification samples ("...has invited you to sign the following waiver")
notification-fanout	Any of the above, plus manual school messages	Notification / All Notification screens on both apps
qr-attendance-processing	QR scan at check-in	qrCode screen; Dashboard Attendance KPI card (Present/Absent); also confirmed (24 Aug 2026) to mark the matching Booking Completed and increment StudentRank.classesAttended — see Section 6.1
class-occurrence-generation (new, confirmed 24 Aug 2026)	A recurring TimetableSlot, on a rolling schedule	Materializes bookable Class rows ahead of time from active TimetableSlots; independent one-off Classes bypass this entirely and are created directly (Section 6.1); skips a School whose platform SubscriptionPlan is in the read-only degraded state (Section 10.2) — a lapsed School stops generating new bookable inventory rather than silently accumulating Classes nothing can book (straight fix, 25 Aug 2026, Pass 4)
class-cancellation-processing (new, confirmed 25 Aug 2026)	A School cancels a Class that has any Bookings at all — sold Class Pack/Friend Pass tickets, or Subscription/Weekly Pass/Trial Membership general-access Bookings	Always builds a list of affected credit-records — the Student's own sourceMembershipId entry plus one per guest attendee-list entry (Section 6.1) — for any Upcoming Booking on the cancelled Class that drew a Class Pack/Friend Pass credit (straight fix, 25 Aug 2026, Pass 4: excludes a Booking the Student already self-cancelled before the cutoff, Section 6.1 — its credit

Queue / job	Triggered by	Evidence in the designs
	(broadened 25 Aug 2026: the original wording covered ticket-holders only, leaving general-access Bookings on a cancelled Class to fall through into a false No-Show later; Trial Membership added to this bucket the same day, Pass 4 straight fix — it's non-consuming exactly like Subscription/Weekly Pass and had been left out)	was already credited back through that separate, pre-existing flow, so re-including it here would double-compensate); resolved 25 Aug 2026, Pass 4: the School's class-cancellation-policy (Section 6.1) is Auto-Refund (money back, via Stripe Refund API or a manual Cash/Bank ledger reversal, Section 10.2), Auto-Credit (the credit restored, no money moves), or Manual (the default — leaves the list for staff to resolve case-by-case, same pattern as the existing manual Cash/Bank Transfer transitions); a Friend Pass entry is always forced to Auto-Credit regardless of the School's policy, since it was never paid for and there's no money to refund; once a Booking's credit-record(s) resolve, its own status is set to Cancelled too (straight fix, 25 Aug 2026, Pass 4 — the same false-No-Show risk the general-access bucket below is guarded against, applied symmetrically here); separately (broadened 25 Aug 2026), notifies every Subscription/Weekly Pass/Trial Membership Booking on the Class — no credit involved, so no refund/credit list entry — and cancels those Bookings outright rather than leaving them to be wrongly marked No-Show once the Class's original end time passes; also expires/cancels any outstanding WaitlistEntry for the Class instead of letting a Notified entry's claim window run out against a Class that no longer exists
booking-no-show-expiry (new, logic review 25 Aug 2026)	A Class's end time (or QR-attendance-end date/time) passes with a Booking still Upcoming	Marks the Booking No-Show — closes the state machine so a credit spent at booking time is never left indefinitely reversible and Attendance KPIs stay accurate; does not touch StudentRank.classesAttended and does not refund the credit
waitlist-promotion-processing (new, confirmed 25 Aug 2026)	A Booking is Cancelled OR marked No-Show on a Full Class (freeing a seat — No-Show added 25 Aug 2026, closing a gap where a seat freed by a no-show was never offered to the waitlist), or a claim window expires unclaimed	Notifies the next Waiting WaitlistEntry and starts its claimByDeadline (School's waitlistClaimWindow, Section 6.1); on expiry without a claim, cascades to the next entry in position order; a Claim converts the entry into a real Booking, drawing a credit for the first time (Section 6.1)
translation-cache-invalidation	Translation content edited	translationsUpdate screen feeding a CMS-backed i18n layer
dashboard-kpi-aggregation	Scheduled (e.g. nightly/hourly)	Dashboard KPI cards (Attendance, Membership, Training Time, Booking) imply pre-aggregated reads rather than live joins
apple-developer-membership-renewal-reminder	Scheduled (daily check against each TenantAppConfig's Apple membership renewal date)	Section 5.5's escalating 30/7/day-of reminder requires a recurring check against Apple credential status, since a lapsed Developer Program membership pulls every app under it from sale
tenant-app-credential-grace-period-purge	Scheduled (e.g. daily)	Section 5.5's 90-day post-downgrade grace period requires a job to purge ULM8's own stored copy of a

Queue / job	Triggered by	Evidence in the designs
		School's Apple/Google credential once the window lapses without re-upgrade; the purge transactionally checks the credential's own re-upgrade timestamp immediately before deleting, so a same-day re-upgrade always wins over a same-day purge (resolved 26 Aug 2026, Pass 8, Decision 27)
franchise-fee-usage-reporting (new, confirmed 25 Aug 2026)	Scheduled (monthly, ahead of each Per-Headcount Franchise's billing cycle)	Section 6.1's Franchise.feeModel (Per-Headcount) and the new FranchiseFeeCharge entity require each School's active-student-count reported to Stripe as metered usage before the cycle's charge computes — mirrors Stripe's usage-based billing pattern rather than a bespoke invoicing job (Section 10.2); runs at a fixed T-1-day offset before the charge, with affiliation/subscription changes inside that window handled per Decision 35 (Section 12.1) rather than left to whichever number happened to be reported — generalized Pass 8, Decision 40, to also run for any School with an active whiteLabelApp entitlement, independent of Franchise.feeModel, reporting active-student usage on the same T-1-day fixed-offset schedule for that PlatformCharge case
account-deletion-processing (new — Section 12.1 Decision 44)	Enqueued by DELETE /users/me (Section 7) after re-authentication	No screen surfaces this directly — it's a backend cascade job, not a UI flow: WaiverSignature/StudentRank/Notification rows referencing the account are hard-deleted, while Booking/Transaction/Membership rows are anonymized (personal fields nulled, foreign key retained) rather than deleted outright, since a School's own financial and attendance records can't simply disappear out from under it; AuditLogEntry rows are retained, with Decision 43's subjectUserId left to go null on the deleted user's own prior entries. The retention window before this job runs, and any jurisdiction-specific variation, is the still-open Section 12.2 legal-review item — only the technical deletion/cascade mechanism is resolved here (Decision 44).
chargeback-pattern-restriction (new — Section 12.1 Decision 68)	Scheduled sweep across Disputed Transactions platform-wide	No screen surfaces this directly — it's a backend cross-tenant pattern job, not a UI flow: counts lost disputes per Student across every RoleGrant platform-wide and, past a threshold, restricts that Student to Cash/Bank Transfer payment going forward; on trigger, notifies the Student's current School(s) via the existing notification-fanout job. Never exposes cross-School data to any human view — only the resulting restriction, on a Student a School already has legitimate visibility into (Decision 68).

- Default retry/backoff/alerting policy — resolved 26 Aug 2026, Pass 8, Decision 23: every job above gets 3 attempts with exponential backoff and a failure alert on final exhaustion, except stripe-webhook-processing, which continues to lean on Stripe's own Smart Retries (Section 10.2) rather than duplicating them. This closes the gap where jobs like otp-delivery or franchise-fee-usage-reporting had no stated failure behavior at all — a silently-failed run previously had no path to anyone noticing.

10. Payments — Stripe Integration & Manual Methods

Per the locked decision in Section 1.2: Stripe is the only live payment gateway built; Cash and Bank Transfer are supported as manually-recorded ledger entries; GoCardless and PayPal (both present in the Figma payment-method screens) are explicitly out of scope.

10.1 What the designs show for payments

- Card capture (student, mobile): paymentMethod (saved-card list, set default) and paymentInformation (add card — holder name, card number, MM/YY, CVV, country, zip) — a standard Stripe Elements/Payment Method flow.
- Purchases: one-time (Class Pack — renamed from "Single Pass / Drop-In", Section 6.1; also covers Friend Pass and Weekly Pass, both added since this bullet was written), Trial, and recurring Subscription memberships; franchise-level plans with "Auto Renew" (straight fix, 25 Aug 2026 — Pass 4: this bullet is Figma-literal and had gone stale against Section 6.1's MembershipPlan updates).
- Per-tenant gateway configuration (School Portal): each School/Franchise enters its OWN Stripe Publishable Key + Secret Key (see stripePaymentInfo, "Stripe Keys" screens) — as designed, this is a direct multi-account "keys stored in our DB" model, not Stripe Connect. Section 10.4 below changes this for the build.
- Ledger: transactionsHistory shows separate Pending/Available balances per gateway account (Stripe, Cash, Bank Transfer — GoCardless excluded per decision; Bank Transfer added 25 Aug 2026, Pass 4 straight fix, since it's one of the three methods actually built, Section 10.2), and the canonical Dashboard's "Latest Transaction" table shows a Payment Method column with values like "Stripe - Credit Card" and "Cash".

10.2 Proposed backend flow

- PaymentsModule wraps the Stripe SDK; a School/Franchise onboards through Stripe Connect (Express or Standard) rather than pasting in a secret key, and PaymentsModule constructs a scoped Stripe client from the resulting Connected Account id (Section 10.4) — the addPaymentMethod / stripePaymentInfo screens are redesigned around this handoff instead of raw key entry.
- One-time purchases → PaymentIntent; Subscriptions → Stripe Subscription + Price objects mirroring each School's MembershipPlan records.
- Webhook endpoint (POST /payments/webhooks/stripe) verifies signatures and pushes events onto the stripe-webhook-processing BullMQ queue for idempotent, asynchronous handling (payment_intent.succeeded/failed, invoice.paid, invoice.payment_failed — added 25 Aug 2026, Pass 4, fired on each Stripe Smart Retry attempt, Section 9 — customer.subscription.updated/deleted, charge.dispute.created/updated/closed — added Pass 8, Decision 55); idempotency is enforced by deduping on Stripe's event id before processing, since Stripe may redeliver the same webhook more than once (straight fix, 25 Aug 2026 — Pass 4: 'idempotent' was asserted without saying how).
- Cash and Bank Transfer purchases skip Stripe entirely: a Transaction row is created with method=CASH|BANK_TRANSFER and status=Pending, referencing the MembershipPlan being purchased but creating no Membership row yet (Section 6.1). School Owner/Manager confirms payment via PATCH /transactions/{id}/confirm (Section 7), which atomically transitions the Transaction to Successful — not a separate "Paid" value, the real enum is Successful/Pending/Failed/Refunded/Disputed (though only Stripe-paid Transactions can ever reach Disputed, added Decision 55) — and creates the resulting Membership Active (rewritten 26 Aug 2026, Pass 5, replacing the earlier status-flip model and correcting a stale "PAID" reference that never matched the actual enum).
- Platform SubscriptionPlan billing (resolved, 25 Aug 2026 — Pass 4): ULTM8 charges the Franchise/School directly for platform access via SubscriptionPlan — Stripe Subscription + Price objects, the same pattern already used for MembershipPlan (Section 6.1) — not a plan the Franchise resells onward to its Schools. See Section 6.1, Section 12.1.

- Franchise-fee collection — Stripe-automated (resolved, 25 Aug 2026 — Pass 4, supersedes an earlier manual/invoice-first recommendation after review): a Franchise's fee from its member Schools (Franchise.feeModel — Flat or Per-Headcount, Section 6.1) is charged automatically via Stripe, not invoiced manually — a new FranchiseFeeCharge entity (Section 6.1) records each charge. Flat fee: a standing Stripe Subscription against the School's saved payment method, mirroring Membership billing exactly. Per-Headcount fee: Stripe metered/usage-based billing — the new franchise-fee-usage-reporting job (Section 9) reports each School's active-student-count to Stripe monthly, and Stripe computes the charge from the Franchise's configured per-head price. Either way, the charge is a Stripe Direct/Destination charge against the School's saved payment method that settles directly into the Franchise's own connected PaymentAccount, not the platform's balance — structurally the same Direct/Destination-charge pattern Membership billing already uses for Student→School, aimed instead at School→Franchise — the same destination-charge structure Section 10.4's Stripe Connect model relies on to keep the platform out of the funds-custody chain, though whether that structure actually satisfies money-transmitter exemptions in every jurisdiction ULTM8 operates in is still pending legal confirmation (Section 12.2), not a closed question. ULTM8 takes no cut of this charge — it passes through untouched; ULTM8's own revenue comes from its separate platform and white-label SubscriptionPlan billing. The card network's own FX handles any currency mismatch between the School's card and the Franchise's payout currency for this flow specifically; the broader multi-currency policy for MembershipPlan/SubscriptionPlan pricing itself is resolved 26 Aug 2026, Pass 5, matching this exact pattern for ULTM8's own revenue and staying independent per-currency for Student-facing MembershipPlan pricing (Section 11.2, Section 12.1).
- White-label add-on pricing — ULTM8 to School, metered only, no flat fee (corrected 25 Aug 2026 — Pass 4, same day as first drafted): the branded-app upgrade (Section 5.3) is billed as a single metered \$0.99–\$1.99/active-student/month component — exact rate still to be set, Section 12.2 — charged by ULTM8 to the School (or the Franchise, where it manages rollout centrally), reported the same way as the Per-Headcount Franchise fee above. The payer decides privately whether and how to recoup this from Students — pass-through, embedding it in membership pricing, or absorbing it; no Student-facing 'app charge' feature is built.
- Failed/declined Franchise-fee and white-label-fee payments are handled by the same recurring-payment failure mechanism as Membership billing (webhook-driven; dunning/grace-period timing is a separate open decision for this pass, governing all three alike).
- Refunds — Stripe Refund API for money, a separate Auto-Credit path for class credit (resolved 25 Aug 2026, Pass 4): a cancelled Class's affected Class Pack/Friend Pass Bookings are resolved one of two mutually exclusive ways, matching the School's class-cancellation-policy (Section 6.1). Auto-Refund gives the money back — Stripe Refund API for anything card-paid, a manual Cash/Bank Transfer ledger reversal (same staff-driven pattern as the confirm-to-Successful transition, Section 7; corrected 26 Aug 2026, Pass 5 — "PAID" was never a real status value) for anything that never touched Stripe — and the spent credit is not also restored, since that would double-compensate. Auto-Credit restores the spent credit to the Membership's classesRemaining instead and moves no money at all — this is the same mechanism already used when a Student cancels their own Booking before the Class's cancellation-charge cutoff. A Friend Pass entry is always resolved via Auto-Credit regardless of the School's policy setting, since it was gifted at £0 and there's no money to give back. Manual policy (the default) leaves both choices to staff, per ticket-holder.
- Partial refunds and idempotency: an unscoped multi-class Class Pack (5/10-class tiers) only had one credit spent on the cancelled Class, not the whole pack, so Auto-Refund on it refunds price ÷ classesIncluded, not the full amount — the customer keeps their remaining credits. Transaction.refundedAmount accumulates across multiple such partial refunds against the same purchase (several of its classes could be cancelled separately over time) and is validated against the original amount before each new refund is allowed through. Since a single Booking can carry several independent credit-records (the Student's own plus one per guest, Section 6.1), the Stripe Refund API call's idempotency key is derived from (Booking id, membershipId) rather than the Booking id alone — straight fix, 25 Aug 2026, Pass 4 — and both POST /payments/refund and POST /bookings/{id}/credit-restore (Section 7) target and act on one named credit-record the same way. Both endpoints also only ever act on a credit-record the class-cancellation-processing job has already flagged pending (refundResolution still null); a repeat call — a BullMQ retry on the Stripe path, or on the cash/bank

ledger reversal, which has no Stripe-style idempotency key of its own — finds the record already resolved and no-ops instead of double-processing it.

- Who can authorize which path (Section 8.2): Auto-Credit is a narrow, no-money action, so it's Branch Staff-accessible, same shape as their existing membership-status-check endpoint. Auto-Refund moves real money out, so it stays School Owner/Manager-only — the same separation most POS systems draw between taking a sale and authorizing a refund. A FranchiseFeeCharge refund follows the same money-only-with-authority principle, but the authorizing party is the Franchise Owner, not the School, since the Franchise's own PaymentAccount is the one holding the money.
- Recurring-payment retry and dunning mechanism (resolved 25 Aug 2026, Pass 4): every recurring charge in this document — Membership Subscription renewals, Franchise-fee Flat and Per-Headcount billing, and the platform/white-label SubscriptionPlan — already runs on a Stripe Subscription object, so retries lean on Stripe's own configurable retry schedule (Smart Retries) rather than a custom BullMQ retry scheduler; ULTM8 reacts via the existing webhook queue instead of re-implementing retry logic. How much control ULTM8 has over that schedule depends on Connect account type (Express: ULTM8 configures it programmatically per Connected Account at onboarding, Section 10.4; Standard: the account holder's own Stripe Dashboard settings apply, and ULTM8 can only set default guidance) — this doesn't resolve the Express-vs-Standard question already open in Section 12.2, it raises the stakes of resolving it, since retry-schedule control now depends on the answer too.
- Subscription-type Membership renewal failure (resolved 25 Aug 2026, Pass 4): reuses the exact same mechanism already confirmed for voluntary cancellation (Section 6.1) — the Membership stays Active while the underlying Stripe subscription is past_due, however long the configured retry schedule runs, and only Expires (triggering the identical same-day Booking sweep) once Stripe exhausts retries and cancels it. A failed charge and a voluntary cancellation land on the Student identically. Requires Stripe as the payment method for any Subscription-type MembershipPlan (Section 6.1) — auto-recurring billing has no Cash/Bank Transfer equivalent. invoice.payment_failed (added to the webhook list above) feeds notification-fanout so the Student is warned during the retry window, not just silently locked out at the end.
- Franchise-fee failure (resolved 25 Aug 2026, Pass 4): no School-side lockout — ULTM8 isn't party to the Franchise-School commercial relationship beyond facilitating the charge, so enforcement (chasing payment, terminating affiliation) stays the Franchise's own, off-platform call. Instead, a new franchiseFeeSubscriptionStatus field on School (Active / Past Due / Canceled, Section 6.1) is kept synced via the same customer.subscription.updated/deleted webhook and is visible to both the School itself and the Franchise Owner's existing roster (Section 8.2) — distinct from FranchiseFeeCharge's own per-charge Successful/Pending/Failed/Refunded/Disputed history, which tracks each individual charge, not the ongoing subscription's health. invoice.payment_failed also notifies the School via notification-fanout.
- Platform SubscriptionPlan and white-label failure (resolved 25 Aug 2026, Pass 4): billed as two separate Stripe Subscription objects — core platform access and the whiteLabelApp metered add-on — so a billing issue in one can't take down the other (Section 6.1). The white-label subscription stays School-scoped even when “the Franchise manages rollout centrally” pays it (only the payment method charged changes, mirroring the Direct/Destination-charge routing this document already uses for Franchise-fee and Membership billing) — so a failure only ever freezes one School's branded app, never cascades. On cancellation it reuses Section 5.5's frozen-at-last-build behavior unchanged; whiteLabelApp itself becomes a value synced from its own subscription's status rather than a manually-set flag. On the core platform SubscriptionPlan being cancelled, the subscribing tenant's portal access degrades to read-only — full view of existing data, no new Classes/Bookings/payments — rather than deletion or forced logout; a School's own failure degrades only that School, and a Franchise's own failure degrades only the Franchise Owner's own actions (profile/branding/feeModel edits), never cascading to member Schools' own, separately-paid access (Section 6.1). class-occurrence-generation (Section 9) skips materializing new Class rows for a degraded-state School, so “no new Classes” holds even for jobs, not just user actions.
- Refund-API failure (resolved 25 Aug 2026, Pass 4 — closes the item deferred from the refund/credit-mechanism decision, Section 12.1): if a Stripe Refund API call itself errors — declined, a stale charge, insufficient Connected Account balance — the affected credit-record falls back into the existing Manual

bucket (Section 6.1, Section 10.2) with an explicit failed-attempt flag and reason recorded, plus a staff notification via notification-fanout, rather than silently stalling indistinguishable from a credit-record nobody has attempted yet.

- Synchronous checkout failure — resolved 26 Aug 2026, Pass 8, Decision 29: if the synchronous PaymentIntent-creation call to Stripe itself errors (network, Stripe-side degradation) rather than the charge being declined, the Student sees a plain retryable error at checkout — no charge is queued or assumed, and no Transaction row is created until Stripe actually responds. This is distinct from, and doesn't change, the already-specified async webhook/dunning path above for a charge that Stripe did accept.

10.3 Explicitly out of scope (per decision)

- GoCardless direct-debit integration (Access Token / Client Id / Client Secret / Webhook fields were designed but will not be built).
- PayPal integration (present only as a row in the Payment Information table, no dedicated config screen was found).
- In-App Purchase note (clarifying, 25 Aug 2026 — Pass 4): every Stripe-charged purchase in this document — Membership billing, Franchise fees, the white-label add-on — is for a real-world service (martial-arts class access, franchise affiliation, a branded app upgrade), not a digital good consumed inside the app, which is the category Apple's and Google's mandatory in-app-purchase policies actually target; this should still be confirmed against each store's current guidelines before launch rather than assumed, since enforcement of that boundary has shifted over time.

10.4 Security note — payment-credential custody (decided, revised from the original recommendation)

The designs show each tenant's raw Stripe secret key being entered directly into the app. The original version of this document flagged that as a weaker security model than Stripe's own recommended multi-tenant pattern and listed it as an open question. Following the follow-up architecture discussion in Section 4, this is now a decision, not an open question:

- **Primary path — Stripe Connect:** Schools and Franchises onboard via Stripe Connect (Express or Standard). ULTM8 stores only a Connected Account id per PaymentAccount; the platform never receives, stores, or decrypts a tenant's raw secret key at any point. This removes an entire class of risk (a database compromise can no longer expose live payment credentials for every tenant) rather than mitigating it with encryption. Added stakes, 25 Aug 2026, Pass 4: which of Express or Standard a tenant onboards through also now determines who controls that tenant's recurring-payment retry/dunning schedule (Section 10.2, Section 9) — Express lets ULTM8 configure it programmatically; Standard leaves it to the account holder's own Dashboard. This doesn't resolve the Express-vs-Standard question already open in Section 12.2, it's a further reason to resolve it.
- **Fallback — secrets manager:** For any credential that genuinely can't move through Connect (to be confirmed against Stripe's actual onboarding constraints for this business type), the fallback is a dedicated secrets manager — AWS Secrets Manager specifically, resolved 26 Aug 2026, Pass 5 alongside the rest of the AWS-native environments decision (Section 11.6) — rather than an application-level encrypted database column — access is scoped, rotatable, and independently audited from the application's own database.

This also directly shapes Platform Admin: because credentials are Connect account ids or secrets-manager references rather than plaintext-recoverable keys, no Platform Admin role — including "Full" admin — has a code path that returns a decrypted tenant secret key. See Section 4.5 for how this integrates with the Platform Admin isolation architecture.

11. Non-Functional Considerations

11.1 Internationalization & localization

- 4 languages observed: English (UK), Portuguese (BR), Spanish, Arabic. Arabic implies RTL layout support on both React and React Native, which is a real UI engineering cost not otherwise called out in the stack decisions.
- Content is CMS-backed (translationsList/Create/Update/Detail), not static JSON — the backend must serve translation content dynamically, with caching (Redis) and invalidation (BullMQ job, Section 9). Resolved 26 Aug 2026, Pass 5: i18next on both React and React Native, backed by i18next-http-backend rather than the more common static-bundle pattern, so it actually serves this CMS-backed content dynamically; a version-stamp field on cached translations lets the existing invalidation job's Redis bust reach already-open sessions, not just new ones. RTL for Arabic is two separate mechanisms, not one: React (web) uses CSS logical properties (margin-inline-start and equivalents); React Native has no CSS at all, so it needs `I18nManager.forceRTL/allowRTL` plus start/end layout props instead — and switching direction on Native requires an app relaunch, a real platform limitation, not an implementation gap. Full detail in Section 12.1.

11.2 Multi-currency

- 6 currencies observed: GBP, EUR, USD, BRL, AED, MYR. The designs do not show whether prices convert automatically between currencies or are entered independently per currency by each school. Resolved 26 Aug 2026, Pass 5: Student-facing MembershipPlan pricing stays independent per-currency, School-set, with no automatic conversion — a School prices in its own local currency and that is what a Student pays, full stop. ULTM8's own revenue — the platform SubscriptionPlan, the white-label add-on, and Franchise-fee — all quote and bill in a single anchor currency (USD, matching the white-label add-on's \$0.99-\$1.99/active-student/month reference range — exact rate still to be set, Section 12.2), with the card network's own FX handling conversion at charge time; this matches the pattern Franchise-fee already established (Section 10.2), rather than introducing a new one. Transaction/Invoice gains a currency field to record which currency each charge actually settled in (Section 6.1). Full detail in Section 12.1.

11.3 Media & file storage

- Banner images (classes, memberships), profile/gallery photos, and signed-waiver / invoice PDFs all need persistent, servable storage. Resolved 26 Aug 2026, Pass 5: Cloudflare R2 (S3-compatible object storage with zero egress fees — roughly \$15/month vs. a comparable ~\$900/month on S3 at 1TB stored/10TB served, the read-heavy traffic pattern a public banner/gallery-photo library produces), paired with Cloudflare Images as a companion decision for on-the-fly resizing and thumbnailing, since R2 itself has no image-transformation feature. Cloudflare Images' own pricing is separate from R2's: \$5 per 100,000 images stored/month, \$1 per 100,000 images delivered/month, plus \$0.50 per 1,000 unique transformations/month beyond the first 5,000 free — small at this platform's likely scale, but additive to R2's own cost and worth carrying as its own line item rather than folding into the R2 estimate above. Signed-waiver and invoice PDFs are served from R2 directly, no transformation needed. This still leaves data-residency/GDPR/LGPD compliance open across this platform's markets — see Section 12.2. A brief R2 outage blocking waiver/invoice viewing is accepted as a documented risk rather than built around, given R2's own high published uptime — resolved 26 Aug 2026, Pass 8, Decision 36. Full detail in Section 12.1.

11.4 Notifications

- Push (mobile), email (web has an "Email notifications" settings screen), and SMS (OTP) are all implied. Resolved 26 Aug 2026, Pass 5: push goes direct to FCM (Android)/APNs (iOS) rather than through a third-party aggregator, since BullMQ already drives notification fan-out server-side; SMS/OTP uses Twilio Verify specifically (not Twilio's generic Messaging API) — UAE currently has alphanumeric sender-ID registration on

regulatory hold and Malaysia requires domestic sender-ID pre-registration that gets overwritten with a random operator-assigned numeric ID regardless (so registered branding never actually displays to the recipient), but Verify's managed routing still delivers OTPs reliably in both markets; email uses Postmark for transactional delivery, worth a monitoring note given 2026 reports of abrupt account suspensions post-acquisition; AWS SES is the named fallback (resolved 26 Aug 2026, Pass 8, Decision 34), with a failed send routed to a dead-letter queue and surfaced to Platform Admin rather than silently dropped. A Twilio Verify outage no longer strands every login/registration/recovery flow at once: a failed SMS-OTP send falls back to an email-delivered OTP through the same Postmark/SES path (resolved 26 Aug 2026, Pass 8, Decision 33). Full detail in Section 12.1.

11.5 Security

- 6-digit passcode as the primary credential — needs explicit lockout/rate-limiting policy regardless of how Section 12's open question resolves.
- Payment credentials are never held by ULTM8 as raw, decryptable secrets — Stripe Connect account ids and AWS Secrets Manager references only (Section 4.5, Section 10.4).
- Tenant data isolation is enforced at the database via PostgreSQL Row-Level Security, not only in application code (Section 4.3).
- Platform Admin staff authenticate against a separate identity realm (SSO + mandatory 2FA), distinct from the customer-facing AuthModule, with least-privilege sub-roles rather than one Admin bucket (Section 4.4).
- Every Platform Admin action against tenant data is written to an immutable audit log (Section 4.7), and cross-tenant data isolation is enforced by a required CI test suite, not code-review alone (Section 4.8). Class Pack / Friend Pass credit decrement at booking time (Section 6.1) must be an atomic, transactionally-locked operation — two simultaneous booking requests against the same low-credit Membership could otherwise both pass an eligibility check before either decrements, overselling a single credit twice (logic review, 25 Aug 2026). The same atomicity requirement applies to a WaitlistEntry Claim (Section 9), which draws a credit for the first time through an identical decrement path, and to the Class capacity/Full check itself (Section 6.1) — two Students requesting the last open seat at the same instant must be serialized the same way, or a Class can be overbooked independently of any credit logic (logic review, 25 Aug 2026).
- A School's App Store Connect / Google Play credentials, once collected for the white-label app upgrade, are held only in AWS Secrets Manager (never the application database) and are scoped to the MobileAppPublishingModule's own service identity — same custody model as Stripe Connect credentials (Section 4.5, Section 5.4).
- Permissions screen (mobile) requests Device Location, Push Notification, Email Notification, Phone Contacts, Google Calendar, Device Storage, and Camera & Media Library access — each needs a corresponding OS-level permission flow and a documented purpose (particularly Phone Contacts and Google Calendar, whose in-app use isn't shown on any other screen — flagged in Section 12).
- Session/token revocation (Section 8.3) and in-tenant role permissions (NestJS Guards + @Roles(), Section 8.3) are separate, already-specified mechanisms neither previously cross-referenced here: revocable, rotating access/refresh tokens with reuse detection bound customer sessions to a 15-minute TTL (5 minutes for Platform Admin) — an accepted, not eliminated, window on a live permission downgrade — while Guards/@Roles() enforce in-tenant role permissions, distinct from PostgreSQL RLS's tenant/Branch scoping (Section 4.3). RoleGrant creation is already gated through this same Guards/@Roles() pattern. class-validator DTO validation (Section 3) is the platform's injection-protection baseline on every NestJS endpoint.
- Owner-tier account protection and abuse resistance — resolved 26 Aug 2026, Pass 7: rather than extending mandatory login MFA to School Owner/Manager and Franchise Owner (a largely non-technical, self-service Owner persona where blanket 2FA adds real per-login friction — the same friction-vs-risk tradeoff behind excluding GoCardless, Section 12.1), step-up MFA is required only before sensitive actions — payout/bank-detail changes, Auto-Refund, franchise-fee refunds — matching the risk these roles actually carry. Failed login/OTP-verify attempts get a per-account escalating lockout, decoupled from the existing per-IP rate

limiting (above); Twilio Verify's Fraud Guard is enabled and the OTP-send endpoint itself gets a send-rate limit, closing the SMS-pumping/toll-fraud gap alongside the brute-force one. Resolves the MFA question left open in Section 12.2.

- Client-side token custody — resolved 26 Aug 2026, Pass 7: access/refresh tokens are stored in httpOnly, Secure, SameSite cookies for the web apps (School Portal, Platform Admin) and the OS keychain/keystore on mobile; localStorage is explicitly ruled out, since an XSS vulnerability anywhere in either web app would otherwise be a direct path to full account takeover, defeating the refresh-token rotation/reuse-detection model above, which assumes a token isn't trivially exfiltrable.
- Encryption baseline — resolved 26 Aug 2026, Pass 7: RDS storage is encrypted via AWS KMS — a create-time-only setting that must be enabled before the first deploy, not retrofitted later without a snapshot-restore migration — R2 storage is encrypted at rest, and TLS 1.2+ is enforced on all traffic. Stated explicitly as a non-negotiable baseline rather than left to default configuration, since neither encryption at rest nor TLS was specified anywhere in this document until now.
- Audit-log immutability — resolved 26 Aug 2026, Pass 7: enforced at the database layer, not the application layer alone — UPDATE and DELETE are revoked on the AuditLogEntry table (Section 4.7) for every role, including the break-glass credential (Section 4.3), which is provisioned as a non-superuser, non-table-owner role specifically so this restriction can't be bypassed even during emergency access. Exact retention period is a compliance/legal input, not an engineering one — left open, Section 12.2.
- Ownership validation on purpose-built non-RLS endpoints — resolved 26 Aug 2026, Pass 7: membership-status-check (Section 8.2) and the Franchise Owner School-roster read (GET /franchises/{id}/schools, Section 8.2) exist specifically to bypass table-level RLS, so each now independently validates its path-parameter target against the caller's own RoleGrant scope before returning a result — a check the CI cross-tenant isolation suite (Section 4.8) doesn't cover, since that suite tests RLS specifically, not bespoke-endpoint ownership. A parallel CI check is added for this endpoint family. credit-restore is unaffected — it already enforces the same Booking-level branch_id RLS as Branch Staff's other Booking access (Section 8.2).
- Rate limiting extended to booking/waitlist/credit endpoints — resolved 26 Aug 2026, Pass 7: the same per-user/per-IP throttling pattern used on /auth/otp/* and /auth/login (above) now also covers the class-booking, waitlist-claim, and credit-restore endpoints. The atomicity fixes above close the race-condition/overselling bug; this closes the separate volumetric-abuse risk (queue-flooding a popular class, credit-probing) the atomicity fix doesn't touch.
- CI/CD hardening — resolved 26 Aug 2026, Pass 7: Dependabot is enabled as a required GitHub Actions status check alongside the existing RLS cross-tenant-isolation gate (Section 4.8, Section 11.6); CodeQL runs in advisory, non-blocking mode rather than gating merges, avoiding the false-positive triage burden a small team can't sustain as a merge blocker. GitHub Actions authenticates to AWS via OIDC federation to a scoped IAM role — no static, long-lived IAM access keys stored as GitHub secrets.
- Security alerting — resolved 26 Aug 2026, Pass 7: CloudWatch alarms fire on a fixed-threshold count of auth-failure/lockout events and on booking/credit-claim attempts exceeding a fixed per-user/per-IP rate — concrete, fixed-threshold triggers rather than open-ended anomaly detection, riding on the AWS stack already chosen (Section 11.6). This is the only place customer-facing abuse becomes visible; the Platform Admin audit log (Section 4.7) covers admin actions only.
- Payment-card scope — resolved 26 Aug 2026, Pass 7: ULTM8's payment UI uses Stripe-hosted fields/redirect only, for both Stripe Connect onboarding and checkout — no raw card, bank, or account-verification data ever touches ULTM8's own frontend or backend — targeting PCI SAQ-A, consistent with the Stripe Connect custody model already committed to in Section 10.4.
- App Store Connect / Google Play credential blast radius — resolved 26 Aug 2026, Pass 7: unlike Stripe (moved to Connect specifically to remove this single-point-of-compromise risk, Section 10.4), the MobileAppPublishingModule's service identity holds every School's scoped app-store credential centrally (Section 5.4) with no equivalent per-tenant isolation of the pipeline itself — a compromise there could reach every School's live app at once. Compensating controls: a fixed rate limit on build/submission volume per

tenant per day, and the dual-approval pattern already required for store-side unpublish (Section 5.5) is extended to abnormal or bulk publish actions too.

11.6 Environments & delivery

- No dev/staging/production environment strategy, CI/CD approach, or hosting target was specified in the confirmed stack or shown in the designs. Resolved 26 Aug 2026, Pass 5: AWS — RDS for PostgreSQL, ElastiCache for Redis, ECS Fargate for the NestJS API and BullMQ workers — with GitHub Actions for CI/CD, gated by a required GitHub branch-protection status check running Section 4.8's automated cross-tenant isolation test suite before merge. This also resolves the Vault-vs-AWS-Secrets-Manager shortlist named in Section 4.5, Section 5.4, and Section 10.4, in favor of AWS Secrets Manager specifically. Backup/DR — absent elsewhere in this document despite this being a payments- and audit-log-bearing platform — is RDS automated backups with point-in-time recovery, Multi-AZ. Data-residency/GDPR/LGPD compliance across this platform's markets remains a separate, still-open legal-review item — Section 12.2. Full detail in Section 12.1.
- Data-layer scale hardening — resolved 26 Aug 2026, Pass 8, Decision 30: every RLS-scoped table gets a composite index leading with its tenant column plus the columns actually filtered/sorted on (status, date) — a migration convention, not new infrastructure. AuditLogEntry is range-partitioned by month on created_at from the start, independent of the still-open exact retention duration (Section 12.2) — the only way to make future archival possible once UPDATE/DELETE are revoked (Section 11.5). RDS Proxy sits in front of RDS with a per-Fargate-task connection-pool ceiling, so a booking-rush autoscale event can't exhaust RDS's own connection limit. ElastiCache Redis gets an explicit eviction policy (volatile-ttl/allkeys-lru, not the default noeviction, which fails writes outright) plus BullMQ's removeOnComplete/removeOnFail so completed-job data doesn't silently fill a small node.
- Observability & tracing — resolved 26 Aug 2026, Pass 8, Decision 31: beyond Pass 7's security-specific alarms, a general CloudWatch dashboard covers Fargate/ALB error rate, p95 latency, and RDS CPU/connections, backed by a /health endpoint on each service. Bull Board (admin-authenticated, read-only) gives manual visibility into all 18 BullMQ queues, plus a CloudWatch alarm on failed-job count and oldest-waiting-job age per queue. Every request carries an X-Request-Id, propagated into BullMQ job data, with structured JSON logging (e.g. pino) throughout the NestJS → BullMQ → Stripe-webhook chain — so a support question ('why did School X see wrong data at 3am') is traceable, not a cold grep across three systems.
- Deploy rollback & migration discipline — resolved 26 Aug 2026, Pass 8, Decision 32: Prisma is the chosen ORM, resolving the open item in Section 12.2 — better NestJS ecosystem fit and more mature migration tooling than the alternative. A bad deploy's standard revert path is redeploying the previous Fargate task-definition revision; schema changes follow an expand/contract convention (additive-first, backward-compatible within a release, destructive cleanup only in a later one) as a stated house rule, since not every Postgres migration is cleanly reversible and this is a payments- and audit-log-bearing platform.

12. Gaps & Open Questions

Everything below is either undesigned, inconsistent, or ambiguous in the Figma files, or a technical convention the confirmed stack does not specify. Nothing in this section has been decided on your behalf — recommendations are offered only as a starting point for your decision.

12.1 Decisions already made this session (for the record)

- Payment scope: Stripe + manual Cash/Bank-Transfer only; GoCardless and PayPal out of scope.
- App boundary (revised 21 Aug 2026): School Portal and Platform Admin are TWO separately deployed applications in a shared monorepo from day one — not one RBAC-gated app. Full architecture in Section 4.
- Canonical design version: the newer dashboard/KPI-driven screen set, where a feature exists twice.
- Tenancy: Franchise → Schools → Branches, with independent (non-franchised) Schools also supported.
- Payment-credential custody (new, 21 Aug 2026): move to Stripe Connect as the primary model, with a secrets manager as fallback — UTM8 does not custody raw tenant secret keys. Full detail in Section 10.4.
- Tenant isolation (new, 21 Aug 2026): enforced at the database via PostgreSQL Row-Level Security, with a required CI test suite asserting cross-tenant access is impossible. Full detail in Section 4.3 and Section 4.8.
- Platform Admin identity (new, 21 Aug 2026): a separate identity realm from customer accounts, with SSO, mandatory 2FA, and least-privilege sub-roles rather than a single Admin bucket. Full detail in Section 4.4.
- White-label mobile app distribution (new, 21 Aug 2026): every School is included in a single shared "UTM8" Student app by default; a paid SubscriptionPlan upgrade adds a School's own branded app on both stores. Full detail in Section 5.
- White-label publish ownership (new, 21 Aug 2026): UTM8 runs the build/submit pipeline on each School's behalf via a scoped, revocable API key/service-account credential rather than interactive account access — Schools do not self-serve releases. Full detail in Section 5.4 (revised, Pass 3, 25 Aug 2026 — see below).
- White-label downgrade policy (new, 21 Aug 2026): a branded app is frozen at its last build on downgrade (not unpublished), with a 90-day credential grace period before UTM8 purges its own stored copy; full cancellation gets a graceful in-app degraded state instead. Full detail in Section 5.5 (revised, Pass 3, 25 Aug 2026 — see below).
- White-label build timing (new, 21 Aug 2026): the publishing pipeline is built alongside the initial platform, not deferred until the first paying upgrade, so the branded-app option is a feature Schools are told UTM8 offers from launch. Full detail in Section 5.4.
- Grading/ranks data model & flows (new, 24 Aug 2026): a working, tested reference implementation now backs the Rank and StudentRank entities and validates stripe-tier progression, skill-gating, years-in-rank gating for Black Belt and above, promotion/downgrade/bulk-grading with an auditable rank history, and a coach-facing Grading Board. Full detail in Section 2.3; expands Section 6.1 and Section 7's RanksModule.
- Class ↔ Timetable & attendance-to-grading linkage (new, 24 Aug 2026): a Class may either be generated from a recurring TimetableSlot or created directly as an independent one-off event; a QR scan at check-in marks the matching Booking Completed, and that Completed status is what increments StudentRank.classesAttended toward the next grade — there is no separate Attendance entity. Full detail in Section 6.1, Section 7, and Section 9.
- Membership → booking → grading → monthly billing loop (confirmed, 25 Aug 2026): an active Membership gates a Student's ability to book Classes; those Classes are linked to curriculum/skills; Booking completion drives grading progress; and the whole loop resets each month when the Membership's Stripe subscription renews — continued access depends on that renewal succeeding. Full detail in Section 1.3, Section 6.1, and Section 10.2.
- One-off Class monetization — Class Pack / Weekly Pass (revised 25 Aug 2026): a single-class ticket is a Class Pack MembershipPlan with classesIncluded=1 and a scoped Class set to that specific one-off Class; the same

Class Pack type also covers the upsell of buying more classes for more money — School-configurable higher-credit tiers (e.g. 5-class, 10-class) left unscoped for general access across any regular Class. Weekly Pass remains a separate time-boxed-unlimited type. Either way a Membership is created carrying classes-remaining and/or an expiry date instead of a recurring billing cycle, and the existing Booking-requires-Active-Membership rule (Section 6.1) governs access — a scoped pack additionally restricts which Class can be booked. Full detail in Section 6.1.

- Friend Pass — bring-a-guest is not free by default (confirmed, 25 Aug 2026): a Student's own Membership never extends to a guest in their Booking's attendee list. A guest is only covered if the School explicitly gifts a Friend Pass — a new MembershipPlan type, always £0, reusing the same scoped-Class mechanism Class Pack already has, so the School can scope a gift to one specific Class or leave it general, per gift. Full detail in Section 6.1.
- Subscription cancellation/downgrade timing (confirmed, 25 Aug 2026): cancelling runs out the current paid period — no proration, no immediate cutoff. The Student keeps access through what they already paid for, then the Membership Expires when the period ends. A downgrade to a cheaper plan takes effect at the next renewal, not immediately. Uses Stripe's native `cancel_at_period_end` flag and the `customer.subscription.updated/deleted` webhooks already listed in Section 10.2 — no new payment engineering required. Full detail in Section 6.1 and Section 10.2.
- **Logic review, 25 Aug 2026 — Class Pack credit timing was a real bug:** as previously specified, a credit was only spent when a Student actually attended (QR scan), so a no-show never consumed anything — a 1-class pass could be re-booked indefinitely just by not showing up, and a Class could be phantom-overbooked at no cost to anyone. Fixed: credits (and the booked seat) are now reserved the moment a Booking is created; cancelling before the Class's existing cancellation-charge cutoff (Section 6.1) refunds it, a No-Show does not. Full detail in Section 6.1 and Section 9.
- **Logic review, 25 Aug 2026 — Booking now tracks its source Membership, and free access is spent first:** Booking gained a `sourceMembershipId` field — without it there was no way to know which credit to decrement or refund. When a Student holds more than one Active Membership, a general Subscription/Weekly Pass is now checked before any paid Class Pack/Friend Pass credit, so a Student is never accidentally charged a credit they didn't need to spend. Full detail in Section 6.1.
- **Logic review, 25 Aug 2026 — waiver sign-off is now wired into booking eligibility:** Class already carried a `terms/waiver-required` flag and `Waiver/WaiverSignature` already existed as entities, but nothing connected them — a Student could book a waiver-required Class without ever signing one. Booking creation now also requires a `Signed WaiverSignature` when that flag is set. Full detail in Section 6.1.
- **Logic review, 25 Aug 2026 — No-Show state and atomic credit decrement:** Booking's status enum gained `No-Show`, set by a new background job (`booking-no-show-expiry`, Section 9) once a Class ends with no QR scan, closing a state-machine gap where an unattended Booking simply stayed `Upcoming` forever. Separately flagged in Section 11.5: the credit decrement itself must be an atomic, row-locked operation, or two simultaneous booking requests against a low-credit Membership could both succeed and oversell a single credit.
- **Rank-gating on Class booking (confirmed, 25 Aug 2026):** reuses Rank's existing `eligibleClassTypes` field rather than new schema — it now governs both grading credit AND booking eligibility, cumulative by ladder order (a rank inherits every class type eligible at any rank below it in that discipline). Hard block by default; an Instructor/Staff member can override per Booking (`overriddenBy` + `overrideReason`, always recorded). The cumulative-by-order semantics is this review's proposal answering your question, not something you specified outright — flagging for correction if that's not the intent. Full detail in Section 6.1.
- **Correction, 25 Aug 2026 — rank-gating moved from belt-level to stripe-level:** the bullet above assumed all stripes within a belt share one eligibility list — wrong, per your example that a White 3-stripe commonly has Blue Belt class access most White 0-stripe Students don't. `eligibleClassTypes` now lives on each stripe tier individually, `StudentRank` gained a `currentStripe` field to track exactly where a Student sits, and the cumulative order flattens belt+stripe into one sequence per discipline (White-0...White-3, Blue-0...) so a School can let an advanced stripe reach into the next belt's classes directly. `PromotionEvent` also gained a

Stripe Award type so stripe increments get their own accurate history entry instead of being recorded as belt promotions. Full detail in Section 6.1.

- **Weekly Pass corrected to non-consuming (confirmed, 25 Aug 2026):** Membership's own field list previously said Weekly Pass decremented classesRemaining like a Class Pack, directly contradicting the free-access-first booking rule that already called it non-consuming, and the earlier one-off-monetization decision that called it "time-boxed-unlimited." Weekly Pass never sets a credit count — it grants the same unlimited access as a Subscription, just governed by an expiry date instead of a recurring billing cycle. Full detail in Section 6.1.
- **Stripe grading has no auto-promotion, ever (confirmed, 25 Aug 2026):** a Stripe Award and a belt Promotion are always two separate, coach-initiated decisions — the system never infers one from the other, including when a Student reaches a belt's highest configured stripe tier (stripe counts are School-set per belt, not a fixed number like 3). At that point Stripe Award simply isn't offered; the coach's next action is a manual Promotion. What IS automatic is class access following the grading: the instant currentStripe changes, booking eligibility updates immediately off that stripe tier's eligibleClassTypes, with no separate unlock step. Full detail in Section 6.1 and Section 7.
- **Grading progress resets at every checkpoint, not per belt (confirmed, 25 Aug 2026):** each stripe and each belt is graded as its own complete stage of the journey, not a fraction of one running belt-level count. classesAttended, per-skill sign-off, and the computed readiness bucket/progress % all reset to the CURRENT checkpoint in the flattened belt+stripe sequence — reaching White-1 does not carry credit toward White-2. currentStripe likewise resets to the new Rank's first stripe tier on every Promotion or Downgrade. Full detail in Section 6.1.
- **Friend Pass capped at one guest, one visit (confirmed, 25 Aug 2026):** a name/contact field on the guest wouldn't actually close the loophole — nothing checks it at the door. Instead, Friend Pass is capped at classesIncluded=1 always (the "School-set higher tiers" upsell is Class Pack-only); a School inviting the same person again, or a second friend, issues a second Friend Pass — a deliberate, staff-attributed action rather than a Student stretching one gift across multiple people. The same classesIncluded=1 cap applies to any scoped Class Pack, since a one-off Class has only one occurrence and extra credits against it could never be spent. Full detail in Section 6.1.
- **Logic-review audit round, 25 Aug 2026 — cross-section consistency fixes:** an adversarial multi-pass review (data model, concurrency, grading logic, payments, cross-section consistency) found several places where earlier edits hadn't been fully propagated. Fixed: the two stale Section 12.2 bullets claiming rank-gating and the waitlist were "considered, not built" were removed — both were already confirmed and built, and those bullets flatly contradicted Section 6.1/Section 7/Section 9 (the same cleanup the Walk-in-attendance bullet already got correctly). PromotionEvent gained from-stripe/to-stripe fields, since a Stripe Award previously left from-rank/to-rank unchanged and recorded no data about what actually changed. Transaction/Invoice's status enum gained Failed/Refunded, since a declined payment and an Auto-Refund cancellation both need a terminal state the original two-value enum lacked. Section 7's RanksModule gained stripe-award and bulk-stripe-award endpoints, since the API surface didn't expose the action needed to create a Stripe Award event at all.
- **Logic-review audit round, 25 Aug 2026 — concurrency and cancellation-race fixes:** Section 11.5's atomic-locking requirement previously covered only the booking-time credit decrement; it now also covers a WaitlistEntry Claim (same decrement path) and the Class capacity/Full check itself (a separate race from credit-locking — two Students can both see the last open seat simultaneously). waitlist-promotion-processing (Section 9) now triggers on a No-Show as well as a Cancellation, since a seat freed by a no-show was previously never offered to the waitlist. class-cancellation-processing (Section 9) was broadened from "Class Pack tickets" to cover Friend Pass tickets and Subscription/Weekly Pass Bookings too, and now also resolves outstanding WaitlistEntries — closing a race where a Manual cancellation policy (the default) left Bookings sitting Upcoming long enough for booking-no-show-expiry to wrongly mark them No-Show and forfeit a credit the School, not the Student, was responsible for cancelling.
- **Logic-review audit round, 25 Aug 2026 — remaining validation defaults:** each Rank stripe tier gained a stable id (currentStripe can't safely reference a positional index), with a School Portal validation rule that Rank order and stripe-tier order must be unique and contiguous per discipline, since the cumulative

eligibleClassTypes computation depends on it. A Student can no longer hold two simultaneous Active general-access (Subscription/Weekly Pass) Memberships at the same School, removing an undefined tie-break case. An Instructor override on Booking now explicitly bypasses the rank gate only, never capacity — an override on a Full Class still routes through the waitlist. “Downgrade takes effect at next renewal” is now scoped explicitly to Subscription plans, since Class Pack/Weekly Pass/Friend Pass aren't recurring and have no renewal event. A Subscription's cancellation-sweep now also cancels the Student's own future Bookings dated after their Membership's Expire date, rather than leaving them dangling against a lapsed Membership. Full detail in Section 6.1, Section 7, Section 9, and Section 11.5.

- **Architecture review round, 25 Aug 2026 — Platform Admin cross-tenant access hardening:** an adversarial four-lens audit of Section 4 (architecture, tenant isolation, security, cross-section consistency) found the RLS bypass role had no mechanism keeping it inside the audited API layer — a raw database credential would be invisible to both Section 4.7's audit log and Section 4.8's CI gate. Fixed: the bypass role is never issued as a raw credential to a human; routine cross-tenant work goes through the API and the existing audit log, with a rare break-glass exception path (short-lived secrets-manager credential, dual approval, automatic notification, independent pgAudit logging) for genuine ad-hoc SQL needs. Full detail in Section 4.3.
- **Architecture review round, 25 Aug 2026 — CI isolation gate moved earlier:** Section 4.9's rollout sequencing previously didn't require Section 4.8's automated cross-tenant isolation tests until General Availability, the very last milestone — while real payment credentials could already be live two milestones earlier. The CI gate now becomes required at the same milestone as onboarding any real school with a live payment credential, closing the window where actual customer payment data existed with zero automated isolation testing. Full detail in Section 4.9.
- **Architecture review round, 25 Aug 2026 — Platform Admin sub-roles defined:** Section 4.4's “read-only Support vs. full Platform Admin” example was too vague to implement safely once Platform Admin has visibility into live payment credentials. Replaced with three explicit least-privilege sub-roles: Support (read-only account-metadata plus audited impersonation), Billing/Payments Ops (PaymentAccount status and credential rotation, never a decrypted secret), and Full Platform Admin (the only tier that can assign sub-roles or invoke break-glass). Full detail in Section 4.4.
- **Architecture review round, 25 Aug 2026 — Branch made a real RLS boundary:** Branch previously had no enforcement mechanism of its own. Branch is now RLS-enforced (Section 4.3) using the same database-level mechanism as tenant isolation, keyed on branch_id for TimetableSlot, Class, and Instructor; Student, Membership, and Booking stay School-scoped since a Student may train across multiple Branches under one Membership. This also partially resolves Section 12.2's open Branch data-model question — Branch needs, at minimum, its own id and a school_id foreign key. Full detail in Section 4.3 and Section 6.2.
- **Architecture review round, 25 Aug 2026 — cross-section consistency fixes:** the same audit found several places where Section 4's architecture hadn't been propagated elsewhere in the document. Fixed: Section 12.2's Franchise-vs-School PaymentAccount ownership question was restored after being silently dropped in an earlier reorganization, now flagged more urgently given the RLS finding above. Section 3.1's background-jobs summary was updated from 7 to the current 13 jobs listed in Section 9. Section 3.1's component map and Section 4.2's monorepo package table now acknowledge the white-label multi-app distribution model (Section 5) rather than describing a single undifferentiated Student app, and Section 4.2 gained a package line for the white-label build/publish pipeline infrastructure. Section 3.2 gained a worked Platform Admin-side request-flow example tracing a payment-credential investigation through the isolation architecture, since the three existing examples were all customer-facing.
- **Pass 2 (Authentication & Roles, Section 8), 25 Aug 2026 — Instructor becomes a fourth customer-facing role:** grading (PromotionEvent.gradedBy), booking overrides (Booking.overriddenBy), and QR attendance all assumed an authenticated instructor actor, but no login existed to satisfy it — every action was really an Owner rubber-stamp with no real audit trail behind it. Instructor now gets its own login, School- and Branch-scoped, provisioned by School invite; permissions cover attendance scan, grading, and booking override only — not payments, School settings, or instructor management. No new app: reuses React Native and/or React Portal with a role-gated coach view. Full detail in Section 8.2, Section 8.3, Section 6.1.

- **Pass 2, 25 Aug 2026 — session/token revocation made concrete:** Section 4.4's "revoked immediately on offboarding" promise had no TTL, revocation-list, or reuse-detection mechanism behind it — a terminated identity's access token stayed valid until it naturally expired. Now: 15-minute access tokens for customer identities (5 minutes for Platform Admin, with no long-lived refresh equivalent) plus a Redis-backed, per-session revocable refresh-token store (Section 3.1) with rotation and reuse detection — a reused refresh token revokes the whole token family. Extended to every role, not just Platform Admin, closing the gap where a fired School Staff member or an ex-Owner's session after a School changes hands had no described revocation path. Caveat, stated plainly: this still bounds a live permission downgrade (not just full termination) to the access-token TTL window — an accepted, not eliminated, limitation. Full detail in Section 8.3.
- **Pass 2, 25 Aug 2026 — Branch Staff gets real RLS, not a blanket School view:** Branch Staff previously had no scope of its own. Now: real branch_id Row-Level Security on TimetableSlot, Class, Instructor, and (new this round) Booking. Membership and Transaction are genuinely School-wide, not Branch-scapable, so Branch Staff gets no raw row access to either — only a narrow, purpose-built membership-status-check endpoint (modeled on RanksModule's existing GET /students/{id}/eligibility pattern) returning an active/expired signal, never underlying records. Chosen over both an easier application-layer filter and a simpler blanket-School-visibility option, as the one consistent with this project's own established philosophy — real RLS where a physical-location boundary genuinely exists, least-privilege purpose-built reads elsewhere. Full detail in Section 8.2, Section 6.1, Section 7.
- **Pass 2, 25 Aug 2026 — Franchise Owner redefined around franchising-brand privacy, not corporate-chain ownership:** the prior Section 8.2 text gave Franchise Owner "everything a School Owner can do, across all Schools under their Franchise" — a cross-tenant RLS escalation. Corrected: a Franchisor (licensor) never gets a School Owner's (licensee's) access to that School's own data or accounts — a materially different relationship from the Guardian model's full-access one. Franchise Owner's actual scope: manage Franchise profile/branding and a new feeModel setting (Flat or Per-Headcount franchise-fee billing, School-under-Franchise, configurable per Franchise); a read-only Schools roster (address, owner/operator name and contact, brand-compliance/good-standing status, franchise-fee payment status, active student count for headcount billing) — a purpose-built narrow read, not an RLS escalation. This also resolves Section 12.2's long-open Franchise-vs-School PaymentAccount question (Section 6.2): a School's PaymentAccount is always its own; franchise-fee collection is a wholly separate payment relationship. Fee-charging mechanics deferred to Pass 4; legal review recommended on what a Franchisor may collect. Full detail in Section 8.2, Section 6.1, Section 6.2.
- **Pass 2, 25 Aug 2026 — Guardian/minor account model introduced:** a Guardian account (full adult login, holds payment method and waiver-signing authority) creates linked, lightweight Student profiles for minors with no independent login by default. An optional limited login (provisional age 13) grants read-only own-schedule/rank access only — never payment or waiver capability. At the age of majority (provisional 18) the person self-registers a fresh, independent Student account rather than migrating the linked profile. Both ages are explicitly provisional pending real legal review (children's-data-privacy and contract-capacity law are jurisdiction-dependent, and the platform spans multiple markets) — flagged as a new open item in Section 12.2 rather than silently finalized. Full detail in Section 6.1, Section 8.2.
- **Pass 2, 25 Aug 2026 — RoleGrant replaces an implicit role model, and multi-School role-holding is enabled now rather than deferred:** roles/scopes are now tracked as explicit, auditable, revocable RoleGrant records (user_id, role, scope, granted_by, granted_at, revoked_at) rather than an inferred column — consistent with this pass's session-revocation decision above. Sessions/JWTs are built from a User's full set of currently-active grants at login/refresh time. A side effect the audit flagged as lower-priority and originally deferred — one identity holding, say, an Instructor grant at one School and a Staff grant at another — falls out of this structure for free, and is enabled as of this round rather than left for later. Full detail in Section 6.1, Section 8.3.
- **Pass 2, 25 Aug 2026 — straight fixes from the same audit:** AdminUser's Key-fields cell still said "sub-role (e.g. Support / Full Admin)", stale from before Section 4.4's 3-tier Platform Admin split — corrected to name all three tiers. PromotionEvent named its grading actor three different ways in the same entity row

("Instructor/Owner" in Key-fields, "Instructor/User" in Relationships, vs. "Instructor/Staff" elsewhere in the document for Booking overrides and Membership gifting) — narrowed to "Instructor" alone for grading specifically, a coaching-specific action, while Booking and Membership correctly keep the broader "Instructor/Staff" for their genuinely operational/business actions. Booking gained a denormalized branch_id field to support the Branch Staff RLS decision above without a join through Class. Section 8.2's role table gained a sentence noting a person can hold more than one role, including the same role at more than one School. Full detail in Section 6.1, Section 8.2.

- **Pass 3 (White-Label Mobile App Distribution, Section 5), 25 Aug 2026 — TenantAppConfig narrowed to School-only, not Franchise:** the entity's Relationships cell said "Belongs to School or Franchise", but Pass 2's Franchise Owner redefinition (Section 8.2) gives a Franchisor no access to School-level operational data — a Franchise-level branded app would be exactly that. Confirmed: TenantAppConfig belongs to a School only; a Franchise-wide branded app is out of scope, consistent with the franchising-brand-privacy model. Full detail in Section 6.1.
- **Pass 3, 25 Aug 2026 — publishing credential model: scoped API keys, not shared account access:** Section 5.4 originally described UTM8 requesting "Admin access" to each School's Apple Developer and Google Play account. Reviewed against the alternative (a team-member invite using UTM8's own identity) and corrected to the right model for the right reason: a scoped, revocable App Store Connect API key and Google Play Developer API service account per School, provisioned through a guided School Portal flow and stored only in the secrets manager (Section 4.5), fetched per build job rather than held as a standing interactive session — this fits a continuous automated pipeline in a way an interactive login doesn't, independent of the suspension-risk argument. Two constraints captured as part of the same decision rather than left implicit: Apple's API key file is downloadable only once, so the guided flow must capture it at creation; and only the Account Holder can accept Apple's/Google's developer agreements, which UTM8 cannot do on a School's behalf — documented as a support-escalation limitation, not solved. Full detail in Section 5.4.
- **Pass 3, 25 Aug 2026 — Platform Admin gains a fourth sub-role, Publishing Ops:** Section 5.3 said the whiteLabelApp entitlement and its pipeline are "managed from the Platform Admin app" without naming which of the three existing sub-roles (Section 4.4) that meant — a gap the 3-tier split exists specifically to avoid leaving implicit. Reviewed for role-sprawl risk on a small team and for scope creep before confirming: Publishing Ops covers TenantAppConfig CRUD, pipeline retry/support operations, and credential status-read for renewal monitoring, plus dual-approval-gated store-side unpublish (Section 5.5) — RoleGrant's cheap multi-grant model (Section 6.1) means one person holding several sub-roles isn't real overhead. Toggling the whiteLabelApp entitlement itself stays a Full Platform Admin action, the same rare/structural class of change as creating a SubscriptionPlan — Publishing Ops operates the pipeline once the entitlement exists, it does not grant it. Full detail in Section 4.4, Section 5.3, Section 8.2.
- **Pass 3, 25 Aug 2026 — Publishing Ops actions audit the same way as the rest of Platform Admin; break-glass stays scoped to UTM8's own credentials:** every Publishing Ops action (TenantAppConfig CRUD, pipeline retry, credential access via the secrets manager) writes to AuditLogEntry under the existing Section 4.7 pattern — an application of precedent, not a new decision. Corrected on review: an earlier draft proposed extending Section 4.3's break-glass procedure to cover manual access to a School's own Apple/Google account, which contradicts the credential decision above's premise that UTM8 never holds an interactive credential for a School's account in the first place — there is nothing to "break glass" into. Break-glass stays scoped to emergency access to UTM8's own stored Apple/Google API keys and service-account credentials; the Account-Holder-must-personally-accept-agreements case is a documented support-escalation limitation, not a security procedure. Full detail in Section 4.3, Section 4.7.
- **Pass 3, 25 Aug 2026 — Student migration from the shared app to a newly-branded one gets a persistent, not one-time, nudge:** when a School upgrades, its existing Students are already on the shared UTM8 app (Section 5.2) and don't move automatically. Refined on review from a single one-time notification, which is easy to miss and can't be meaningfully reissued, to a persistent, non-naggy in-app prompt that stays visible until a Student installs the branded app or dismisses it, with an optional School-controlled firmer nudge available after a set delay if adoption stays low. This is a genuinely different problem from Section 5.5's downgrade path — downgrade needs zero Student action since there's no separate app to move to, while

some tail of Students never migrating on upgrade is an accepted, not eliminated, outcome. Full detail in Section 5.4.

- **Pass 3, 25 Aug 2026 — downgrade/cancellation redesigned around a credential grace period, a full-cancellation carve-out, and a billing-credit fault boundary:** the most heavily re-reviewed decision this pass, through several rounds of self-critique that each found a genuine gap. The frozen-at-last-build design (Section 5.5) is kept for a downgrade where the School stays active, but is now bounded: ULTM8 retains the School's Apple/Google credential for a 90-day grace period after downgrade, then purges its own stored copy via a new scheduled job (Section 9) — long enough to cover a billing hiccup or a considered decision, short enough that ULTM8 isn't indefinitely holding unused access; the School's own account and credential copy are untouched. Full cancellation of the whole School (not just the add-on) was originally handled the same way as downgrade; corrected, since a fully-cancelled School's backend access is a different situation, to a graceful in-app degraded state instead of either silent breakage or an over-automated auto-unpublish — store-side removal, if wanted, is a separate, slower, dual-approval-gated Publishing Ops action (Section 8.2), explicitly not the break-glass procedure. General tenant/School cancellation semantics beyond this branded-app-specific handling are not otherwise specified in this document; flagged as a new open item (Section 12.2) rather than invented here. Terminology and mechanics corrected: “stays live” describes ULTM8 declining to force-unpublish, not immunity from Apple's/Google's own review and takedown policies; ULTM8 cannot force-revoke a credential on Apple's or Google's side, only stop using and eventually purge its own copy. Each School's own store fees are independent of the ULTM8 subscription and keep being charged regardless of whiteLabelApp status — disclosed at upgrade (Section 5.4) and repeated at downgrade; since Google's \$25 fee is one-time and only Apple's \$99/year is recurring, the renewal-monitoring job and escalating 30/7/day-of reminder (Section 9) apply to Apple only, corrected from an earlier draft that applied the mechanism to both. A billing-credit fault boundary was added: a platform-caused build/submission failure or delay is creditable at support's discretion, a School-caused content rejection is not, judged case by case rather than auto-classified. Considered and explicitly declined: a formal billing-dispute appeals mechanism — general customer-support process, not white-label architecture. Full detail in Section 5.5, Section 6.1, Section 9, Section 12.2.
- **Pass 3, 25 Aug 2026 — straight fixes from the same audit:** Section 5.1's competitive-precedent framing (named competitors) and its Google Play citation (“Google explicitly recommends...”) were both unsourced claims presented as fact — softened to ULTM8's own risk-based reading without asserting specifics that can't be verified. Section 5.4's credential-custody sentence borrowed the Stripe secrets-manager pattern by name without noting a real difference — Stripe is moving toward Connect specifically so tenant secrets are never held at all, while Apple/Google publishing credentials have no Connect-equivalent and are always held — corrected to say so explicitly, and paired with the same monitored-rotation treatment already planned for APNs/FCM credentials. TenantAppConfig gained a Play App Signing enrollment field, missing from the original key-fields list. The callout box in Section 5.4 gained two more expectation-setting bullets: Apple's Organization-type enrollment needing a D-U-N-S number that can itself take weeks, and Google Play's closed-testing gate before a new app is eligible for production release. Section 3.1's background-jobs summary was updated from 13 to 15 to include the two new jobs from Section 5.5. Two Section 12.1 entries from the original 21 Aug 2026 decision — “White-label publish ownership” and “White-label downgrade policy” — described the pre-Decision versions of those decisions and would have contradicted the corrected Section 5.4/5.5 text; both updated to match. Full detail in Section 5.1, Section 5.4, Section 3.1, Section 12.1.
- **Pass 3 follow-up, 25 Aug 2026 — fallback confirmed if the Apple compliance risk isn't resolved:** the BLOCKING Apple Guideline 4.2.6/4.3 risk flagged in Section 12.2 is not open-ended — if a compliant path can't be confirmed (pre-submission consultation and/or legal review, Section 12.2) before the publishing pipeline ships, the branded-app tier is dropped from v1 entirely rather than shipped unresolved: every School launches on the shared ULTM8 app only (Section 5.2), and the branded-app upgrade is revisited as a later add-on only once (if) a compliant path is confirmed. Full detail in Section 5.2, Section 12.2.
- **Waitlist for full Classes (confirmed, 25 Aug 2026):** new WaitlistEntry entity, FCFS by position, costs no credit while Waiting. A freed seat (Booking cancelled) notifies the next entry with a claim window — School-configurable via waitlistClaimWindow (Section 6.1), default 2 hours; an unclaimed window cascades to the next person (booking-no-show — see waitlist-promotion-processing, Section 9). A Class's Full status now

counts every attendee across all Bookings (including Friend Pass guests), not Booking count (Section 6.1). Full detail in Section 6.1, Section 7, and Section 9.

- Workshop access vs. general Membership (confirmed, 25 Aug 2026): by default, a one-off Class is NOT covered by a Student's Subscription or Weekly Pass — it always requires its own scoped Class Pack ticket. The School can override this per Class via the new membership-inclusion flag (Section 6.1), opting specific workshops into general-membership access. Default is opt-out (ticket required); opt-in is a School decision, per Class.
- Grading credit for one-off Classes (confirmed, 25 Aug 2026): a one-off Class's Booking counts toward StudentRank.classesAttended under the same existing rule as any other Class — Rank's eligibleClassTypes field (Section 6.1) determines which class types count toward a given discipline's next rank. A workshop only contributes if the School has set its class type to one the target Rank already requires (i.e. the School has added it to that discipline's curriculum) — no separate mechanism was needed.
- Cancellation refund/credit policy (confirmed, 25 Aug 2026; broadened and detailed in Pass 4 — see the Pass 4 refund/credit-mechanism entry below): cancelling a Class with any Bookings that drew a Class Pack/Friend Pass credit always generates an affected credit-records list (new class-cancellation-processing job, Section 9); Subscription/Weekly Pass/Trial Membership Bookings on the same Class, which never drew a credit, are cancelled outright instead. Each School sets a class-cancellation-policy (Section 6.1): Manual (default) leaves the credit-records list for staff to resolve case-by-case; Auto-Refund or Auto-Credit resolves each credit-record automatically, without a staff choice — though a cash/bank-paid Auto-Refund still needs one staff step to confirm the physical money movement, the same manual pattern used to record a Cash/Bank Transfer payment in the first place.
- Walk-in attendance — resolved, not supported (confirmed, 25 Aug 2026): there is no scan-without-a-Booking path. A Student must book a Class in the app before attending; QR check-in only ever matches against a pre-existing Booking. This closes the open question previously flagged in Section 12.2.
- **Pass 4 (Payments, Section 10.1/10.3), 25 Aug 2026 — Franchise Subscription Plans resolved as ULTM8's own platform billing, and franchise-fee collection made Stripe-automated:** the long-open question of whether 'franchiseSubscriptionPlans' was ULTM8 charging the Franchise/School for platform access or a plan a Franchise resells onward (Section 12.2) is resolved as the former — SubscriptionPlan is ULTM8's own platform revenue (Section 6.1), evidenced by Franchise's own entity row already describing it as subscribing to a platform SubscriptionPlan; School's matching relationship, missing from its own entity row, was added to match (straight fix). Separately, and materially different: a Franchise's own fee charged to its member Schools (feeModel — Flat or Per-Headcount, Section 6.1) is collected via a new dedicated FranchiseFeeCharge entity, not SubscriptionPlan and not a School-level PaymentAccount (Section 8.2, Pass 2). Charging mechanics, deferred since Pass 2, are now decided: initially explored as a manual/invoice-first design over money-transmission concerns with a Stripe account-to-account transfer, then reviewed and corrected on explicit direction toward full Stripe automation — a Stripe Direct/Destination charge against the School's saved payment method, settling directly into the Franchise's own connected PaymentAccount, structurally identical to the Direct/Destination-charge pattern Membership billing already uses for Student→School (Section 10.2) — intended to avoid money-transmission exposure by keeping the platform out of the funds-custody chain, though whether it actually qualifies for the relevant exemptions in every jurisdiction is a still-open legal question (Section 12.2), not yet settled by the architecture alone. Flat fees run as a standing Stripe Subscription; Per-Headcount fees use Stripe's metered/usage-based billing, fed by a new franchise-fee-usage-reporting background job (Section 9) reporting each School's active-student-count monthly. ULTM8 takes no cut of this charge. The white-label add-on's pricing (Section 5.3) was clarified in the same pass — and corrected the same day, Section 12.2 — to be metered-only, no flat fee: a single metered \$0.99-\$1.99/active-student/month component, exact rate still to be set, billed by ULTM8 to the School (or Franchise, centrally); the payer decides privately how to recoup this from Students, and no Student-facing charge feature is built. Franchise-disclosure-law legal review remains open (unaffected by the automation decision — it governs what may be collected, not how) and a new School-Franchise-affiliation-change lifecycle gap was flagged rather than solved. Full detail in Section 6.1, Section 7, Section 8.2, Section 9, Section 10.2, Section 12.2.

- Pass 4 (Payments), 25 Aug 2026 — Refund and credit mechanism resolved, plus a real Booking data-model gap closed:** the previously-undefined Auto-Refund/Auto-Credit class-cancellation-policy settings (School entity, Section 6.1) are now two mutually exclusive remediations for the same forfeited credit — Auto-Refund gives the money back via Stripe Refund API or a manual Cash/Bank ledger reversal, with the credit staying spent; Auto-Credit restores the credit with no money movement, matching the mechanism already used for a Student's own pre-cutoff cancellation (that field's wording corrected from 'refunded back' to 'credited back' for consistency now that the two terms are formally distinct). A Friend Pass entry is always forced to Auto-Credit regardless of School policy, since it was gifted at £0. Multi-class Class Pack refunds are computed per-credit ($\text{price} \div \text{classesIncluded}$), not full-amount, and Transaction gained a cumulative, guarded refundedAmount field so several partial refunds against the same purchase can't exceed its original amount. A genuine pre-existing gap surfaced along the way: Booking's single sourceMembershipId field couldn't represent a multi-attendee Booking drawing against several different Memberships at once (the Student's own plus a Friend Pass per guest) — each attendee-list entry now carries its own membershipId, refundResolution, and resolvedBy, and sourceMembershipId gained the same pair for the Student's own credit. Trial Membership, which had been left out of the 'no credit involved, cancel outright' bucket alongside Subscription/Weekly Pass, was added (straight fix). Authorization was split on money-vs-no-money grounds: Auto-Credit is Branch Staff-accessible (no money moves), Auto-Refund is School Owner/Manager-only, and a disputed FranchiseFeeCharge refund is Franchise Owner-only, since the Franchise's own PaymentAccount holds that money — mirroring the separation most POS systems draw between taking a sale and authorizing a refund. An Auto-Credit that lands on an already-Expired Membership reactivates it only if it expired from hitting 0 credits, not if its own expiry date had already passed. A final adversarial pass the same day hardened four more edges: the job now excludes a Booking the Student already self-cancelled pre-cutoff, so its credit can't be double-compensated; a resolved ticket-holder Booking's own status now flips to Cancelled too, closing the same false-No-Show risk already guarded against for general-access Bookings; both endpoints' idempotency keys and targeting are now per-credit-record (Booking id + membershipId), not per-Booking, since one Booking can hold several; and credit-restore now explicitly states its branch_id RLS and that it only ever acts on a credit-record the job has already flagged pending. Full detail in Section 6.1, Section 7, Section 8.2, Section 9, Section 10.2, Section 12.2.
- Pass 4 (Payments), 25 Aug 2026 — Recurring-payment failure handling resolved (Decision 4), plus the deferred refund-API-failure case closed:** every recurring charge (Membership Subscription, Franchise-fee, platform/white-label SubscriptionPlan) already ran on a Stripe Subscription object, so retries lean on Stripe's own Smart Retries via the existing webhook queue rather than a custom BullMQ scheduler. Subscription-type Membership renewal failure reuses the exact same Active→Expired sweep mechanism already confirmed for voluntary cancellation — stays Active while past_due, Expires (and sweeps future Bookings) only once Stripe exhausts retries; Subscription-type MembershipPlans now require Stripe as the payment method, since auto-recurring billing has no Cash/Bank Transfer equivalent. Franchise-fee failure gets no School-side lockdown — ULTM8 isn't party to that commercial relationship — just a new franchiseFeeSubscriptionStatus field on School, visible to both the School and the Franchise Owner. The core platform SubscriptionPlan and the whiteLabelApp add-on are now billed as two separate Stripe Subscriptions, so a failure in one can't take down the other; white-label stays School-scoped even under centrally-managed Franchise billing, so a failure never cascades across Schools, and on cancellation it reuses Section 5.5's frozen-at-last-build behavior. A cancelled core platform SubscriptionPlan degrades the tenant's portal to read-only — full view, no new Classes/Bookings/payments — rather than deletion; a Franchise's own failure degrades only the Franchise Owner's own actions, never cascading to member Schools' own, separately-paid access, and class-occurrence-generation stops materializing new Class rows for a degraded-state School. invoice.payment_failed was added to the handled webhook list, feeding notification-fanout so Students/Schools are warned during the retry window rather than silently locked out. Separately, a failed Stripe Refund API call (deferred from the refund/credit-mechanism decision) now falls back to the existing Manual bucket with an explicit failed-attempt flag and staff notification, instead of stalling indistinguishably from a never-attempted case. Which of Express or Standard a tenant onboards through (Section 10.4) now also determines who controls its retry schedule — raising, not resolving, the stakes of that already-open Section 12.2 item. Full detail in Section 6.1, Section 7, Section 8.2, Section 9, Section 10.2, Section 10.4, Section 12.2.

- **Pass 4 (Payments), 26 Aug 2026 — GoCardless exclusion reconsidered and reaffirmed (Decision 5):** the UK-market economic case was reviewed — GoCardless direct debit runs roughly 1% + 20p capped at £4 per collection versus Stripe's 1.5%+20p on UK cards (2.5–3.25%+20p on EEA/international cards) plus Stripe Billing's own ~0.7% recurring-volume surcharge, uncapped, and bank-mandate collections see materially fewer involuntary failures than card-based ones since a Direct Debit mandate doesn't expire the way a card does. Reversing the exclusion would mean designing and building a second full gateway integration from scratch — its own onboarding/mandate lifecycle analogous to Section 10.4's Stripe Connect design, its own webhook set feeding the same dunning machinery Decision 4 just built — and GoCardless's bank-debit rails (Bacs/SEPA/ACH/BECS) don't cover this platform's BRL, AED, or MYR markets (Section 11.2) regardless, so it could only ever sit alongside Stripe for a GBP/EUR/USD slice, never replace it. Weighed against that, Section 1.2's exclusion is reaffirmed as-is — GoCardless and PayPal remain out of scope for this build, no new gateway work added, no open item carried forward.
- **Pass 5 (NFR/consistency sweep), 26 Aug 2026 — Membership creation redesigned around payment settlement, not a Pending status (Decision 6):** Membership.Status is Active/Expired only — Pending is removed entirely, closing a real bug the first draft of this decision introduced (a £0 Friend Pass, with no Transaction ever created, would have been stuck Pending forever). A Membership row is now created, not status-flipped, at the moment its payment settles: any Stripe charge succeeding — one-time PaymentIntent (Class Pack, Weekly Pass, Trial) or Subscription — creates it Active immediately via the existing stripe-webhook-processing job (Section 9); Cash/Bank Transfer payments wait on staff confirmation via a new PATCH /transactions/{id}/confirm endpoint (School Owner/Manager-accessible only, not Branch Staff, since Transaction carries no branch_id to scope by; idempotent) that atomically flips the Transaction Pending→Successful and creates the Membership Active in one step; any Cash/Bank-eligible plan priced at £0 skips the wait and is created Active immediately, since there's nothing to confirm. Transaction/Invoice gains a reference to the MembershipPlan being purchased, without which staff confirming a Transaction wouldn't know what to create. The phrase "at purchase" is generalized everywhere to mean creation time, so the existing rule blocking two simultaneous Active general-access Memberships is re-validated at the actual moment of creation, not just checkout-click time — closing a race where a Student could start a Cash purchase, separately buy a Stripe Subscription that settles first, then have staff confirm the stale Cash purchase days later and end up with two Active Memberships at once; the Stripe-path collision is handled by attempting a refund (falling back to Decision 4's existing failed-refund handling only if that refund call itself fails, plus cancelling the Subscription), the Cash/Bank-path collision by rejecting the second confirmation outright rather than creating a second Membership. Section 2.2 and Section 9's Figma-literal citations of "Membership Active/Pending/Expire" are reconciled with a note that Pending was a design-file artifact — the real Pending status lives on the Transaction, not the Membership. Full detail in Section 6.1, Section 3.2, Section 9, Section 10.2, Section 12.1.
- **Pass 5, 26 Aug 2026 — Multi-currency policy resolved, split by who sets the price (Decision 7):** Student-facing MembershipPlan pricing stays independent per-currency, School-set, with no automatic conversion — a School prices in its own local currency and that is what a Student pays, full stop; this favors pricing stability over a theoretically "fairer" converted price that would drift with exchange rates outside anyone's control. UTM8's own revenue — the platform SubscriptionPlan, the white-label add-on, and Franchise-fee — all quote and bill in a single anchor currency (USD, matching the white-label add-on's \$0.99-\$1.99/active-student/month reference range — the exact rate is still an open item, Section 12.2), with the card network's own FX handling conversion at charge time; this isn't a new pattern, it's the one Franchise-fee already used (Section 10.2) applied consistently to the other two. Transaction/Invoice gains a currency field, since without one, refund and reconciliation math against a per-currency-priced MembershipPlan can't work. Full detail in Section 11.2, Section 6.1, Section 12.1.
- **Pass 5, 26 Aug 2026 — Internationalization and RTL mechanism specified (Decision 8):** i18next on both React and React Native, backed by i18next-http-backend rather than the more common static-bundle pattern, so it actually serves the CMS-backed translation content (Section 11.1) dynamically rather than requiring a rebuild per copy change; a version-stamp field on cached translations lets the existing translation-cache-invalidation job's Redis bust (Section 9) reach already-open sessions, not just new ones. RTL for Arabic is two separate mechanisms, not one, since the original recommendation only covered the web side: React

(web) uses CSS logical properties (margin-inline-start and equivalents); React Native has no CSS at all, so it needs `l18nManager.forceRTL/allowRTL` plus start/end layout props instead, and switching direction on Native requires an app relaunch — a real platform limitation to design around, not an implementation gap to close. Full detail in Section 11.1, Section 12.1.

- **Pass 5, 26 Aug 2026 — Object storage / CDN provider selected (Decision 9):** Cloudflare R2 — S3-compatible object storage with zero egress fees, which matters here because banner images and gallery photos are read far more often than written; at roughly 1TB stored/10TB served, R2 runs about \$15/month against a comparable ~\$900/month on S3, almost entirely because of S3's per-GB egress pricing. R2 itself has no image-transformation feature, so Cloudflare Images is named as a companion decision for on-the-fly resizing/thumbnailing of banner, gallery, and profile images; signed-waiver and invoice PDFs are served from R2 directly, no transformation needed. This resolves the storage half of Section 11.3's open item but not the jurisdictional half — data-residency/GDPR/LGPD compliance across this platform's markets is a separate, still-open legal-review item (Section 12.2). Full detail in Section 11.3, Section 12.1.
- **Pass 5, 26 Aug 2026 — Push, email, and SMS/OTP providers selected (Decision 10):** Push goes direct to FCM (Android)/APNs (iOS) rather than through a third-party aggregator, since BullMQ already drives notification fan-out server-side and an aggregator would add cost without adding capability here. SMS/OTP uses Twilio Verify specifically — the purpose-built OTP product, not Twilio's generic Messaging API — since it manages code generation, delivery, and expiry as one flow rather than requiring that logic be built on top of raw SMS sending; UAE currently has alphanumeric sender-ID registration on regulatory hold and Malaysia requires domestic sender-ID pre-registration that gets overwritten with a random operator-assigned numeric ID regardless (so registered branding never actually displays to the recipient), but Verify's managed routing still delivers OTPs reliably in both markets. Email uses Postmark for transactional delivery — deliverability is solid, but 2026 has seen reports of abrupt account suspensions post-acquisition, worth a monitoring note and a fallback provider kept in reserve rather than treating this as a closed, single-vendor decision. Full detail in Section 11.4, Section 3.1, Section 3.2, Section 9, Section 12.1.
- **Pass 5, 26 Aug 2026 — Environments, CI/CD, and hosting resolved (Decision 11):** AWS — RDS for PostgreSQL, ElastiCache for Redis, ECS Fargate for the NestJS API and BullMQ workers — with GitHub Actions for CI/CD, gated by a required GitHub branch-protection status check running Section 4.8's automated cross-tenant isolation test suite before merge, stated concretely rather than left as an implied "gated by" relationship. This also resolves the Vault-vs-AWS-Secrets-Manager shortlist already named in Section 4.5, Section 5.4, and Section 10.4 in favor of AWS Secrets Manager specifically, now that the surrounding infrastructure is AWS-native. Backup/DR — absent from this document until now, despite this being a payments- and audit-log-bearing platform — is RDS automated backups with point-in-time recovery and Multi-AZ. Data-residency/GDPR/LGPD compliance across this platform's UK/EU/Brazil/UAE/Malaysia footprint remains open and needs real legal review — a new Section 12.2 item, same treatment as the Guardian minor age-thresholds and franchise-disclosure-law items already flagged there. Full detail in Section 11.6, Section 4.5, Section 10.4, Section 12.2, Section 12.1.
- **Pass 7 (Security, Section 11.5), 26 Aug 2026 — Owner-tier account protection and logout/anti-abuse policy resolved (Decision 12):** step-up MFA (not blanket login MFA) now gates sensitive actions for School Owner/Manager and Franchise Owner — payout/bank-detail changes, Auto-Refund, franchise-fee refunds — answering the MFA question left open in Section 12.2 without adding login friction for a largely non-technical Owner persona. Failed login/OTP-verify attempts get per-account escalating logout decoupled from IP-based limiting, and Twilio Verify's Fraud Guard plus an OTP-send rate limit close the SMS-pumping gap. Full detail in Section 11.5.
- **Pass 7, 26 Aug 2026 — Client-side token custody resolved (Decision 13):** httpOnly/Secure/SameSite cookies for the web apps, OS keychain/keystore on mobile, localStorage explicitly ruled out. Full detail in Section 11.5.
- **Pass 7, 26 Aug 2026 — Encryption baseline resolved (Decision 14):** RDS encrypted via AWS KMS, R2 encrypted at rest, TLS 1.2+ enforced platform-wide — stated explicitly rather than left to default configuration, since neither was specified anywhere in this document before this pass. Full detail in Section 11.5.

- **Pass 7, 26 Aug 2026 — Audit-log immutability enforcement resolved (Decision 15):** UPDATE/DELETE revoked on the audit table at the database layer for every role, including the break-glass credential, which is provisioned as a non-superuser, non-table-owner role so this can't be bypassed. Retention period stays open pending legal/compliance input, Section 12.2. Full detail in Section 11.5, Section 4.3.
- **Pass 7, 26 Aug 2026 — Ownership validation on purpose-built non-RLS endpoints resolved (Decision 16):** membership-status-check and the Franchise Owner School-roster read now independently validate their path-parameter target against the caller's own RoleGrant scope, closing an IDOR gap RLS doesn't cover since these endpoints exist specifically to bypass it; credit-restore was checked and found already covered by existing Booking-level RLS. Full detail in Section 11.5, Section 8.2.
- **Pass 7, 26 Aug 2026 — Rate limiting extended to booking/waitlist/credit endpoints (Decision 17):** the same per-user/per-IP throttling already used on auth endpoints now also covers class-booking, waitlist-claim, and credit-restore — a volumetric-abuse risk distinct from the atomicity/race-condition fixes above. Full detail in Section 11.5.
- **Pass 7, 26 Aug 2026 — CI/CD hardening resolved (Decision 18):** Dependabot is a required GitHub Actions status check; CodeQL runs advisory/non-blocking rather than gating merges, given the false-positive triage burden a small team can't sustain as a hard gate. GitHub Actions authenticates to AWS via OIDC federation, not static long-lived IAM keys. Full detail in Section 11.5, Section 11.6.
- **Pass 7, 26 Aug 2026 — Security alerting resolved (Decision 19):** CloudWatch alarms on fixed-threshold auth-failure/lockout spikes and booking/credit-claim volume, riding on the AWS stack already chosen — the first place customer-facing abuse becomes visible, distinct from the Platform Admin audit log's admin-action-only coverage. Full detail in Section 11.5.
- **Pass 7, 26 Aug 2026 — PCI-DSS scope stated explicitly (Decision 20):** Stripe-hosted fields/redirect only for both checkout and Connect onboarding, no raw card/bank data touches ULTM8's own infrastructure, targeting SAQ-A — a direct consequence of Section 10.4's existing Stripe Connect custody model, simply never stated in PCI terms before. Full detail in Section 11.5, Section 10.4.
- **Pass 7, 26 Aug 2026 — App Store Connect / Google Play credential blast radius addressed (Decision 21):** unlike Stripe's Connect-based isolation, ULTM8's publishing-service identity holds every School's app-store credential centrally with no per-tenant isolation of the pipeline itself; a fixed build/submission rate limit per tenant per day and extending Section 5.5's existing dual-approval pattern to abnormal/bulk publish actions are the compensating controls, chosen over a heavier per-tenant KMS re-encryption given this project's scale. Full detail in Section 11.5, Section 5.4, Section 5.5.
- **Pass 8 (Round 1: implementability, timing, outage-resilience, scale, operability), 26 Aug 2026 — API conventions resolved (Decision 22):** standard {error: {code, message}} envelope on every non-2xx response, cursor-based pagination on every list endpoint, /v1 URI prefix from the first deploy. Full detail in Section 7.
- **Pass 8, 26 Aug 2026 — Default job retry/backoff/alerting policy resolved (Decision 23):** 3 attempts, exponential backoff, failure alert on exhaustion, for every job except stripe-webhook-processing (which already leans on Stripe's own Smart Retries). Full detail in Section 9.
- **Pass 8, 26 Aug 2026 — Missing field types/units/validation closed (Decision 24):** Class.capacity is a nullable positive integer (null = unlimited); User.phone is E.164, validated at registration; User.DOB is ISO 8601 date-only; School.waitlistClaimWindow is stored as integer minutes (default 120). Full detail in Section 6.1.
- **Pass 8, 26 Aug 2026 — Timezone convention resolved (Decision 25):** all datetimes are stored in UTC; School gains a required IANA timezone field, and every date-only comparison (expiry, grace-period day-counts, Class scheduling) resolves against it, not server-local time — closing a gap that had zero prior mention anywhere in this document despite five markets spanning at least four distinct timezone bands (UK, Brazil, UAE, and Malaysia alone run from UTC+0-ish to UTC+8; the EU market's own zone was never pinned to a specific country anywhere in this document, so it may add a fifth). Full detail in Section 6.1.
- **Pass 8, 26 Aug 2026 — Membership expiry made live-computed (Decision 26):** date-based expiry (Class Pack, Friend Pass, Weekly Pass) is evaluated live at every access/booking check against the stored expiry date, not flipped by a scheduled job — removing a lag window of the same shape Section 11.5's atomicity fixes already closed elsewhere, just left open here. Full detail in Section 6.1.

- **Pass 8, 26 Aug 2026 — Credential-purge same-day race closed (Decision 27):** tenant-app-credential-grace-period-purge transactionally checks the credential's re-upgrade timestamp immediately before deleting, so a same-day re-upgrade always wins over a same-day purge. Full detail in Section 9, Section 5.5.
- **Pass 8, 26 Aug 2026 — Guardian revocation cascade closed (Decision 28):** revoking or deleting a Guardian account cascades revoked_at onto every linked minor's own RoleGrant, so a minor's limited-login access can't outlive the Guardian relationship that authorized it — unlike every other permission downgrade in this document, this one had no bound at all until now. Full detail in Section 6.1, Section 8.2.
- **Pass 8, 26 Aug 2026 — Synchronous Stripe checkout failure UX resolved (Decision 29):** a failed PaymentIntent-creation call (network, Stripe-side degradation) shows the Student a plain retryable error — no charge queued, no Transaction row created until Stripe actually responds. Distinct from the already-specified async webhook/dunning path. Full detail in Section 10.2.
- **Pass 8, 26 Aug 2026 — Data-layer scale hardening resolved (Decision 30):** composite indexes leading with the tenant column on every RLS-scoped table; AuditLogEntry range-partitioned by month from the start; RDS Proxy plus a per-task connection-pool ceiling; Redis given an explicit eviction policy. None of this was stated anywhere before this pass, on a platform whose entire tenancy model depends on per-row RLS checks at scale. Full detail in Section 11.6.
- **Pass 8, 26 Aug 2026 — Observability & tracing resolved (Decision 31):** a general CloudWatch app-health dashboard (error rate, p95 latency, RDS load) and per-service /health endpoints, beyond Pass 7's security-specific alarms; Bull Board plus queue-health alarms across all 18 BullMQ queues; an X-Request-Id propagated through the NestJS → BullMQ → Stripe-webhook chain with structured JSON logging throughout. Full detail in Section 11.6.
- **Pass 8, 26 Aug 2026 — Deploy rollback, migration discipline, and ORM resolved (Decision 32):** Prisma is the chosen ORM, resolving the open item in Section 12.2; a bad deploy reverts by redeploying the previous Fargate task-definition revision; schema changes follow an additive-first, backward-compatible expand/contract convention, since not every Postgres migration is cleanly reversible on a payments-bearing platform. Full detail in Section 11.6.
- **Pass 8, 26 Aug 2026 — Twilio Verify outage no longer locks out login entirely (Decision 33):** a failed SMS-OTP send falls back to an email-delivered OTP through the existing Postmark/SES path, closing the sharpest finding this round — OTP had been the ONLY login/registration/recovery path, with no fallback of any kind. Full detail in Section 11.4.
- **Pass 8, 26 Aug 2026 — Postmark fallback named, failure handling specified (Decision 34):** AWS SES is the named fallback provider (already on AWS, no new vendor relationship needed) — Section 11.4 previously said only that a fallback was 'worth keeping in reserve' without naming one. A failed transactional email now routes to a dead-letter queue and surfaces to Platform Admin rather than silently vanishing. Full detail in Section 11.4.
- **Pass 8, 26 Aug 2026 — franchise-fee-usage-reporting stale-report risk resolved (Decision 35):** the job runs at a fixed T-1-day offset before the charge; an affiliation or subscription-state change inside that window routes to manual review rather than letting Stripe settle a charge against stale numbers — this moves real money into a Franchise's connected account, so it shouldn't auto-resolve on a number that may already be wrong. Full detail in Section 9, Section 10.2.
- **Pass 8, 26 Aug 2026 — R2 read-availability risk accepted (Decision 36):** a brief Cloudflare R2 outage blocking waiver/invoice viewing is accepted as a documented risk rather than built around, given R2's own high published uptime — adding a cache/fallback layer for a low-probability, short-duration risk isn't proportionate for this platform's scale. Full detail in Section 11.3.
- **Pass 8 Round 2, 26 Aug 2026 — Signed-waiver and invoice PDF access control specified (Decision 37):** Section 11.3 and Section 6.1 named Cloudflare R2 as the object-storage provider for banner/gallery images alongside signed-waiver and invoice PDFs, without saying the latter two need access control the former two don't — banner and gallery images are meant to be publicly viewable, waivers and invoices are not. Waiver and invoice PDFs now live in a separate, private R2 bucket; the download endpoints already named in Section 7 mint a short-TTL (5-minute) presigned GET URL only after the existing RLS/ownership check passes,

mirroring the credential-custody pattern Section 10.4 already uses — no PDF is ever served from a publicly-readable path.

- **Pass 8 Round 2, 26 Aug 2026 — Stripe webhook tenant-derivation specified (Decision 38):** Section 7's webhook endpoint (POST /payments/webhooks/stripe, Section 10.2) verifies the Stripe signature and dedupes on event id, but never stated how a given event is matched back to the correct tenant. The target PaymentAccount is now resolved from the event's own Stripe Connect account field (the field Stripe itself sets on every Connect event), not from any client-supplied metadata on the event payload, which is untrusted input an attacker could shape. Any client-supplied metadata is cross-checked against that resolved account for consistency, logged and dropped on mismatch, rather than trusted outright.
- **Pass 8 Round 2, 26 Aug 2026 — Platform Admin impersonation scope narrowed (Decision 39):** Section 4.4's impersonation flow and Section 8.3's RoleGrant model together left open whether an impersonation token inherits every RoleGrant the support target holds — including grants on Schools unrelated to the case under investigation — or only the one tenant being impersonated. It's the latter: the impersonation token is minted scoped to the single RoleGrant matching the specific tenant identified when impersonation was initiated, not the target user's full grant set, so impersonating a multi-School Instructor or Franchise Owner to help with one School never exposes the others.
- **Pass 8 Round 2, 26 Aug 2026 — White-label usage-reporting job generalized (Decision 40):** Section 9's franchise-fee-usage-reporting job (Section 10.2's metered-billing mechanism) only fires for Franchise-affiliated Schools on the Per-Headcount fee model, but Section 5.3's whiteLabelApp metered add-on is available to any School regardless of Franchise affiliation or fee model — an independent School, or a Flat-fee-Franchise School, with the add-on had no job reporting its usage to Stripe at all. The job is generalized to run for any School with an active whiteLabelApp entitlement (Section 6.1), independent of Franchise.feeModel, reporting active-student usage on the same T-1-day fixed-offset schedule Decision 35 already established for the Per-Headcount case.
- **Pass 8 Round 2, 26 Aug 2026 — Partial-refund rounding rule specified (Decision 41):** Section 6.1 and Section 10.2 define a multi-class Class Pack refund as $\text{price} \div \text{classesIncluded}$ per cancelled class, with Transaction.refundedAmount validated so cumulative refunds can't exceed the original amount, but never specified how successive partial refunds round when $\text{price} \div \text{classesIncluded}$ doesn't divide evenly — independent per-refund rounding can overshoot or undershoot the original charge across several partial refunds against the same purchase. Each refund is now computed as the cumulative entitlement to date ($\text{classes cancelled so far} \times \text{price} \div \text{classesIncluded}$, rounded once to the currency's minor unit) minus whatever has already been refunded against that Transaction, capped by the remaining original amount — rounding happens once per cumulative total, not independently on each individual refund.
- **Pass 8 Round 2, 26 Aug 2026 — Platform-level revenue ledger added (Decision 42):** Section 6.1 gives FranchiseFeeCharge a dedicated entity recording every Franchise-fee charge against a School, but ULTM8's own platform-level revenue — the platform SubscriptionPlan charges and the whiteLabelApp metered add-on from Section 5.3 — had no equivalent ledger entity, only the general Transaction/Invoice record shared with Student-facing Membership billing. A new PlatformCharge entity is added, mirroring FranchiseFeeCharge's shape (School/Franchise reference, charge type — SubscriptionPlan or whiteLabelApp —, amount, currency, Stripe charge/invoice reference, status), giving ULTM8's own revenue the same first-class, queryable ledger the Franchise-fee side already has.
- **Pass 8 Round 2, 26 Aug 2026 — AuditLogEntry data-subject granularity added (Decision 43):** Section 4.7's AuditLogEntry (Section 6.1) records the actor, action, target tenant/entity, before/after, and timestamp of every Platform Admin action, but only at tenant granularity — it doesn't name which individual data subject (Student, Guardian, Instructor) a given action touched, which a future data-subject-access or breach-notification request would need. AuditLogEntry gains a nullable subjectUserId field, populated whenever the target entity is itself owned by or references one Student/Guardian/Instructor record, left null for tenant-wide actions with no single subject. Retention duration for audit-log entries themselves stays governed by the data-residency/GDPR & LGPD open item already tracked in Section 12.2 — there is no separate, uncaptured audit-log-retention question beyond that one.

- **Pass 8 Round 2, 26 Aug 2026 — Account-deletion endpoint and cascade mechanism specified (Decision 44):** Section 2's confirmed deleteAccount Figma screen had no backend counterpart anywhere in Section 7 or Section 9 — no endpoint, no processing job. A DELETE /users/me endpoint is added (Section 7), self-service, requiring re-authentication (matching the step-up-MFA pattern already used for sensitive actions), which enqueues a new account-deletion-processing BullMQ job (Section 9) that cascades per-entity: WaiverSignature/StudentRank/Notification rows are hard-deleted, while Booking/Transaction/Membership rows referencing the account are anonymized (personal fields nulled, foreign key retained) rather than deleted outright, since a School's own financial and attendance records can't simply disappear out from under it. The actual retention window before this job runs, and any jurisdiction-specific variation, stays the open legal-review question already tracked in Section 12.2 — only the technical deletion/cascade mechanism is resolved here.
- **Pass 8 Round 2, 26 Aug 2026 — Client-side mutation idempotency added (Decision 45):** Section 9's job-level idempotency (dedupe on Stripe event id, per-credit-record targeting) covers server-side races, but the client-facing mutating endpoints Section 7 lists — POST /classes/{id}/book, POST /payments/charge, POST /payments/subscribe — had no idempotency mechanism of their own, so a connectivity drop after the request reaches the server but before the client sees a response leads to an ambiguous outcome and a plausible double-submit on reconnect/retry. Each of these endpoints now accepts a client-generated Idempotency-Key header (a UUID minted once per user action, reused verbatim on retry); the server dedupes on (user_id, endpoint, key) for a rolling 24-hour window, returning the original response on a repeat rather than re-processing.
- **Pass 8 Round 2, 26 Aug 2026 — Push-token entity and app-variant routing added (Decision 46):** Section 9's notification-fan-out job and Section 6.1's Notification entity assume push delivery works, but no entity anywhere recorded a device's push token, and Section 5's white-label model means a School's branded app needs its own APNs/FCM credentials (TenantAppConfig, Section 6.1) distinct from the shared Student app's — nothing routed a push to the right credential set. A new DeviceToken entity (userId, platform, token, app variant reference — shared Student app or a specific School's TenantAppConfig, last-seen timestamp) is added; notification-fan-out selects the credential set from the token's app-variant reference, prunes a token on an APNs/FCM bounce response, and falls back to in-app/email delivery (Section 9's existing Postmark/SES path) whenever a user has no live token, for any notification already classified as critical.
- **Pass 8 Round 2, 26 Aug 2026 — Forced-LTR override for numeric/Latin fields added (Decision 47):** Section 11.1 Decision 8's RTL mechanism mirrors layout direction for RTL locales but said nothing about fields that are inherently Latin/numeric regardless of locale — OTP codes, the 6-digit Passcode, card numbers, phone numbers. These fields are now forced dir="ltr" at the component level independent of the active locale's overall RTL/LTR direction, the standard mobile/web pattern for numeric entry inside an otherwise-mirrored layout, so a card number or OTP never visually reverses digit order for an RTL-locale user.
- **Pass 8 Round 2, 26 Aug 2026 — Concurrency-regression CI suite extended (Decision 48):** Section 4.8 makes RLS cross-tenant-isolation tests required-to-merge CI gates, but the platform's other highest-risk state machines — credit-decrement on Booking creation, waitlist-claim races (Section 9's waitlist-promotion-processing job), and Class capacity enforcement under concurrent bookings — had no equivalent CI-gated correctness suite. The same required-to-merge pattern Section 4.8 established for RLS is extended to a dedicated concurrency-regression suite covering these three state machines, run against a real Postgres instance rather than mocked, so the row-level locking Section 11.6 already commits to is exercised, not assumed.
- **Pass 8 Round 2, 26 Aug 2026 — Grading-prototype test-evidence handoff captured (Decision 49):** Section 2.3 states the working grading-system prototype validates specific behavior but 'is not source code to port,' and Section 13 confirms it's a design-audit input, not a build artifact — but the prototype's own test scenarios (the specific grading transitions and edge cases it was built to demonstrate) were never captured anywhere as a reusable acceptance-criteria reference, so that validation effort didn't transfer forward. The prototype's test scenarios are now captured as a named handoff artifact referenced from Section 2.3, to be used as a starting acceptance-criteria set for the production grading feature's own test suite — not ported code, but not lost knowledge either.

- **Pass 8 Round 2, 26 Aug 2026 — Stripe webhook contract-testing added (Decision 50):** Section 9 and Section 10.2 specify Stripe-webhook idempotency and retry behavior in detail, but no testing methodology validated that behavior against Stripe's actual delivery quirks — duplicate delivery, out-of-order delivery, Smart Retries. Webhook contract testing is added to the CI/CD pipeline already established in Section 11.6: Stripe CLI event forwarding against a staging deployment for pre-release manual verification, plus a fixture-replay suite (recorded real Stripe event payloads, replayed duplicated and out-of-order) run in CI to regression-test the dedupe-on-event-id and Smart-Retries-reliant logic Section 9/10.2 already commit to.
- **Pass 8 Round 2, 26 Aug 2026 — Pre-launch load-test plan added (Decision 51):** Section 11.6's scale-hardening decisions — the RDS Proxy connection ceiling and row-level locking for concurrent booking/credit operations — had no stated load-test validating they hold under realistic concurrent load before launch. A load-test plan is added to the pre-launch checklist implied by Section 11.6: a scripted concurrent-booking/credit-claim load test, run against a staging environment sized to match production, gating go-live on the RDS Proxy ceiling and row-level-locking behavior holding under that load rather than being assumed correct from code review alone.
- **Pass 8 Round 2, 26 Aug 2026 — Migration-safety CI check and rollback rehearsal added (Decision 52):** Section 11.6 Decision 32 commits to an expand/contract migration convention and a Fargate task-definition-revision rollback mechanism, but neither was validated by an automated check — a migration that violates expand/contract (e.g. a same-deploy column drop) could still ship, and rollback itself had never been rehearsed. A CI check is added that fails a migration PR if it drops or renames a column/table in the same migration that also changes application code referencing it — the concrete signal that expand/contract was violated; rollback rehearsal — actually redeploying the previous task-definition revision against current data — is added as a required step in the staging release checklist before each production deploy, not left as a theoretical capability.
- **Pass 8 Round 2, 26 Aug 2026 — Bull Board / CloudWatch cross-tenant side channel closed (Decision 53):** Decision 31's Bull Board and CloudWatch tooling gave admin-authenticated visibility into every tenant's BullMQ job payloads and structured logs, unscoped by tenant and outside the audited-access model Section 4 otherwise enforces for every other path into tenant data — a real gap given some job payloads carry Student PII or payment detail (invoice generation, notification fan-out, the raw Stripe event on stripe-webhook-processing). Closed in three parts, deliberately not just an audit log bolted onto an unscoped tool: (1) the internally-triggered jobs (invoice generation, notification-fan-out, membership-expiry reminders, QR-attendance processing, waitlist-promotion, franchise-fee-usage-reporting, credential-purge, class-occurrence generation, booking-no-show expiry, dashboard-KPI aggregation, tenant-app-credential-grace-period purge) now carry only IDs/references in their BullMQ payload and log lines, never raw PII or financial data — the processor re-fetches the real data from Postgres, RLS-scoped, at execution time, so there is nothing sensitive sitting in Redis or a log line to browse; (2) stripe-webhook-processing and otp-delivery, which unavoidably carry a raw external payload, now require each job to carry a stable schoolId, and Bull Board's job-detail view (not its queue-level stats, which stay open) is scoped and audit-logged the same way Decision 39 scopes impersonation — an admin declares which tenant/investigation they're viewing for, and that view is written to the Section 4.7 audit log; (3) CloudWatch Logs, which can't practically be gated per-view, gets a logging-discipline rule instead: log lines carry job metadata (jobId, schoolId, X-Request-Id, status, timing) but never a raw payload body. Queue-level stats and the failed-job/oldest-waiting-job CloudWatch alarms Decision 31 already established are unaffected — the operational-visibility purpose they exist for never required seeing payload content. Full detail in Section 4.7, Section 9, Section 11.6.
- **Pass 8 Round 2, 26 Aug 2026 — “Active student” defined; trial-duration cap added to close the gap it creates (Decision 54):** Section 6.1/10.2's white-label metered add-on and the Franchise's Per-Headcount fee both bill per active student with no definition of the term anywhere. A Student now counts as active at a given School if they hold at least one Membership there with status=Active and type in Subscription, Class Pack, or Weekly Pass — built entirely from the existing Membership.status field, no new signal invented. Friend Pass (explicitly gifted, always £0) and Trial Membership are excluded from the count, so a School isn't billed — by its Franchise or by UTM8 — for goodwill guest visits or prospects it's still trying to convert. Counted per School, not deduplicated across Schools under one Franchise; computed as a point-in-time snapshot at the usage-reporting job's existing T-1-day offset (Decision 35, Decision 40), not summed across

the billing period, so a Student who churns and rejoins mid-month isn't double-counted. Excluding Trial from the count opens a gap of its own — nothing stops a School from keeping a Student on “Trial” indefinitely to dodge the headcount while still granting full product access — so MembershipPlan.expiry for type=Trial Membership is now capped at a platform-enforced maximum of 30 days; a School still sets its own trial length within that ceiling, the same configurability every other plan type already has, validated at MembershipPlan creation/update. A Trial Membership expires on exactly the same live-computed, date-based mechanism as every other type (Decision 26) — no new expiry mechanism, just the existing one with an upper bound. Full detail in Section 5.3, Section 6.1, Section 9, Section 10.2.

- **Pass 8 Round 2, 26 Aug 2026 — Stripe chargeback/dispute handling specified (Decision 55):** charge.dispute.created/updated/closed were entirely unhandled — no webhook, no status value, not even flagged in Section 12.2 — despite disputes clawing back money Stripe had already confirmed as settled. All three events are added to the webhook list on the existing stripe-webhook-processing queue, deduped on Stripe event id the same as every other event; the handler is idempotent to retry and out-of-order delivery by design — it sets local dispute state to whatever status Stripe's own event payload reports (needs_response/under_review/won/lost) rather than treating created/closed as sequential triggers, so a delayed or replayed event can never leave a Transaction stuck frozen or wrongly resolved. This applies across whichever of Transaction, FranchiseFeeCharge, or PlatformCharge the disputed charge id resolves to (Decision 38's Connect-account derivation), each gaining a Disputed status value distinct from Refunded and a nullable disputedAmount field taken straight from the event payload; only Stripe-paid charges are affected, since Cash/Bank Transfer rows have no Stripe charge id to match. While Disputed, POST /payments/refund and POST /bookings/{id}/credit-restore reject any credit-record tracing back to that row's Membership — not only the one credit-record the dispute nominally concerns, since a single purchase can back many separately-resolved Bookings — with an explicit reason recorded, the same explicit-failed-attempt-plus-staff-notification pattern the Refund-API-failure decision (Pass 4) already established. Membership/Booking access is left unchanged while a dispute is pending, mirroring how a failed subscription renewal already rides Stripe's own retry window before anything is revoked. On resolution: won reverts the Disputed status and lifts the freeze; lost force-Expires the Membership through the existing Expiry mechanism, and for a Subscription-type Membership additionally cancels the underlying Stripe Subscription object, so a customer who has just disputed and lost isn't billed again on the next cycle. Notification goes to whoever is financially exposed — School Owner/Manager for a disputed Transaction, Franchise Owner for a disputed FranchiseFeeCharge (mirroring the existing refund-authorization split between the two) — and a disputed PlatformCharge gets a dedicated CloudWatch alarm rather than relying on Bull Board's Decision 53 job-detail visibility, since Platform Admin has no tenant-style notification-fanout inbox of its own to route it to and this is UTM8's own revenue. Deliberately out of scope: how a School actually submits dispute evidence depends entirely on the still-open Express-vs-Standard Stripe Connect decision already flagged in Section 12.2 (the same fork Decision 4 raised for retry-schedule control) — Standard accounts would handle it in their own Stripe Dashboard, Express accounts might need something UTM8 builds, and this isn't the place to guess which; Stripe's own dispute-fee accounting is already visible in Stripe's reporting and isn't duplicated here. Accepted risk, not a gap this closes: a dispute can arrive well after UTM8 has already processed a refund or credit-restore on the same Transaction through the normal flow — the freeze only blocks future refund/credit actions from the moment the Disputed status lands, and can't retroactively undo one that already completed, since dispute timing is controlled by the cardholder's bank, not by UTM8. Full detail in Section 6.1, Section 9, Section 10.2, Section 12.2.
- **Pass 8 Round 2, 26 Aug 2026 — Money-transmission claim downgraded from resolved to pending legal confirmation (Decision 56):** Section 10.2 and Section 12.1's own Pass 4 entry both asserted the Franchise-fee destination-charge structure 'introduces no new money-transmission exposure' as settled fact, citing Section 10.4's Stripe Connect model — but that citation actually established credential custody (tenant secret keys never held by the platform), a real but distinct question from funds custody, which is what money-transmitter law actually turns on. The underlying architecture is sound and unchanged: both the Membership-purchase (Student→School) and Franchise-fee (School→Franchise) flows already use Stripe Direct/Destination charges that settle directly with the receiving party, never touching UTM8's own balance — the correct default regardless of how the legal question resolves. What's corrected is the conclusion drawn from it: both

passages now state this structure is intended to avoid money-transmission exposure rather than asserting it definitively does, and a new Section 12.2 item captures money-transmitter-exemption confirmation as its own still-open legal question, scoped to both payment flows rather than only Franchise-fee, since the same architecture and the same open question apply equally to the platform's higher-volume, consumer-facing Membership-purchase flow. Full detail in Section 10.2, Section 12.2.

- **Pass 8 Round 2, 26 Aug 2026 — Guardian consent recorded as a first-class, revocable artifact (Decision 57):** Guardian consent for a linked minor was never a distinct, auditable record — only WaiverSignature existed, which is School-authored liability/assumption-of-risk text, a different legal concept from consent to collect a child's personal information, and School-scoped where data-collection consent is properly platform-scoped. A new ConsentRecord entity is added: Guardian reference, linked-minor (Student) reference, a policyVersion field naming the specific privacy-notice/data-practice description consented to (a new data-collecting feature is a new policyVersion requiring fresh consent, not silent inheritance), a timestamp, and a status of Active/Withdrawn with its own withdrawal timestamp — matching why AuditLogEntry is its own entity rather than a status field elsewhere: consent needs a demonstrable trail, not a boolean. It carries two tiers, not one: a baseline tier covering the general account/attendance/grading data every Guardian-linked minor account requires, and a separate camera-consent tier tied specifically to the confirmed QR-check-in and profile-photo-capture use cases — mirroring a distinction the mobile Permissions screen already draws by requesting camera access separately from other permissions, not a new granularity invented here. Location, Contacts, and Calendar are deliberately not modeled as their own consent tiers, since no confirmed feature uses any of them yet (Section 12.2's existing Contacts & Calendar item already tracks this); a tier gets added if and when one of those is confirmed as a real feature, the same way camera earned one now. Verification reuses the Guardian's own already-OTP-verified account as the consent context, rather than introducing a new heavier verification mechanism nobody asked for. Withdrawal is scoped per Guardian-minor pair, not per Guardian account — a parent withdrawing consent for one child must not need to delete their whole Guardian account or affect other linked children — and reuses two existing mechanisms rather than inventing new ones: it triggers the same RoleGrant-revocation cascade Decision 28 already applies on Guardian-account revocation, scoped to that one Student, and enqueues the same account-deletion-processing job Decision 44 already built, also scoped to that Student — this is now baseline-tier behavior specifically; camera-tier-only withdrawal was later made proportional by Decision 69, which does not force this cascade. Once a Student is no longer Guardian-linked — whatever the eventual mechanism for that transition turns out to be, still open in Section 12.2's Guardian age-threshold item — its ConsentRecord simply stops being the operative authorization; this doesn't resolve that transition's own mechanics, only states the direct consequence for consent once it happens. This entity is deliberately not COPPA-specific: the age-threshold item already notes children's-data law is jurisdiction-dependent, and GDPR Article 8, UK-GDPR, and LGPD all impose a broadly similar shape — verifiable, revocable, tied to a specific description of data use — so ConsentRecord is built to that general shape, while which age threshold and which verification methods are legally sufficient in which market stays exactly where it already was, in the existing Guardian age-threshold open item, not resolved here. No backfill/migration handling is needed — this is a pre-launch spec with no existing production data — and the policyVersion mechanism already doubles as the answer for any future feature that needs fresh consent after launch. Full detail in Section 6.1, Section 12.2, Section 12.1 Decision 69.
- **Pass 8 Round 2, 26 Aug 2026 — videoScreen/Lesson video-content feature confirmed (Decision 58):** The mobile app's undefined videoScreen and its inconsistent VIDEO nav tab (Section 12.2) are confirmed as a real, planned feature rather than a design-file artifact: short instructional videos, each tagged with the specific skill they teach, surfacing automatically wherever that skill already appears (skill library, a student's profile, the grading flow itself) — the same shape the working grading-system prototype (Section 2.3) already validates with its own Lesson concept, not a new invention. Both formats are in scope: prerecorded videos an instructor uploads ahead of time, and live-streamed sessions. The Lesson entity (Section 6.1) and CurriculumModule (Section 7) move from candidate to confirmed, with Lesson gaining four fields this required: format (Prerecorded/Live), videoRef (the hosted video/stream target — see the new Section 12.2 item below, since where that video actually lives isn't chosen yet), and a captionStatus/captionTrackRef pair. Captioning is an UTM8-built automated pipeline, not a manual-upload or third-party-vendor-transcript

workflow, and the two formats need different mechanics: a Prerecorded Lesson triggers an async speech-to-text job on upload — non-blocking, so the Lesson can publish immediately with captionStatus Pending/Processing and the caption track attaches (captionStatus Ready) once the job completes, rather than holding up the instructor's publish flow — while a Live Lesson needs real-time auto-captioning during the stream itself, which is only as good as whatever live-streaming infrastructure ULTM8 eventually picks (also the new Section 12.2 item below), since not every streaming stack exposes a real-time captioning hook. Accepted, not solved here: automated speech-to-text has a real accuracy ceiling against accents and martial-arts-specific vocabulary (rank/technique names an off-the-shelf model won't know) — the standard mitigation for this class of tool is a custom vocabulary/glossary feed the transcription service is seeded with, a feature every major automated-captioning provider in this space already offers, so this is a configuration decision to make when a vendor is chosen, not new engineering. Deliberately out of scope, each now its own Section 12.2 item rather than resolved by assumption here: which video-hosting/live-streaming vendor videoRef actually points at (Cloudflare R2, already confirmed elsewhere, only covers images/PDFs, not video), and audio description (WCAG 1.2.5) — a distinct, far-less-automatable requirement this captioning pipeline does not satisfy. Full detail in Section 6.1, Section 7, Section 12.2, Section 12.1 Decision 59.

- **Pass 8 Round 2, 26 Aug 2026 — Accessibility target set: WCAG 2.2 AA (Decision 59):** Accessibility had no specified target or process anywhere in this document — a real gap for a consumer product with a mandatory account-creation flow. Target: WCAG 2.2 Level AA across the platform, differentiated by surface rather than treated uniformly, since the three surfaces aren't equally exposed: the Student mobile app gets highest priority — consumer-facing, React Native's native accessibility APIs (VoiceOver/TalkBack) are the implementation path; the School Portal (web) uses ARIA/semantic HTML — staff-facing B2B, but still a workplace tool subject to the same workplace-accommodation expectations as any other staff software; Platform Admin gets a lighter accommodate-on-request posture, since it's ULTM8-internal-only with a small, known user base rather than a mandatory-signup consumer surface. This isn't all new work invented here — two confirmed pieces of the platform already partially satisfy it: Rank's primary/secondary-colour fields (Section 6.1) sit alongside a name and ladder-position for every rank, so a non-colour differentiator already exists in the data model (WCAG 1.4.1, Use of Color) — worth confirming the belt/rank UI actually surfaces name/stripe-count as text or icon rather than a colour swatch alone, not a data-model change; and the Instructor/Staff-scoped attendance-override endpoint (Section 7, PATCH /bookings/{id}/override) already gives a student who can't complete a QR scan a staff-assisted path to a marked-Completed Booking, which is the practical accessible alternative WCAG expects for a physical-scan interaction, with no new engineering needed. One item needs confirmation rather than being settled here: WCAG 2.2 adds Accessible Authentication (3.3.8, Level AA), which restricts login flows built around a cognitive test like remembering or transcribing a code unless an exception applies — the existing 6-digit-Passcode-plus-SMS-OTP pattern likely qualifies under the object-recognition/essential exception, but this should be confirmed explicitly once the still-open Passcode-security-model item (Section 12.2) resolves, since that decision also determines exactly what mechanism 3.3.8 is being checked against. Video accessibility, previously a deferred "when confirmed" note, is now split cleanly by Decision 58: captions (WCAG 1.2.2, Level A) are covered by the confirmed automated pipeline; audio description (WCAG 1.2.5, Level AA) is not, and is tracked as its own open item (Section 12.2), since it's a different and far less automatable technology than captioning, not a variant of the same solved problem. Process commitment: automated accessibility scanning (e.g. axe-core for web, an RN-equivalent linter for the mobile app) runs in CI in advisory, non-blocking mode, mirroring Decision 18's CodeQL precedent and the same reasoning — a small team can't sustain automated a11y findings as a hard merge gate given their false-positive rate. Manual assistive-technology-user testing is deferred to a pre-launch checklist item, with no vendor or cost pinned yet. One EU-specific timing note, not a current-breach one: the European Accessibility Act's compliance deadline (28 June 2025) has already passed as of this document's writing, but the EAA governs products placed on the market after that date — since ULTM8 hasn't launched, there's no grace period to retrofit after the fact, this needs to ship compliant from day one in the EU rather than needing correction post-launch. Separately, and not resolved here: the Account Settings screen list (Section 5) includes an "Accessible" entry alongside Security and Preferences with no corresponding screen anywhere in either Figma file — worth confirming directly with Code Studio whether this was meant to be an accessibility-preferences screen (text size, reduce motion, and similar) that was

never designed, or a leftover design-file label; flagged for follow-up, not guessed at here. Full detail in Section 4, Section 5, Section 6.1, Section 7, Section 8.2, Section 12.1 Decision 58, Section 12.2.

- **Pass 8 Round 2, 26 Aug 2026 — Minimum-supported-client-version policy defined (Decision 60):** No policy existed for how long an old mobile-app build stays compatible with the API — a real gap given the white-label architecture: every core release fans out into N per-School branded rebuilds (Section 5.4), each triggered on that School's own schedule, not ULTM8's, so the API can accumulate a long, unpredictable tail of in-the-wild client builds it never controlled the update timing for. One mechanism covers both distributions — the shared ULTM8 app and every branded rebuild are the same codebase, parameterized per tenant (Section 5.4), not a fork — so there's a single system-wide floor, not two separate policies; what differs is only how long each side's tail of old builds practically runs, short on the shared app since ULTM8 controls its own release and adoption cadence end-to-end, long on the branded side since each School controls its own rebuild. Scoped to the two mobile distributions only: the School Portal and Platform Admin are both browser-served with no per-tenant build fan-out, so every visit loads whatever's currently deployed — there's no stale-build-stuck-on-a-device problem for either, and this decision doesn't extend to them. The default posture is additive-only API evolution — new optional fields, new endpoints, deprecate rather than remove — so an old build keeps working indefinitely under normal change, the same reasoning this document already applies elsewhere when a cheaper default beats an expensive enforcement mechanism (Decision 18's advisory-not-blocking CodeQL gate, for the same small-team-can't-sustain-the-hard-version reason). The version-gate exists only for the rare case additive evolution can't cover — a required auth-flow change, a field a screen depends on being removed. Mechanism: every request carries an exact build number (X-App-Build, precise per-tenant rebuild, not a loose semver); the API tracks a single minSupportedBuild floor that only ever ratchets upward, never multiple historical per-change thresholds. Below it, a request is rejected with a structured body (CLIENT_UPDATE_REQUIRED, minRequiredBuild) rather than a bare status code, so the app can reliably render a forced-update screen instead of a generic error; the one hard exception is the version-check/config endpoint itself, which is never gated, so a fully-blocked client can still learn it needs to update. Below a second, lower deprecated-below value, the request still succeeds but carries a header the app renders as a non-blocking in-app update banner. A minSupportedBuild bump can only take effect after the corresponding deprecated-below banner has already been live for a minimum 30-day lead time — a judgment call, not a figure derived from anything already in this document (the closest existing precedent, Section 5.5's 90-day post-downgrade credential-retention window, governs a different concern, secret retention, not client compatibility, so its number isn't reused here, only its long-enough/short-enough shape of reasoning). That lead time carries real weight because of the other simplification this decision makes: per-endpoint version gating (letting only the specific changed operation block old clients) was considered and rejected as more machinery than a REST API with no existing versioning scheme needs right now — which means a bump blocks the entire app, not one screen, so the notice window is what keeps a global block from landing as a surprise. Accepted, not solved by this policy: a genuine security-driven break won't have the luxury of a 30-day notice window — that's an explicit, documented override path, not a case the standard policy claims to handle gracefully. Full detail in Section 4, Section 5.4.
- **Pass 8 Round 2, 26 Aug 2026 — App-level biometric/PIN re-lock added, mobile only (Decision 61):** The mobile apps specified login-time authentication and Owner-tier step-up MFA for specific sensitive actions (Decision 12) but nothing re-gates the app itself once a session is live — a real gap given what a logged-in session holds: saved payment methods, and now a Guardian's linked-minor profile and consent state (Decision 57). A device left unlocked exposes all of it with no further check. Added: an app-level re-lock, independent of the JWT/refresh-token session underneath — it gates the UI and, more concretely, the stored refresh token itself, not the session's validity. Mechanism: biometric (Face ID/Touch ID, Android biometric prompt) as the primary check, the device's own PIN/passcode as the automatic fallback — deliberately not a new ULTM8-specific PIN, since the app already treats a 6-digit Passcode as the actual account credential (Section 5.3) and a second app-specific PIN of similar shape would be genuinely confusable with it. This isn't only a UI overlay: mobile tokens already live in the OS keychain/keystore, not app storage (Section 11.5), and both platforms support binding a keychain/keystore entry's access to the device's own biometric/PIN unlock directly — so the check gates the token's retrieval at the OS level, not just what the app chooses to render, a real security boundary rather than app code that could have a bug or a bypass. Trigger: a fresh check on

every return from backgrounding, not a timed grace window — the specific platform API achieving that is a build-time choice between each OS's caching/re-prompt options, not pinned down here; a few seconds of debounce for trivial app-switching (copying a verification code and back) is a legitimate UX allowance, but a tolerance for interruption, not a security concession, and not the same thing as a real grace window. One uniform threshold applies to every role rather than a second, stricter tier for Owner/Manager and Franchise Owner — a tempting-looking split, but rejected: the specific risk that tier would exist to cover (payout/bank-detail changes, refunds) is already gated by Decision 12's step-up MFA regardless of how recently the app was opened, so a stricter app-lock timer on top of it would duplicate coverage rather than add any, without covering a risk that isn't already covered. Real residual, not resolved here: that argument holds for Owner/Manager and Franchise Owner because Decision 12 already covers their specific sensitive actions, but it doesn't extend to Guardian — Decision 57 deliberately reuses the Guardian's ordinary OTP-verified login as its consent context rather than adding a heavier gate, so for Guardian specifically this uniform re-lock is the only mechanism standing between an unlocked device and both a saved-payment-method purchase and a consent-record change for someone else's child, not one layer among several the way it is for Owner-tier. Whether Guardian-specific actions (consent withdrawal, payment reuse) warrant their own step-up extension to Decision 12 is a related but separate question, deliberately not decided here rather than silently assumed as already covered. Scoped to the two mobile distributions only, the same scoping this pass applied to Decision 60: the web apps (School Portal, Platform Admin) hold tokens in httpOnly/Secure/SameSite cookies, not device-resident storage (Section 11.5), have no biometric-hardware equivalent, and an open browser tab is a different, already-standard problem the existing cookie/token-TTL mechanism already covers — not an open gap this decision needs to solve. Full detail in Section 11.5, Section 12.1 Decision 12, Section 12.1 Decision 13, Section 12.1 Decision 57.

- **Pass 8 Round 2, 26 Aug 2026 — Staging/test-data provisioning strategy defined (Decision 62):** Section 11.6 resolved where dev/staging/production live (AWS: RDS/ElastiCache/ECS Fargate) but never said what data populates them — a real gap given what's now in this model: WaiverSignature, Decision 57's Guardian ConsentRecord, and R2-held signed-waiver and invoice PDFs. The common default of copying production into staging is also the most common way regulated data ends up somewhere with weaker access controls and more people able to see it, and that risk is sharper here specifically because GDPR/LGPD compliance is already a confirmed open item (Section 12.2, data residency) — quietly copying the same regulated data into a second, lower-security environment is exactly the kind of processing-outside-the-consented-purpose that item already flags for the primary environment. Default: synthetic seed data, never a production copy, as standing infrastructure for both local dev and staging alike — one deterministic, schema-driven seed/factory script serving both, covering the real enum/status space the app needs to exercise, rather than hand-authored fixtures that go stale as the schema evolves. This is deliberately not a one-time factory: when a real bug surfaces in production, the fix includes adding a synthetic fixture that reproduces its shape, not the customer's actual data, so genuinely weird real-world cases (a Guardian linked across two Schools, an orphaned FranchiseFeeCharge from a School that changed affiliation) become permanent regression coverage without the seed corpus ever needing a real record. Payment data needs no synthesizing at all, structurally rather than by policy: staging runs on Stripe's own separate test-mode credentials and test cards, the same way ULTM8 itself never touches raw card numbers in production (Section 10.4) — there's nothing payment-shaped to accidentally copy because the two environments are wired to different Stripe accounts entirely. Object storage gets the same treatment: staging holds its own small set of synthetic sample PDFs/images, never customer waiver or invoice files, independent of R2's own backup/versioning story still being a loose end (Section 13). Rejected: anonymized-production-copy as the default, the other common industry pattern — correctly anonymizing a relational, multi-tenant dataset with cross-references (a Guardian linked to a minor, RLS tenant boundaries, a consent record tied to both) is genuinely hard to get right, and given GDPR/LGPD compliance is already unresolved for this platform, adding whether an anonymization pipeline is itself legally sufficient as a second open legal question compounds risk instead of reducing it. The rare real-data need — debugging an actual customer-reported issue — doesn't loosen the staging default; it gets its own narrow path reusing mechanisms this document already built rather than new ones: a break-glass action restricted to the Full Platform Admin sub-role, the only tier that can invoke break-glass at all (Section 4.4), which itself writes an AuditLogEntry naming the specific tenant and data subjects touched, not just tenant-level (Decision 43's data-subject granularity), and doesn't persist the extract in the shared staging

environment afterward. Directly relevant to, and cross-referenced from, the still-open Data residency/GDPR item (Section 12.2) — this decision doesn't resolve that item, but a synthetic-first staging policy is evidence toward how it eventually will. Full detail in Section 4.4, Section 10.4, Section 11.6, Section 12.1 Decision 43, Section 12.2.

- **Pass 8 Round 3, 26 Aug 2026 — RDS Proxy connection-budget verification required (Decision 63):** Decision 30 added a per-Fargate-task connection-pool ceiling in front of RDS Proxy, stating this prevents a booking-rush autoscale event from exhausting RDS's own connection limit — but nothing bounds total task count for either the API tier or the BullMQ-worker tier (no stated ECS autoscaling maximum for the API tier, no stated worker-fleet sizing), and Decision 51's load-test plan only scripts API-tier booking/credit-claim traffic, never worker-tier concurrency. Whether the per-task ceiling alone actually delivers on Decision 30's stated claim depends on specifics of how RDS Proxy's connection pooling behaves under this stack's traffic pattern (Prisma's query/transaction shape in particular) that shouldn't be asserted as settled fact here — that's a build-time verification question against current AWS RDS Proxy documentation and Prisma's own connection-pooler compatibility guidance, not something resolved by architectural reasoning in a spec review. Two things don't need to wait on that verification, since they're worth doing on their own merits regardless of how it resolves: an explicit ECS autoscaling maximum for the API tier (bounds worst-case AWS spend and blast radius, independent of the connection-pooling question), and confirmation of whether the worker fleet is fixed-size or itself autoscales, with the same kind of ceiling applied if it does. Decision 51's load test is the actual arbiter, not documentation-level reasoning: extended to run at the stated autoscaling ceiling, with whatever Prisma/RDS Proxy pooler configuration the verification step lands on, concurrently with worker-tier peak load (at minimum franchise-fee-usage-reporting and dashboard-KPI-aggregation, the heaviest-touching jobs) rather than validating each tier in isolation — whether RDS's connection count actually stays bounded under that combined test is the real answer, independent of which specific pooling mechanism turns out to be responsible. Full detail in Section 11.6 Decision 30, Section 12.1 Decision 32, Section 12.1 Decision 51.
- **Pass 8 Round 3, 26 Aug 2026 — Trial-reissuance fee dodge closed (Decision 64):** Decision 54 defined “active student” (the shared basis for both the Franchise's Per-Headcount fee and ULTM8's own white-label metered fee) and capped a single Trial Membership at 30 days so a School couldn't leave a Student on Trial indefinitely. It didn't cap how many times a School can re-grant Trial to the same Student: nothing stops issuing a fresh 30-day Trial the moment the previous one expires, chaining Trials back-to-back forever. Because both fees key off the identical active-student definition, this one gap dodges two separate revenue streams at once while the Student keeps full product access throughout. A hard lifetime cap (one Trial ever, per Student per School) would close it completely but doesn't match how these businesses actually operate — re-offering a trial to a prospect who dropped out and is taking another look years later is normal, legitimate use, not abuse. Closed instead by extending Decision 54's existing mechanism rather than adding a new one: the same 30-day ceiling now applies cumulatively across all Trial Memberships for a given Student-School pair within a rolling 12-month window, not per individual Trial record. A School can still re-trial a lapsed prospect at any time; what it can't do is chain Trials back-to-back to stay permanently under the billable threshold. Enforced at the same point Decision 54 already validates — MembershipPlan creation/update for type=Trial Membership — by summing the Student's Trial days at that School over the trailing 12 months and rejecting a new Trial that would push the total past 30. Full detail in Section 6.1, Section 9, Section 10.2, Section 12.1 Decision 54.
- **Pass 8 Round 3, 26 Aug 2026 — Trial-volume visibility added at the incentive-aligned level (Decision 65):** Decision 64's cap is keyed to a Student identity that costs nothing to throw away and recreate — there's no email/phone uniqueness constraint on User registration, and Decision 44's account-deletion job anonymizes personal fields, so delete-and-re-register produces a Student with zero Trial history and no way for Decision 64's 12-month check to see across that boundary. A technical dedup strong enough to catch this (matching hashed contact info across deleted accounts) needs a key-based HMAC, not a plain hash — email and phone are low-entropy enough that a plain hash is practically reversible by guessing candidates — and even done correctly, the result is pseudonymized, not anonymized, data under GDPR, since ULTM8 itself holds the means to re-match it; retaining that against an erasure request is a real, unresolved legal question, not one this review can settle. Nor does it help to rely on staff simply recognizing a repeat visitor: this platform confirms multi-branch Franchises and remote video-based Lesson access (Decision 58), so there's no reliable

in-person backstop to lean on. The deciding factor against building the technical fix is incentive, not just cost: the abuse costs the Franchise its Per-Headcount fee and costs UTM8 its white-label fee — it doesn't cost the School anything, so a signal visible only to the School lands with the one party that has no financial reason to act on it. Added instead, at near-zero cost: the existing dashboard-KPI-aggregation job (Section 9) is extended to surface Trial-to-paid conversion-rate and Trial-volume trends at the Franchise level for Franchise-affiliated Schools, and as a Platform Admin-visible metric for the white-label-fee side — the two parties actually losing revenue, not the School. This is a weak, noisy signal, not a precise one — a conversion-rate drop has many innocent explanations — and is not itself presented as closing the loophole; see the new Section 12.2 item for the harder question this doesn't resolve. Full detail in Section 9, Section 12.1 Decision 44, Section 12.1 Decision 54, Section 12.1 Decision 64, Section 12.2.

- **Pass 8 Round 3, 27 Aug 2026 — QR check-in liveness gap bound to the existing open design item (Decision 66):** The Student's own camera scans an externally-displayed code, not the reverse (Section 2.2); nothing today stops photographing that code and sending it to someone who isn't physically present, letting them claim grading credit (StudentRank.classesAttended, via the qr-attendance-processing job) for a class they didn't attend. Closing Walk-in attendance to a pre-existing Booking only (already resolved, Section 12.2) doesn't touch this — the exploit still needs a valid Booking, just not a physically-present scanner. What the scan writes on success is fully specified (Booking marked Completed, StudentRank incremented, no separate Attendance entity); what the code itself contains, and who generates or displays it, has never been specified — this is the same gap already flagged in Section 12.2 as the School Portal's "QR Code" sidebar item having no designed screen in either Figma file. Where that screen belongs isn't a fresh guess: only three clients are confirmed (Section 3.1) — School Portal, Platform Admin, and the Student app — and Instructor's own School/Branch-scoped login (Section 8, Pass 2) has no client to run in except School Portal, so a staff-facing QR display sits there, most plausibly as a check-in-station browser session rather than a personal device. This decision doesn't reopen whether "QR Code" gets designed this phase — that's still open per Section 12.2 ("confirm whether these need to be designed before backend work starts, or scoped out of this phase") — it only binds what happens if and when it is: the screen must not default to a single static code for the whole Class occurrence, since that reproduces today's exact spoofing vector with a design behind it instead of closing it. It should instead display a time-boxed, rotating code for the duration of the Class occurrence, closing the photograph-and-send-remotely vector while leaving the default self-service scan flow unchanged for students who are actually present. The specific rotation interval is an implementation-time tuning parameter, not asserted here. The existing Instructor/Staff-scoped attendance-override endpoint (Section 7, PATCH /bookings/{id}/override) remains the accessibility fallback regardless, unaffected by this decision. Full detail in Section 2.2, Section 3.1, Section 6.1, Section 7, Section 8, Section 9, Section 12.2.
- **Pass 8 Round 3, 27 Aug 2026 — Unverified self-reported DOB bypassing Guardian consent closed via a shared entry guard (Decision 67):** Independent Student registration (registerScreen: email/phone, 6-digit passcode, social login, SMS OTP) has no age gate anywhere in it; User.DOB is only collected afterward, in the mobile app's "personalize your plan" wizard (Section 2.2), and nothing reads it back to route the account anywhere — the Guardian/consent model only ever engages when a Guardian account proactively creates a linked minor profile (Section 8.2, Pass 2). So a minor can hold a fully independent account — real payment method, real Class bookings, a self-signed WaiverSignature (typed name plus signature, no ID check, Section 2.2/6.1) — with Decision 12 and Decision 57's consent protections never engaging. Closed with one shared precondition rather than three separate patches: whether a Student is self-attested-adult by DOB, or Guardian-linked, checked lazily at the first point the account attempts a consequential action — Membership purchase, a booking-triggered payment, or waiver-signing, whichever comes first — rather than gating registration itself. This requires DOB to be a mandatory precondition of that first gated action, closing the open question of whether the onboarding wizard can currently be skipped at the point it actually matters rather than at registration. If the check fails, that action is blocked with a redirect toward proper Guardian-linked enrollment. Deliberately not done, and why: DOB collection is not moved into the registration screen itself, and an existing independent account is never retroactively converted or migrated into a Guardian-linked one — the confirmed model already commits to a one-way-only account-type transition (age of majority moves linked to independent via a fresh self-registration, never the reverse, Section 8.2), and building a reverse migration path would be new capability against that precedent, not enforcement. A

blocked self-registered account is therefore a practical dead end: the family completes a proper Guardian-linked profile separately, the original account likely goes unused, and no reconciliation or merge mechanism is being built for that — an accepted minor loose end, not a hidden defect. Residual risk, explicitly accepted rather than solved: nothing here catches someone who deliberately misrepresents their own DOB at entry — the same self-attestation limit essentially every consumer platform accepts, including under COPPA's own "actual knowledge" framing, not a gap UTM8 is expected to close with identity verification, which would be disproportionate for this platform. Full detail in Section 2.2, Section 6.1, Section 8.2, Section 9, Section 12.1 Decision 12, Section 12.1 Decision 57, Section 12.2.

- **Pass 8 Round 3, 27 Aug 2026 — Serial chargeback / friendly-fraud pattern closed via a system-internal job (Decision 68):** Decision 55 handles per-Transaction dispute mechanics well — idempotent webhook handling, a Disputed status, force-Expiry on loss — but tracks nothing across Transactions. Nothing distinguishes a one-off card-fraud incident from a Student who repeatedly attends class, pays via Stripe, then disputes the charge as unauthorized. A lost dispute kills that one Membership, but nothing stops re-registering the next day and repeating it — the same shape of gap Decision 64 already closed for Trial reissuance, left open here on the payment side. Closed with a new chargeback-pattern-restriction background job (Section 9), mirroring the platform's existing pattern (dashboard-KPI-aggregation, credential-purge, qr-attendance-processing: computing internally across tenant boundaries, surfacing only a consequence, never raw cross-tenant data): it counts lost disputes per Student across all their RoleGrants platform-wide, and past a threshold — a business/implementation parameter, not asserted here — restricts that Student to Cash/Bank Transfer payment going forward, the only methods this build supports that cannot be charged back, since they're staff-confirmed manual records with no card network involved (GoCardless remains excluded, Decision 5). Global platform-wide scope is deliberately fine despite the standing invariant that one School's data is never visible to another — this job never exposes cross-School data to any human view; each School only ever sees the restriction already applied to a Student they already have legitimate visibility into, the same as any other account restriction. On trigger, the affected Student's current School(s) are notified via the existing notification-fanout job (Section 9), the same channel Decision 55 already uses for dispute notices, so staff aren't confused when a card option disappears. What this deliberately doesn't solve: a Student who wants to evade it can simply register a fresh account, resetting the count to zero — the identical account-recreation loophole Decision 65 already found and left honestly unresolved for Trial abuse, not solved differently here. This is a proportionate backstop for the default case — particularly valuable for higher-volume Schools or multi-Branch setups where no single staff member sees the whole pattern — not a claim that determined, sophisticated abuse is fully closed; a School's own informal awareness of repeat local customers remains a first line this doesn't replace. Full detail in Section 9, Section 12.1 Decision 5, Section 12.1 Decision 55, Section 12.1 Decision 64, Section 12.1 Decision 65.
- **Pass 8 Round 3, 27 Aug 2026 — Decision 57's consent-withdrawal cascade made proportional to tier (Decision 69):** Decision 57's ConsentRecord deliberately carries two tiers — baseline, and a camera-consent tier bundling QR-check-in and profile-photo-capture together, since OS-level camera permission on iOS/Android is a single per-app grant, not scopable per feature, not a new granularity invented there. But its withdrawal mechanism doesn't respect that split: any withdrawal triggers the full Decision 28 RoleGrant-revocation cascade plus the Decision 44 account-deletion-processing job, uniformly — correct for baseline withdrawal, which underpins the account's existence at all, but disproportionate for camera-tier-only withdrawal, which would delete a Student's entire account and grading history over a Guardian's narrower preference about camera use specifically. Fixed by splitting the consequence by tier: baseline-tier withdrawal keeps the existing full cascade unchanged. Camera-tier-only withdrawal, with baseline remaining Active, instead marks only that ConsentRecord Withdrawn, blocks future self-service check-in at POST /attendance/scan (Section 7) by checking that Student's camera-tier ConsentRecord status server-side before accepting a scan — the mobile UI should also stop offering the scan option as a courtesy, but that's not the enforcement, a consent boundary can't rely on a client choosing to respect it — and clears any already-stored profile photo, since it was only ever lawfully collected under the now-withdrawn consent basis. RoleGrant, account, and grading history remain untouched. The Instructor/Staff-scoped override endpoint (PATCH /bookings/{id}/override), already built as the accessibility fallback, becomes that Student's ongoing path to check-in. Accepted cost, not solved here: routing every future check-in through staff override is a real,

recurring operational load on School staff for as long as the withdrawal stands — that endpoint was sized for occasional accessibility use, not this. No non-camera self-service alternative exists in the confirmed architecture to avoid it, and inventing one is out of scope here; flagged instead as its own new Section 12.2 item, since it would serve both this population and the pre-existing accessibility one. A second, separate new Section 12.2 item also flags that no consent-management interface — for withdrawing either tier, or viewing consent status at all — exists anywhere in the confirmed designs; this decision specifies backend behavior ready for whenever such a screen is built, not a claim that Guardians can exercise it today. Full detail in Section 6.1, Section 7, Section 9, Section 12.1 Decision 12, Section 12.1 Decision 28, Section 12.1 Decision 44, Section 12.1 Decision 57, Section 12.1 Decision 59, Section 12.1 Decision 61, Section 12.2.

- **Logic review, 27 Aug 2026 — RoleGrant scope fields made role-specific:** RoleGrant's scope was documented only as prose — `school_id` and/or `branch_id` and/or `franchise_id`, "as applicable to the role" (Section 6.1) — with no enforced mapping of which columns a given role actually populates, and its role enum itself was stale: it listed "SchoolStaff" as a single value, a name that matches neither of the two roles Section 8.2 actually splits that access into (School Owner/Manager, School-wide; Branch Staff, Branch-scoped only) — Branch Staff was missing from the enum entirely. Fixed in Section 6.1 directly rather than left as decision-log-only prose: RoleGrant's role enum now lists all six actual customer-facing roles, and a database CHECK constraint maps the valid scope-column combination per role — including Guardian, which carries no tenant scope at all, since guardianship is a relationship to a specific linked minor Student profile, not a grant against a School, Branch, or Franchise. Full detail in Section 6.1, Section 8.2.
- **Logic review, 27 Aug 2026 — Bulk-grading endpoints given an explicit batch cap:** POST `/schools/{id}/ranks/bulk-promote` and POST `/schools/{id}/ranks/bulk-stripe-award` (Section 7) each grade multiple students in one call, with no documented limit on how many student ids a single request may include — unlike every list endpoint's cursor-based pagination, resolved platform-wide by Decision 22. A School running a single ceremony against its full roster could plausibly submit hundreds of students in one payload. Added: a fixed per-request cap (e.g., 200 students), with the School Portal client batching sequential calls beyond it — the same operability posture Decision 22 already established for reads, applied here to a bulk write instead. Full detail in Section 7, Section 12.1 Decision 22.
- **Logic review, 27 Aug 2026 — Decision 30's composite-index convention extended to fields added since:** Decision 30 established a composite index leading with the tenant column plus whatever's actually filtered or sorted on, for every RLS-scoped table — a migration convention, not a one-time list. Two fields added in later passes are exactly the shape that convention targets and were never explicitly called out: Transaction.status (Disputed, added Decision 55), queried per-Student platform-wide by Decision 68's chargeback-pattern job, needs an index on (`student_id`, `status`), not tenant-led, since that job's whole point is computing across tenant boundaries; and ConsentRecord's tier field (Decision 57, made withdrawal-proportional by Decision 69) is checked server-side on every POST `/attendance/scan` call and needs (`student_id`, `tier`, `status`) to keep that check off the request's hot path. Neither is a literal instance of Decision 30's tenant-leading pattern — both are justified departures from it, not applications of it: the first is deliberately not tenant-led since Decision 68's job spans every tenant by design; the second doesn't need to be, since it's the RLS policy itself, not the index, that enforces tenant isolation, and a request already scoped to one Student benefits more from `student_id` leading. Flagged here as adaptations of Decision 30's underlying goal — index what's actually queried — rather than literal applications of its tenant-first phrasing, so neither is missed at migration-writing time. Full detail in Section 6.1, Section 12.1 Decision 30, Section 12.1 Decision 55, Section 12.1 Decision 57, Section 12.1 Decision 68, Section 12.1 Decision 69.
- **Logic review, 27 Aug 2026 — Scheduled jobs' run time made timezone-aware, consistent with Decision 25:** Decision 25 resolved that every date-only comparison resolves against a School's own IANA timezone, not server-local or UTC wall-clock time — but Section 9's own scheduled jobs (dashboard-kpi-aggregation, apple-developer-membership-renewal-reminder, tenant-app-credential-grace-period-purge, franchise-fee-usage-reporting, among others) are each specified only as "daily"/"nightly"/"monthly", with no anchor time — implicitly a single global run instant. For a platform already spanning five markets across at least four distinct timezone bands (Decision 25's own corrected count), a single fixed run time lands at very different local hours per School — fine for jobs that aren't date-boundary-sensitive, but a silent misfire risk for the ones that are, franchise-fee-usage-reporting's own "fixed T-1-day offset before the charge" being the clearest case. Added:

day-boundary-sensitive jobs run late enough in UTC (or windowed) that the platform's furthest-west served timezone has already crossed local midnight before any per-School date logic evaluates, rather than assuming a single instant is "today" everywhere at once — extending Decision 25's own principle to the scheduling mechanism, not just entity-level comparisons. Full detail in Section 9, Section 12.1 Decision 25.

- **Logic review, 27 Aug 2026 — R2 bucket versioning added for legally-significant stored documents:** Signed-waiver and invoice PDFs are served from R2 directly (Section 11.3) — the platform's only durable copy of each. Unlike AuditLogEntry, which Decision 15 already hardens against accidental loss by revoking UPDATE/DELETE at the database layer, nothing protects these R2 objects from an accidental overwrite or delete via a buggy deploy or an over-privileged credential. Added: R2 bucket versioning (or equivalent object-lock) enabled specifically on the buckets holding waiver and invoice PDFs — mirroring the protective posture Decision 15 already applies to the audit log, for the same reason: both are records a School or regulator may need to produce later, not files it's acceptable to silently lose. Full detail in Section 11.3, Section 12.1 Decision 15.

12.2 Still open — please advise

- **Guardian minor age-thresholds are provisional, pending legal review (new, 25 Aug 2026):** the Guardian/linked-minor model (Section 6.1, Section 8.2) defaults a limited-login age of 13 and an age-of-majority self-registration cutover at 18. Both are placeholders, not confirmed legal positions — children's-data-privacy law and contract-capacity rules are jurisdiction-dependent, and the platform already spans multiple markets (Section 11.2). Needs real legal review before these ages ship. Related, added Pass 7: whether a linked-Student login under this age threshold can directly request the mobile Permissions screen's Device Location/Contacts/Calendar/Camera access (Section 11.5, and the Contacts & Calendar item below), or whether those should stay gated behind the Guardian's own account, needs the same legal review.
- **MFA for Owner-tier roles** — resolved 26 Aug 2026, Pass 7: step-up MFA (not blanket login MFA) now gates sensitive actions — payout/bank-detail changes, Auto-Refund, franchise-fee refunds — for School Owner/Manager and Franchise Owner, matching the risk these roles carry without adding login friction for a largely non-technical Owner persona. Full detail in Section 11.5, Section 12.1.
- **Object storage / CDN provider** — resolved 26 Aug 2026, Pass 5: Cloudflare R2 (zero-egress object storage) paired with Cloudflare Images for on-the-fly resizing/thumbnailing, for banner images, gallery photos, profile photos, signed-waiver PDFs, and invoice PDFs. Full detail in Section 11.3, Section 12.1.
- **SMS/OTP provider** — resolved 26 Aug 2026, Pass 5: Twilio Verify (the purpose-built OTP product, not Twilio's generic Messaging API); the on-screen OTP expiry ("1 minute and 17 seconds") remains sample data, not a spec — the actual expiry window is a config value. UAE currently has alphanumeric sender-ID registration on regulatory hold and Malaysia requires domestic sender-ID pre-registration that gets overwritten with a random operator-assigned numeric ID regardless (so registered branding never actually displays to the recipient), but Verify's managed routing still delivers OTPs reliably in both. Full detail in Section 11.4, Section 12.1.
- **Push notification & email providers** — resolved 26 Aug 2026, Pass 5: push goes direct to FCM (Android)/APNs (iOS) rather than a third-party aggregator; email uses Postmark for transactional delivery (monitoring note: 2026 reports of abrupt account suspensions post-acquisition, so a fallback provider is worth keeping in reserve). Full detail in Section 11.4, Section 12.1.
- **API style** — REST vs GraphQL is not specified. Section 7's module breakdown assumes REST as the idiomatic NestJS default; confirm before implementation starts.
- **ORM** — resolved 26 Aug 2026, Pass 8, Decision 32: Prisma — better NestJS ecosystem fit and more mature migration tooling. Full detail in Section 11.6, Section 12.1.
- **Passcode security model** — is the 6-digit "Passcode" the entire account credential, or a PIN layered on a longer password not shown in the designs? This changes both the auth architecture and the security posture materially.

- **Stripe Connect onboarding fit** — Section 10.4 commits to Stripe Connect as the primary model; still to confirm is whether Connect's own onboarding requirements (e.g. Express vs Standard accounts, per-country availability) fit every market ULTM8 plans to launch in, and which flows genuinely need the secrets-manager fallback. Higher-stakes as of 25 Aug 2026, Pass 4: this choice now also determines who controls a tenant's recurring-payment retry/dunning schedule (Section 10.2, Section 10.4) — resolving it is no longer just an onboarding-fit question. UAE checked directly, Pass 8 Round 3: Express accounts are confirmed available there for a foreign platform like ULTM8 (per Stripe's own current documentation), so Section 10.4's existing Express-or-Standard architecture holds without change — but UAE connected-account holders are required to provide a valid UAE trade license during verification, so an individual or informal instructor without a registered business entity likely can't onboard a School's PaymentAccount there. Other markets in Section 1.1's currency list haven't been checked the same way yet.
- **Branch data model** — "Branches" is a distinct sidebar item and appears in franchiseInformation's "Affiliated School Information," but no Branch-specific field screen (branchInformation / updateBranch) had extractable field detail beyond its screen name — confirm what data a Branch actually needs to capture. Partially resolved (confirmed 25 Aug 2026): Section 4.3's Branch-level RLS decision means Branch needs, at minimum, its own id and a school_id foreign key; full field capture (address/contact detail) is still open.
- **Sidebar items with no designed screen** — QR Code, Notification, Rules, Subscription History, Reports, and Customer Services all appear in the School Portal sidebar navigation but have no corresponding screen in the Figma file ("Rules" resolves to a single text label, not a screen). Confirm whether these need to be designed before backend work starts, or scoped out of this phase. QR Code specifically carries a binding constraint beyond just needing a design, per Section 12.1 Decision 66: if and when it is designed, it must display a time-boxed, rotating code per Class occurrence, not a single static code for the whole occurrence.
- **"User" role** — the translations CMS groups content under Student / School / Admin / User. The first three map cleanly to the role model in Section 8.2; "User" does not map to anything concrete in the designs — confirm what it refers to.
- **Franchise-disclosure-law legal review** — new, 25 Aug 2026, Pass 4: Stripe automation uses a destination-charge structure for Franchise-fee collection (Section 10.2) so funds settle directly with the Franchise, keeping the platform out of the custody chain — but whether that structure actually qualifies for a money-transmitter exemption in every jurisdiction ULTM8 operates in is a separate, still-open legal question (see the new money-transmission-exemption item below), not something the architecture alone settles. Independently, the Franchise-disclosure-law question — what a Franchisor may lawfully collect from a franchisee School under the FTC Franchise Rule and equivalent state/international disclosure laws — remains open too, flagged in Section 8.2 since Pass 2, still needs real legal review before this ships to any market.
- **Money-transmission exemption not legally confirmed (new, 26 Aug 2026, Pass 8):** Both the Membership-purchase flow (Student→School) and the Franchise-fee flow (School→Franchise) use a Stripe Direct/Destination charge structured so funds settle directly with the receiving party's own connected PaymentAccount, never touching ULTM8's platform balance (Section 10.2) — the standard pattern platforms use to stay out of money-transmitter territory, and the right technical default regardless of how the legal question resolves. But structuring the money flow correctly isn't the same as confirming the exemption actually applies: money-transmitter licensing is a fragmented, state-by-state and country-by-country regime, and whether ULTM8's specific use of destination charges qualifies for the relevant payment-processor/agent-of-payee exemptions in every market it operates in hasn't been confirmed by counsel. Needs real legal review before this ships to any market, the same standard already applied to the Franchise-disclosure-law question above.
- **School changing Franchise affiliation** — new, 25 Aug 2026, Pass 4: a School's PaymentAccount, MembershipPlan set, and (new) FranchiseFeeCharge history are all keyed to its Franchise relationship, but the document doesn't say what happens if a School leaves one Franchise for another, or becomes independent — does FranchiseFeeCharge history move, freeze, or get orphaned? Not designed; flagged rather than assumed.
- **Currency conversion** — resolved 26 Aug 2026, Pass 5: Student-facing MembershipPlan pricing stays independent per-currency, School-set, no automatic conversion; ULTM8's own revenue (platform

SubscriptionPlan, white-label add-on, Franchise-fee) all quote and bill in a single anchor currency (USD), with the card network's own FX handling conversion at charge time — the same pattern Franchise-fee already used. Full detail in Section 11.2, Section 12.1.

- **Late-cancellation fee mechanism** — new, 25 Aug 2026, Pass 4: Class and MembershipPlan both carry a cancellation charge field alongside the refund-fee date cutoff, implying a Student who cancels after that cutoff may owe a fee — but this pass resolved refunds (money or credit coming back to the Student), not a new charge going the other way. Whether and how that fee is actually collected (charged to the Student's saved card via Stripe, or simply forfeited value with no new charge) is a related but separate question, deliberately not answered here.
- **Ambiguous mobile Profile stats** — the Profile and Coach Profile screens show four unlabeled numbers (870 / 120k / 354k / 3.8Gb). Meaning is not stated anywhere in the designs (possible reads: posts/followers/following/storage) — needs product clarification before the Profile data model can be finalized.
- **Inconsistent mobile bottom navigation** — most screens show HOME / RANKING / SCHEDULE / SEARCH / NOTIFICATION / PROFILE; Coach Profile and video-adjacent screens show HOME / VIDEO / SEARCH / OFFERS / PROFILE instead. Confirm whether this is an intentional context switch (e.g. a social/content mode) or a design inconsistency to be resolved to one nav.
- **Lightly-specified mobile screens** — offerScreen, dealsDeals, summaryAndCart, and the map/search-filter screens exist but carry little to no field-level detail in the designs, so their data models are effectively undefined. videoScreen has been removed from this list — resolved via Decision 58 and the item below.
- **videoScreen/Lesson feature** — **RESOLVED, Decision 58 (26 Aug 2026, Pass 8)**: confirmed: UTM8 does build this feature — short instructional videos, each tagged with the specific skill they teach, surfacing automatically wherever that skill already appears (skill library, a student's profile, the grading flow itself) rather than living in a separate video library a student browses on its own — matching the working grading-system prototype's own Lesson entity (Section 2.3). This is now the confirmed answer to what videoScreen and the mobile app's inconsistent VIDEO nav tab were meant to be — see Section 6.1's Lesson entity and Section 7's CurriculumModule, both now confirmed rather than candidate. Full detail in Section 12.1 Decision 58.
- **Possible copy/paste artifacts** — a few items look like design-file artifacts rather than intentional features and should be confirmed rather than built as-is: two identical "Microsoft" login buttons on the same login/register screens; near-duplicate frames such as "createSchoolProfi" vs "createSchoolProfile" and "Forgot Password" vs "forgotPassword"; and a forgotPassword frame that contains two duplicated copies of the same form content.
- **No design system / token file** — both files use only generic community UI kits (Material 3, Simple Design System, Apple OS kits), not a bespoke component library. There are no documented design tokens (colors, spacing, type scale) to hand to the frontend team as a source of truth.
- **Environments & delivery** — no dev/staging/production, CI/CD, or hosting strategy was specified anywhere in scope for this document. Resolved 26 Aug 2026, Pass 5: AWS (RDS/ElastiCache/ECS Fargate) with GitHub Actions CI/CD gated by a required branch-protection status check on Section 4.8's isolation test suite; RDS automated backups with point-in-time recovery, Multi-AZ, for backup/DR; this also settles the Vault-vs-AWS-Secrets-Manager shortlist (Section 4.5, Section 5.4, Section 10.4) in favor of AWS Secrets Manager. Full detail in Section 11.6, Section 12.1.
- **Data residency / GDPR & LGPD compliance (new, 26 Aug 2026, Pass 5)**: the environments decision (Section 11.6) settles infrastructure but not jurisdiction — this platform spans the UK, EU, Brazil, UAE, and Malaysia, and AWS RDS/ElastiCache/ECS Fargate can be deployed in region-specific data centers, but nothing in this document yet specifies which tenant's data must stay in which region, or how GDPR (EU/UK) and LGPD (Brazil) data-subject rights (access, deletion, portability) get implemented against a single shared multi-tenant database using Row-Level Security rather than per-region physical separation. Needs real legal/compliance review before this ships to any market — same treatment as the Guardian minor age-thresholds item and the franchise-disclosure-law item, both already flagged for legal review rather than assumed. Sub-processor

disclosure is a related, distinct gap, added Pass 7: GDPR Art. 28 requires publishing a processor list, and Stripe, Twilio, Postmark, FCM/APNs, AWS (RDS/KMS/Secrets Manager, Section 11.6), and Cloudflare are all named data-processing vendors with none disclosed anywhere in this document — flagged here since both need the same legal-review pass. Related, added Pass 8: Decision 62's synthetic-first staging/test-data policy doesn't resolve this item, but is directly relevant evidence toward how it eventually will, since it settles that regulated data isn't processed a second time in a lower-security environment regardless of how the region/RLS question above resolves.

- **Contacts & Calendar permissions** — the mobile Permissions screen requests Phone Contacts and Google Calendar access, but no other screen shows a feature that uses either (e.g. no "invite from contacts" or "sync to calendar" flow was found) — confirm the intended feature before implementing these permission requests. Related, added Pass 7: this also needs to be scoped against a linked-Student login under the Guardian model's provisional age threshold (above) — whether a minor's own login can request these permissions directly, or whether they stay gated behind the Guardian's account.
- **White-label add-on pricing** — Section 5.3 and Section 10.2 confirm the branded-app upgrade's pricing structure (25 Aug 2026, Pass 4; corrected same day to remove an earlier flat-fee component that isn't part of the model) — a single metered per-active-student/month charge billed to the School, no flat fee — but the exact rate has not been set (you've mentioned \$0.99 and \$1.99 as reference points) — needs your decision.
- **Shared "ULTM8" app store approval** — the included, shared Student app (Section 5.2) needs to itself pass App Review as a legitimate aggregator/directory app, not a template shell — recommend confirming this positioning with Apple (e.g. via a pre-submission consultation) before the first branded-app upgrade goes live, since 4.2.6 enforcement is judged case-by-case.
- **BLOCKING — Apple App Store compliance risk in the two-tier white-label model (Section 5.2), flagged for developer pickup before build:** the shared/branded two-tier structure that Section 5 is built around — one "aggregator" app plus per-School branded rebuilds from the same codebase — is exactly the shape Apple's Guideline 4.2.6 and 4.3 (spam/template-farm) enforcement targets, and Apple has historically rejected new submissions and pulled already-approved apps in enforcement sweeps once a template-farm pattern is identified across many similar listings from one source. This is a real risk to whether the branded-app tier can ship or stay live at all, not a routine open item — every other decision in this pass (Section 5.4's credential model, Section 5.5's downgrade/cancellation handling, the Publishing Ops role) is downstream of this risk being resolved. Recommend a pre-submission consultation with Apple (mirroring the existing item above on the shared app's own aggregator-app positioning) and/or legal review before the publishing pipeline is built, not after the first branded app is submitted. Fallback, confirmed 25 Aug 2026: if this risk cannot be resolved, the branded-app tier is dropped from v1 entirely — every School ships on the shared app only (Section 5.2) — rather than shipping it unresolved.
- **White-label deep-link / Team ID domain strategy — not yet specified:** Section 5.1 lists a deep-link domain as one of the things baked into each School's branded binary at build time, but the document doesn't say whether each branded app gets its own domain (and Apple Associated Domains / Google Digital Asset Links entry) or shares one across all branded apps under a single Team ID — this affects both the build pipeline's parameterization (Section 5.4) and how a Universal Link/App Link resolves to the right School's app on a device with several branded apps installed. Needs a decision before the build pipeline is implemented.
- **General tenant/School cancellation & offboarding — not designed (new, 26 Aug 2026, Pass 6):** Section 5.5 specifies what happens to a School's branded (white-label) app on downgrade or full cancellation, but general account cancellation/offboarding for a School (or Franchise) — data retention, access shutoff timing, Student-facing consequences, Branch/Instructor RoleGrant handling, financial close-out — is not otherwise specified anywhere in this document. Flagged per Section 5.5 and Section 12.1 rather than assumed; this item was promised in both those places but never actually added here until this pass caught the gap. Data retention and deletion policy on exit (or on a GDPR/LGPD erasure request) is the same undesigned gap from a different angle, added Pass 7 — including what happens to R2-held signed waivers and invoice PDFs, since the Pass 5 backup/DR resolution (Section 11.6, Section 12.1) covered only RDS point-in-time recovery/Multi-AZ, not R2's own backup/versioning story.

- **Real-world-service vs. in-app-purchase classification — confirm before launch (new, 26 Aug 2026, Pass 6):** Section 10.3 argues every Stripe-charged purchase in this document — Membership billing, Franchise fees, the white-label add-on — is a real-world service, not a digital good, so it falls outside Apple's and Google's mandatory in-app-purchase requirement; but enforcement of that boundary has shifted over time and should be confirmed against each store's current guidelines before launch rather than assumed, the same treatment already given to the other Apple/Google compliance items above.
- **StudentRank readiness-bucket thresholds and progress-% formula — not actually defined (new, 26 Aug 2026, Pass 8):** Section 6.1 and Section 7 both reference a computed Ready/Almost Ready/Not Ready bucket and a progress % (powering the Grading Board, GET /students/{id}/eligibility), described only as "computed" — no threshold separates the buckets, and it's unstated whether progress % blends classes-attended and skill-sign-off ratios, or how weighted. This is a product call, not an engineering default — flagged rather than invented. Needs your input: either the actual thresholds/formula, or confirmation that Section 2.3's working prototype's own logic should be treated as the reference implementation here.
- **Video hosting/streaming infrastructure not yet chosen (new, 26 Aug 2026, Pass 8):** Decision 58 confirms the Lesson video feature and gives Lesson a videoRef field, but not what videoRef actually points at. Cloudflare R2, already confirmed for images/PDFs (Section 11.6), doesn't cover this — it's object storage, not a video-transcoding/adaptive-streaming/live-ingest service, and Prerecorded and Live Lessons need real-time ingest and playback infrastructure R2 alone doesn't provide. AWS IVS (or an equivalent AWS media service) is a natural candidate given the existing AWS-centred stack (Section 11.6), but that's an observation, not a decision — needs an explicit choice, including its cost model, since video storage and especially egress bandwidth scale very differently from the images/PDFs R2 was chosen for and could become a meaningful line item at any real usage volume. Blocks building the actual Lesson upload and live-session flows, not just their accessibility layer.
- **Audio description not resourced (new, 26 Aug 2026, Pass 8):** Decision 58's automated captioning pipeline satisfies WCAG 1.2.2 (Captions, Level A) for Lesson videos, and Decision 59 sets a WCAG 2.2 AA target platform-wide — but AA also includes 1.2.5 (Audio Description), which captioning does not satisfy: audio description is a narrated track describing visually-conveyed information (a technique's hand position, footwork) during natural pauses, not text of the spoken audio, and unlike captioning it isn't a mature, reliably-automatable technology today — it's typically either human-authored and recorded, or produced by a specialist vendor. No decision made here on how it gets resourced (in-house production as part of the instructor-upload flow, an outside vendor, or an explicit accepted-risk deferral with a target date) — flagged as its own open item so Decision 59's AA target isn't read as fully met for video content until this is actually resolved.
- **Guardian-specific step-up authentication not extended (new, 26 Aug 2026, Pass 8):** Decision 61's app-level re-lock deliberately applies one uniform threshold rather than a stricter tier for every role, reasoning that Owner/Manager and Franchise Owner's specific sensitive actions (payout/bank-detail changes, refunds) are already gated by Decision 12's step-up MFA regardless of re-lock state. That reasoning doesn't extend to Guardian: Decision 57 deliberately reuses the Guardian's ordinary OTP-verified login as its consent context rather than adding a heavier gate, so a Guardian's saved-payment-method purchases and ConsentRecord withdrawal for a linked minor have no step-up equivalent at all — the app-level re-lock is the only mechanism protecting them, not one layer among several the way it is for Owner-tier. Whether either of those two actions warrants its own step-up extension to Decision 12 — and if so, whether both or only the consent-withdrawal case, given what's actually at stake in each — is a real product call, not decided here rather than silently assumed as already covered. Full detail in Section 12.1 Decision 12, Section 12.1 Decision 57, Section 12.1 Decision 61, Section 12.1 Decision 69.
- **Trial-reissuance prevention mechanism genuinely unresolved (new, 26 Aug 2026, Pass 8):** Decision 65 added cheap, incentive-aligned visibility into the Trial-reissuance fee-dodge pattern (Section 9's dashboard-KPI-aggregation job, surfaced to Franchise and Platform Admin), but deliberately didn't build a precise prevention mechanism, because neither option available is clean: an HMAC-based dedup on email/phone strong enough to actually catch a recreated account survives deletion by design, which means it's pseudonymized personal data ULTM8 continues to hold specifically to re-identify someone who requested

erasure — a real tension with GDPR/LGPD expectations, not a settled legal question; the cheap alternative (the Decision 65 metric) is only a noisy proxy that doesn't distinguish this specific abuse from ordinary sales/conversion variance. Whether to eventually build the stronger mechanism depends on two things this review can't resolve on its own: real legal guidance on whether fraud-prevention-purpose HMAC retention is defensible against an erasure request (the same legal-review track already open for data residency and the money-transmission exemption), and a judgment call on whether the actual abuse rate justifies it once the Decision 65 metric has real data behind it rather than a hypothetical. Flagged rather than built around a guess. Full detail in Section 9, Section 12.1 Decision 44, Section 12.1 Decision 64, Section 12.1 Decision 65.

- **E-signature legal sufficiency for liability waivers, now sharpened by Decision 67 (new, 27 Aug 2026, Pass 8):** The WaiverSignature mechanism as designed is a typed full name plus a signature field — no drawn or biometric signature, no additional verification step (Section 2.2/6.1). Whether that satisfies the legal-sufficiency bar for a liability waiver covering physical martial-arts training has never been evaluated anywhere in this document, and waiver-validity requirements vary by jurisdiction across a platform already spanning at least 6 currency regions (Section 1.1/11.2). Decision 67 makes this sharper, not smaller: it ensures a minor's own self-signature can no longer satisfy the waiver-required check unassisted, routing that case to a Guardian instead — but a Guardian's typed-name signature on behalf of a linked minor faces exactly the same unevaluated legal-sufficiency question, just from the correct signer. Needs real legal review before this ships to any market, the same standard already applied elsewhere in this section (Franchise-disclosure-law, money-transmission exemption, data residency/GDPR/LGPD, Guardian minor age-thresholds).
- **No non-camera self-service check-in alternative (new, 27 Aug 2026, Pass 8):** Decision 69's camera-tier consent withdrawal, and the pre-existing accessibility case Decision 59 already grounds in it, both rely on the same Instructor/Staff-scoped override endpoint (PATCH /bookings/{id}/override, Section 7) as their only fallback to camera-based QR check-in — built and sized for occasional accessibility use, not as a standing per-class mechanism for a Student who can never use the primary scan flow going forward. A lightweight non-camera self-service alternative (e.g., a short entry code, mirroring the platform's own existing 6-digit-passcode login pattern) would serve both populations and reduce the resulting staff burden, but is new engineering surface not designed anywhere in this document. Flagged as a real product opportunity, not built around a guess. Full detail in Section 7, Section 12.1 Decision 59, Section 12.1 Decision 69.
- **No consent-management interface exists anywhere in the confirmed designs (new, 27 Aug 2026, Pass 8):** Decision 57 built ConsentRecord as an auditable, revocable consent trail, and Decision 69 makes its withdrawal behavior proportional to tier — but neither, nor anything else in this document, specifies where a Guardian actually exercises either action. No consent screen, consent-management flow, or withdrawal control appears anywhere in either Figma file, confirmed directly against this document's own text. Decision 57's original text and the Guardian-specific step-up-authentication item both describe what happens once a Guardian "withdraws consent" without ever specifying how that action is triggered — the same shape of gap Section 12.2 already caught once before for the School Portal's undesigned "QR Code" screen, just never connected to this feature until now. A Guardian currently has no confirmed way to exercise any consent right — full or tier-specific — at all. Needs a design decision, a Guardian-facing screen at minimum, before this ships to any market carrying children's-data-protection law, the same standard already applied to comparable items in this section. Full detail in Section 12.1 Decision 12, Section 12.1 Decision 57, Section 12.1 Decision 61, Section 12.1 Decision 69.
- **Automated captions aren't translated into the platform's other supported languages (new, 27 Aug 2026, Pass 8):** The Lesson captioning pipeline (Section 6.1, Section 7, Section 12.1 Decision 59) runs speech-to-text against the instructor's own spoken language and attaches that single-language caption track — it doesn't touch the platform's separate, already-built i18n layer (Section 11.1, Decision 8) that serves UI text in 4 languages (English UK, Portuguese BR, Spanish, Arabic) via i18next. A Student whose display language differs from the instructor's spoken language gets an accurately transcribed caption they may not be able to read. Nothing in the confirmed design connects the two systems — captions are not run through the CMS-backed translation layer, and whether they should be (machine-translating the caption track, vs. leaving this out of scope for launch) is an open product question, not a bug in either system considered alone. Flagged rather than resolved here, the same treatment as this section's other undesigned-vendor gaps. Full detail in Section 6.1, Section 7, Section 11.1, Section 12.1 Decision 8, Section 12.1 Decision 59.

- **A second, unexplained QR-scan endpoint exists alongside the self-service one (new, 27 Aug 2026, Pass 8):**
Section 7's InstructorsModule lists POST /classes/{id}/attendance-scan immediately beside PATCH /bookings/{id}/override — a second, distinct QR-scan endpoint alongside AttendanceModule's own POST /attendance/skan (the self-service Student-initiated scan every other decision in this document, including Decision 66's rotating-code fix and Decision 69's consent gating, is written against). No decision, open item, or Section 9 job description anywhere in this document references /classes/{id}/attendance-scan again after this one listing. Whether it's a leftover/duplicate of /attendance/skan, or an intentional separate Instructor-initiated scan path — e.g. staff manually recording attendance by scanning a Student's own credential rather than the Student scanning a displayed code — nothing in the confirmed designs says, and guessing risks building a second attendance path with none of Decision 57/66/69's consent and liveness protections applied to it. Needs confirmation with Code Studio before backend work starts on either endpoint. Full detail in Section 7, Section 12.1 Decision 66, Section 12.1 Decision 69.

13. Appendix — Verification Notes

This document was produced by: (1) connecting to both Figma files via the Figma MCP integration under the account "Code Studio"; (2) pulling the full node tree (get_metadata) for each file's single page; (3) enumerating all 541 (File 1) and 105 (File 2) top-level frames and de-duplicating to 448 and 94 uniquely-named frames respectively; (4) extracting the nested text-layer content of every screen listed in Section 2's tables directly from the design tree (verbatim field labels and sample values, not reconstructed or guessed); (5) cross-checking the sidebar/navigation item lists against the actual screen inventory to identify undesigned nav items (Section 12); (6) resolving four structural conflicts between the designs and the confirmed stack directly with Code Studio before writing Sections 3, 6–11; (7) a follow-up architecture discussion on 21 Aug 2026 that revised the App Boundary decision and added the Platform Admin isolation architecture (Section 4) and the payment-credential custody decision (Section 10.4); (8) a second follow-up discussion the same day that added the white-label mobile app distribution model (Section 5), covering entitlement, publish ownership, downgrade policy, and build timing; (9) a working, interactive prototype of the grading/ranks system, independently built and iteratively tested (automated regression, zero console errors, coach-facing flows screenshotted and reviewed) on 24 August 2026, validating the data model and flows now reflected in Section 2.3, Section 6.1 and Section 7.

Everything reported as fact in Sections 2–3 and Sections 6–11 traces to a specific Figma node inspected during this process; Section 4, Section 5, and the revised portions of Section 10.4 are engineering and product architecture Code Studio has adopted on ULM8's advice rather than something drawn in Figma, and are labeled as such throughout. Everything not directly shown in the designs and not explicitly confirmed by Code Studio is listed in Section 12 rather than assumed. Everything reported in the new Section 2.3 subsection and the resulting Section 6.1/Section 7 additions is derived from the working grading-system prototype rather than the Figma audit, and is labeled as such throughout. A follow-up clarification on 24 August 2026 additionally resolved the Class/Timetable relationship and the attendance-to-grading linkage (Section 12.1), confirmed directly with Code Studio rather than inferred.

Operational runbook (expanded 26 Aug 2026, Pass 8, Round 2): first-response notes for the failure modes this document has surfaced enough detail about to write a real entry for, grounded in the accepted risks and mitigations already decided elsewhere in this spec rather than generic on-call advice — pending the fuller documentation a live team will still need for anything this pass didn't surface.

- **Stuck Stripe-webhook queue — Section 9, Section 11.6:** Check Bull Board's stripe-webhook-processing queue depth and oldest-waiting-job age first — a backlog here means Memberships aren't activating on payment_intent.succeeded even though Stripe already has the money. Queue-level stats are open to any Platform Admin; the job-detail view requires declaring the tenant/investigation per Decision 53's audit-logged access.
- **RLS CI-gate failure — Section 4.8, Section 11.6:** Treat as a release-blocking signal, never override. It exists specifically to catch a cross-tenant leak before it ships, and a passing build with this check red should be impossible under the branch-protection rule — if one is seen, the branch-protection configuration itself is what to investigate, not the check.
- **Credential-purge job failure — Section 9, Decision 27:** Check whether it's a same-day-re-upgrade case the transactional guard is correctly skipping versus a genuine job error — the two look similar in a shallow log scan but require opposite responses: a skip needs no action, a genuine failure risks a School's stored Apple/Google credential outliving its 90-day grace period unpurged (Section 5.5).
- **Any other background-job backlog or repeated failure — Section 9, Decision 31, Decision 53:** Start with Bull Board's queue depth and oldest-waiting-job age and the failed-job/oldest-waiting-job CloudWatch alarms already wired to alert on this — the same first stop regardless of which of the 18 jobs is involved. Job-detail payloads for internally-triggered jobs carry only IDs, not raw data (Decision 53), so the record to actually inspect is in Postgres, not the queue; only stripe-webhook-processing and otp-delivery carry a raw external payload, and viewing either requires the same declared-tenant audit trail as the item above.
- **R2 outage blocking waiver/invoice viewing — Section 11.3, Decision 36:** This is an accepted, documented risk, not a gap with a built-in failover — Decision 36 accepted it against R2's own high published uptime

rather than engineering around it. First response is confirming it's actually an R2-side outage (Cloudflare's own status page) rather than an application bug; if confirmed, there is no mitigation to invoke, only Cloudflare's own resolution to wait on.

- **SMS-OTP delivery failure — Section 11.4, Decision 33, Decision 34:** Confirm the existing fallback actually fired before treating this as a full outage: a Twilio Verify failure should already be falling back to an email-delivered OTP through Postmark, or SES if Postmark itself is also down (Decision 34). This only becomes a genuine incident if both the primary and fallback paths failed together, not on a single Twilio error.
- **Stripe dispute arriving after a related refund already processed — Section 10.2, Decision 55:** This is an accepted risk, not a bug to fix: Decision 55's freeze only blocks future refund/credit actions from the moment a dispute's Disputed status lands, and can't retroactively undo a refund that already completed, since dispute timing is controlled by the cardholder's bank, not by ULTM8. Document the sequence rather than searching for a missed guard.
- **Platform Admin retains access briefly after revocation — Section 8.3:** A just-revoked Platform Admin session can remain valid for up to 5 minutes — an accepted, not eliminated, window on the live-permission-downgrade JWT TTL, not a security bug. Escalate only if access persists meaningfully past that window.

14. Glossary

Quick-lookup definitions for the entities, status enums, and platform terms used throughout this document. Each entry points to the section carrying the full, authoritative definition — this glossary is a shortcut into that section, not a replacement for it.

14.1 Core entities

- **User (base)** — The customer-facing identity record shared by every role — Student, School Owner-Manager, Branch Staff, Instructor, Franchise Owner, Guardian. A role is never a column on User; it's granted via RoleGrant. See Section 6.1.
- **RoleGrant** — The sole source of truth for which roles a User currently holds and at what tenant scope (school_id/branch_id/franchise_id, mapped per role by a database CHECK constraint). Sessions and JWTs are built from a User's currently-active (revoked_at IS NULL) grants. See Section 6.1.
- **School** — The primary tenant. Owns Branches, Instructors, Classes, MembershipPlans, and TimetableSlots; may optionally belong to a Franchise. See Section 6.1, Section 6.2.
- **Franchise** — An optional parent tenant that groups Schools together and bills them a franchise fee via FranchiseFeeCharge. Independent, non-franchised Schools are a first-class case, not an edge case. See Section 6.1, Section 6.2.
- **Branch** — A real, RLS-enforced sub-location under a School, keyed on branch_id for TimetableSlot, Class, Instructor, and Booking. Student and Membership stay School-scoped, not Branch-scoped. See Section 6.2.
- **PaymentAccount** — Holds a School's or Franchise's Stripe Connected Account reference — no raw secret keys are ever stored (Section 4.5, Section 10.4). See Section 6.1.
- **Transaction** — The per-charge ledger record for Student-to-School payments (Membership purchases). Carries a Successful/Pending/Failed/Refunded/Disputed status. See Section 6.1, Section 10.2.
- **FranchiseFeeCharge** — The School-to-Franchise ledger record for franchise-fee billing (Flat or Per-Headcount), carrying the same status enum as Transaction plus an active-student-count snapshot used for Per-Headcount billing and the Decision 54 Trial/Friend Pass exclusion. See Section 6.1, Section 10.2, Decision 54, Decision 55.
- **PlatformCharge** — ULTM8's own platform-level billing record (School/Franchise SubscriptionPlan fees, white-label app entitlement fees) — mirrors FranchiseFeeCharge's shape but bills ULTM8 itself, not a School→Franchise fee. See Section 6.1, Section 10.2, Decision 42.

- **AuditLogEntry** — An immutable record written on every Platform Admin action against tenant data; UPDATE/DELETE are revoked at the database layer for every role. `subjectUserId` is nullable, populated only when the target entity references one Student/Guardian/Instructor record. See Section 4.7, Decision 15, Decision 43.
- **ConsentRecord** — Guardian consent for a linked minor, recorded as a first-class, revocable, two-tier (baseline / camera-specific) record distinct from `WaiverSignature`. See Section 6.1, Decision 57, Decision 69.
- **DeviceToken** — Routes a push notification to the correct credential set — the shared Student app's or a specific School's white-label app's. See Section 6.1, Decision 46.
- **TenantAppConfig** — A School's white-label mobile app configuration (bundle id, theme tokens, store credentials) — created only when `whiteLabelApp` is entitled, credential retained for a 90-day grace period after downgrade. See Section 6.1, Section 5.3, Section 5.5.
- **Lesson** — A short instructional video tied to a specific Skill, in either Prerecorded or Live format, with its own caption pipeline (`captionStatus/captionTrackRef`). See Section 6.1, Decision 58.

14.2 Status values & role enums

- **RoleGrant.role** — Student / School Owner-Manager / Branch Staff / Instructor / Franchise Owner / Guardian — Platform Admin is excluded, a separate identity realm (Section 4.4). Corrected 27 Aug 2026 to match Section 8.2's actual six-role model. See Section 6.1.
- **Payment status enum (Transaction, FranchiseFeeCharge, PlatformCharge)** — Successful / Pending / Failed / Refunded / Disputed — Disputed added Decision 55. All three entities share this exact enum. See Section 6.1, Section 10.2.
- **School.franchiseFeeSubscriptionStatus** — Active / Past Due / Canceled — synced from the underlying Stripe Subscription via webhook; distinct from `FranchiseFeeCharge`'s own per-charge status above. See Section 6.1, Section 10.2.
- **ConsentRecord.tier** — baseline or camera-specific — baseline covers general account/attendance/grading data; camera-specific covers QR-check-in and profile-photo capture, withdrawable independently. See Section 6.1, Decision 57, Decision 69.

14.3 Acronyms & platform terms

- **RLS** — Row-Level Security — the PostgreSQL mechanism enforcing tenant isolation at the database layer, gated by a required GitHub Actions cross-tenant isolation test suite before merge. See Section 4.3, Section 4.8.
- **SAQ-A** — The lightest PCI-DSS self-assessment tier, achievable because ULTM8 never touches raw card/bank data — Stripe-hosted fields/redirect only, for both checkout and Connect onboarding. See Decision 20.
- **Bull Board / BullMQ** — BullMQ is the Redis-backed job queue library running all 18 of ULTM8's background jobs (Section 9); Bull Board is the admin-authenticated, read-only dashboard giving visibility into every queue's depth and failed/oldest-waiting-job state. See Section 11.6, Decision 31.
- **Connect / Destination charge** — Stripe Connect account models used to route funds directly into a School's or Franchise's own connected account rather than through ULTM8's platform balance. See Section 10.4.
- **IANA timezone** — The timezone-database identifier (e.g. "America/Sao_Paulo") stored per School; every date-only comparison resolves against it rather than server-local or UTC wall-clock time. See Section 6.1, Decision 25.
- **OIDC federation** — The mechanism by which GitHub Actions authenticates to AWS without static, long-lived IAM keys. See Section 11.6, Decision 18.
- **JWT TTL** — The lifetime of an issued access token; a just-revoked Platform Admin session can remain valid for up to 5 minutes on the live-permission-downgrade path — an accepted window, not a bug. See Section 12.1.

- **GDPR / LGPD** — The EU and Brazilian data-protection regimes governing this platform's markets; data-residency and retention-period compliance across all markets remains a still-open legal-review item. See Section 11.6, Section 12.2.